



Technische
Universität
Braunschweig

BACHELORARBEIT

Algorithmische Methoden zu gradbeschränkten Triangulierungen

Fabian Alich

5310894

Informatik

Institut für Betriebssysteme und Rechnerverbund
Abteilung Algorithmik

Erstprüfer

Prof. Dr. Sándor Fekete

Institut für Betriebssysteme und Rechnerverbund
Abteilung Algorithmik

Zweitprüfer

Prof. Dr. rer. nat. Roland Meyer

Institut für Theoretische Informatik

Betreuer

Dr. Phillip Keldenich

Michael Perk

1. September 2025

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Abschlussarbeit selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt. Die wörtliche oder sinngemäße Übernahme von Abschnitten aus Texten Dritter sowie aus eigenen vorangegangenen Veröffentlichungen habe ich kenntlich gemacht.

Ferner versichere ich, dass es sich hier um eine Originalarbeit handelt, die noch nicht in einer anderen Prüfung vorgelegen hat.

Braunschweig, 1. September 2025

Aufgabenstellung

Eine Triangulierung einer Punktmenge ist die Zerlegung ihrer konvexen Hülle in nicht überlappende Dreiecke. Triangulierungen spielen eine zentrale Rolle in der algorithmischen Geometrie und finden vielfältige Anwendungen, etwa in der Computergrafik und geographischen Informationssystemen (GIS). Im Rahmen seiner Bachelorarbeit soll Herr Alich das neuartige kombinatorische Optimierungsproblem Degree-Constrained Triangulations (DCT) untersuchen. Bei diesem Problem geht es darum, für eine gegebene Punktmenge eine Triangulierung zu finden, bei der – zusätzlich zu den klassischen Anforderungen – die Knotengrade (also die Anzahl der angrenzenden Dreiecke bzw. Kanten pro Punkt) gewissen Beschränkungen unterliegen. Ziel der Arbeit ist es, geeignete Verfahren zur Lösung dieses Problems zu entwickeln, zu implementieren und zu evaluieren. Dabei können neben exakten Verfahren (z.B. auf Basis von Integer Programming, Constraint Programming oder Branch-and-Bound) auch heuristische Ansätze (z.B. Greedy- oder Metaheuristiken) betrachtet werden. Von besonderem Interesse sind außerdem Methoden zur Fixierung oder zum Ausschluss bestimmter Kanten, um die Lösungsräume gezielt einzuschränken sowie die Generierung von leichten und schweren Instanzen für das Problem.

Braunschweig, 1. September 2025

Abstract

This thesis examines the problem of degree constrained triangulations (DCT) where each vertex in a triangulation must have a given degree. Several exact and approximate approaches were developed and implemented. These include edge based and triangle based encodings. In addition, strategies for fixing and excluding edges and systematic methods for generating instances for feasible and infeasible cases were introduced.

The experimental evaluation presents a comparison of different solvers such as PySAT, CaDiCaL, OR-Tools and Gurobi as well as various modeling techniques with respect to runtime and success rate. The results show that OR-Tools achieves the best results overall. Optimization based formulations provide approximate solutions in infeasible or more complex cases.

The thesis shows that degree constrained triangulations can be effectively modeled and solved for small input sizes. OR-Tool with an edge based encodings proves to be the most effective solver.

Zusammenfassung

Diese Arbeit behandelt das Problem der gradbeschränkten Triangulationen (Degree Constrained Triangulations, DCT), bei denen jedem Knoten einer Triangulation ein vorgegebener Grad zugeordnet wird. Hierfür wurden exakte und approximative Lösungsansätze entwickelt und implementiert. Dazu gehören kantenbasierte und dreiecksbasierte Codierungen. Zusätzlich wurden Strategien zum Fixieren und Ausschließen von Kanten sowie systematische Verfahren zur Instanzgenerierung für lösbare und unlösbare Fälle entworfen.

Die experimentelle Untersuchung umfasst einen Vergleich verschiedener Solver wie PySAT, CaDiCaL, OR-Tools und Gurobi sowie unterschiedlicher Modellierungstechniken im Hinblick auf Laufzeit und Erfolgsquote. Die Ergebnisse zeigen, dass OR-Tools insgesamt die besten Resultate liefert. Optimierungsbasierte Formulierungen liefern in unlösbaren oder komplexeren Fällen approximative Lösungen.

Die Arbeit zeigt, dass gradbeschränkte Triangulationen für kleine Eingabegrößen effizient modelliert und gelöst werden können. OR-Tools mit der kantenbasierten Codierungen erweist sich dabei als leistungsfähigster Ansatz.

Inhaltsverzeichnis

1. Einleitung	1
1.1. Das Problem	2
1.2. Ergebnisse dieser Arbeit	3
1.3. Verwandte Arbeiten	3
1.3.1. Delaunay Triangulierung	3
1.3.2. Kantenflips in Triangulations	4
1.3.3. Degree constrained minimum spanning tree	5
2. Definitionen	9
3. Lösungsansätze	13
3.1. Entscheidungsproblem	13
3.1.1. Kantenansatz	13
3.1.2. Dreiecksansatz	20
3.2. Optimierungsproblem	21
3.3. Conflict-driven clause learning	23
3.3.1. Unit-Klauseln	24
3.3.2. Konflikte	24
4. Instanzgenerierung	27
4.1. Ja - Instanzen	27
4.1.1. Delaunay	28
4.1.2. Delaunay mit zufälligen Flips	28
4.1.3. Zufällige Kantenerzeugung	28
4.1.4. Iteration	28
4.1.5. Greedy	29
4.2. Nein-Instanzen	29
4.3. Theorie mehrerer Lösungen	30
5. Solver	31
5.1. Generelle Implementierungen	31
5.2. PySAT	32
5.3. OR-Tools	32
5.4. Gurobi	34
5.5. Simplified CDCL Solve	34

6. Auswertung	35
6.1. Versuchsumgebung	35
6.2. Permutation	36
6.3. Mehrmalige Durchläufe	36
6.4. SAT Gradbeschränkung	36
6.5. SAT-Solver Vergleich	37
6.6. Vergleich der Modellierungstechniken	39
6.7. Kanten vorab berechnen	40
6.8. Größerer Timeout	41
6.9. Ja/Nein getrennt	42
6.10. Zielfunktion	42
6.11. <i>Propagation</i> CDCL	43
6.11.1. Visualisierung	43
6.11.2. Relative Anzahl erfolgreicher <i>Propagation</i>	44
6.12. Schwierigkeit der Instanzen	46
6.12.1. Anzahl Lösungen	46
6.12.2. Gradverteilung	46
6.12.3. Verteilung der Kantenlängen	47
6.12.4. Zusammenfassung	48
7. Fazit und Ausblick	49
A. Anhang	53
A.1. Permutation	53
A.2. Mehrmalige Durchläufe	56
A.3. SAT Grad Auswertung	60
A.4. Vergleich der Modellierungstechniken	61
A.5. Theoretische Fixierung von Kanten	66
A.6. Größerer Timeout	67
A.7. Ja/Nein getrennt	68
A.8. Zielfunktion	69
A.9. Anzahl Lösungen	70

Abbildungsverzeichnis

1.1. Eine Ja-Instanz und Nein-Instanz.	2
1.2. Delauney Beispiel.	4
1.3. Beispiel für <i>edge move</i> , <i>edge rotation</i> und <i>edge slide</i> aus [5, p. 70].	5
2.1. Ein Kantenflip in einer Triangulation.	10
3.1. Gesonderte Betrachtung von Knoten auf der Hülle mit Grad zwei.	14
3.2. Zwei Beispielinstanzen an denen Kanten ausgeschlossen werden können.	15
3.3. Kantenausschluss durch Schnittkanten.	16
3.4. Ein Vollständiger Graph mit allen möglichen Triangulierung.	16
3.5. Ein Baum, der die möglichen Entscheidungen eines SAT-Solvers darstellt.	24
4.1. Visualisierung der fünf verschiedenen Instanztypen.	27
4.2. Eine Ja-Instanz mit zwei möglichen Lösungen.	30
6.1. Vergleich von Kardinalitätscodierungen für SAT-Solver.	37
6.2. Vergleich verschiedener SAT-Solver.	38
6.3. Vergleich aller Solver.	39
6.4. Theoretische Ausschließung von Kanten.	41
6.5. Eine Nein-Instanz als Optimierungsproblem betrachten.	43
6.6. Eine Teilinstanz in einem CDCL Solver.	44
6.7. Anzahl erfolgreicher <i>Propagations</i>	45
6.8. Zwei verschiedene Lösungen für eine Instanz.	46
6.9. Gradverteilung der Knoten pro Instanz.	47
6.10. Verteilung der Kantenlängen pro Instanz.	48
A.1. Vergleich mehrfacher Durchläufe mit verschiedenen Permutationen für Gurobi.	53
A.2. Vergleich mehrfacher Durchläufe mit verschiedenen Permutationen für OR-Tools.	54
A.3. Vergleich mehrfacher Durchläufe mit verschiedenen Permutationen für PySAT.	55
A.4. Vergleich mehrfacher Durchläufe mit gleichen Bedingungen für Gurobi.	56
A.5. Vergleich mehrfacher Durchläufe mit gleichen Bedingungen für OR-Tools.	57
A.6. Vergleich mehrfacher Durchläufe mit gleichen Bedingungen für PySAT.	58
A.7. Vergleich von Kardinalitätscodierungen für eine KNF.	60
A.8. Vergleich des Dreiecksansatzes.	61
A.9. Die besten Bedingungen für PySAT finden.	62

Abbildungsverzeichnis

A.10.Die besten Bedingungen für OR-Tools finden.	63
A.11.Die besten Bedingungen für Gurobi finden.	64
A.12.Solverzeiten ohne Vorbereitungszeiten.	65
A.13.Untersuchung der theoretischen Fixierung von Kanten.	66
A.14.Experiment mit Zeitlimiterhöhung von 5 auf 30 Minuten.	67
A.15.Untersuchung, ob Solver bei Ja-Instanzen oder bei Nein-Instanzen besser sind.	68
A.16.Eine Ja-Instanz als Optimierungsproblem betrachten.	69

Tabellenverzeichnis

A.1. Anzahl der Lösungen pro Instanz.	70
---	----

1. Einleitung

Triangulierungen stellen einen zentralen Bestandteil der algorithmischen Geometrie dar. Unter einer Triangulierung versteht man die Zerlegung der konvexen Hülle einer gegebenen Knotenmenge in Dreiecke. In dieser Arbeit wird dieses klassische Problem um eine zusätzliche Bedingung erweitert, indem für jeden Knoten ein Sollgrad vorgegeben wird.

Die gradbeschränkte Triangulierung (Degree Constrained Triangulations, DCT) untersucht, ob es für eine gegebene Knotenmenge eine Triangulierung gibt, bei der jeder Knoten genau einen vorgegebenen Grad besitzt. Das bedeutet, dass für jeden Knoten eine bestimmte Anzahl inzidenter Kanten festgelegt ist. Diese Erweiterung ist nicht nur aus theoretischer Sicht interessant, sondern eröffnet auch praktische Anwendungsmöglichkeiten. Beispielsweise kann in Sensornetzwerken die Verbindungsdichte zwischen Knoten gezielt gesteuert werden. Auch in logistischen Anwendungen lassen sich so Kapazitätsbeschränkungen an Knoten modellieren.

Das Ziel dieser Arbeit ist die Untersuchung verschiedener Verfahren zur Lösung des DCT. Neben der Formulierung als Entscheidungsproblem, für das Methoden wie Integer Programming, Constraint Programming oder SAT-Solver eingesetzt werden, wird das DCT auch als Optimierungsproblem betrachtet. Ein besonderer Schwerpunkt liegt auf Strategien zum Fixieren und Ausschließen von Kanten, um die Suche nach Lösungen zu beschleunigen. Darüber hinaus werden Verfahren zur systematischen Instanzgenerierung entwickelt und beschrieben.

Die Arbeit gliedert sich in sechs Kapitel. Zunächst werden im ersten Kapitel das betrachtete Problem sowie verwandte Probleme erklärt. Im zweiten Kapitel werden grundlegende Definitionen sowie theoretische Grundlagen vorgestellt. Darauf aufbauend werden unterschiedliche Lösungsansätze präsentiert und ihre jeweilige Implementierung beschrieben. Diese umfassen sowohl ein Kanten- als auch ein Dreiecksmodell, die jeweils in Form mathematischer Modelle sowie in konjunktiver Normalform (KNF) umgesetzt werden. Zusätzlich wird ein SAT-Solver durch geeignete Optimierungen an das Problem angepasst. Mit der Implementierung und den erzeugten Instanzen erfolgt anschließend eine Evaluation der verschiedenen Ansätze. Der Fokus liegt dabei auf der Laufzeitanalyse sowie der Findung des besten Lösungsansatzes. Abschließend werden die generierten Instanzen hinsichtlich ihrer strukturellen Eigenschaften und der damit verbundenen Schwierigkeit untersucht.

1. Einleitung

1.1. Das Problem

Das in dieser Arbeit betrachtete Problem sind gradbeschränkte Triangulierungen. Es wird untersucht, ob für eine gegebene Knotenmenge eine Triangulierung existiert, bei der jeder Knoten einen vorgegebenen Grad besitzt. Das Problem wurde erstmalig von Joseph S. B. Mitchell genannt. Dieser fragte nach der Komplexität dieses Problems.

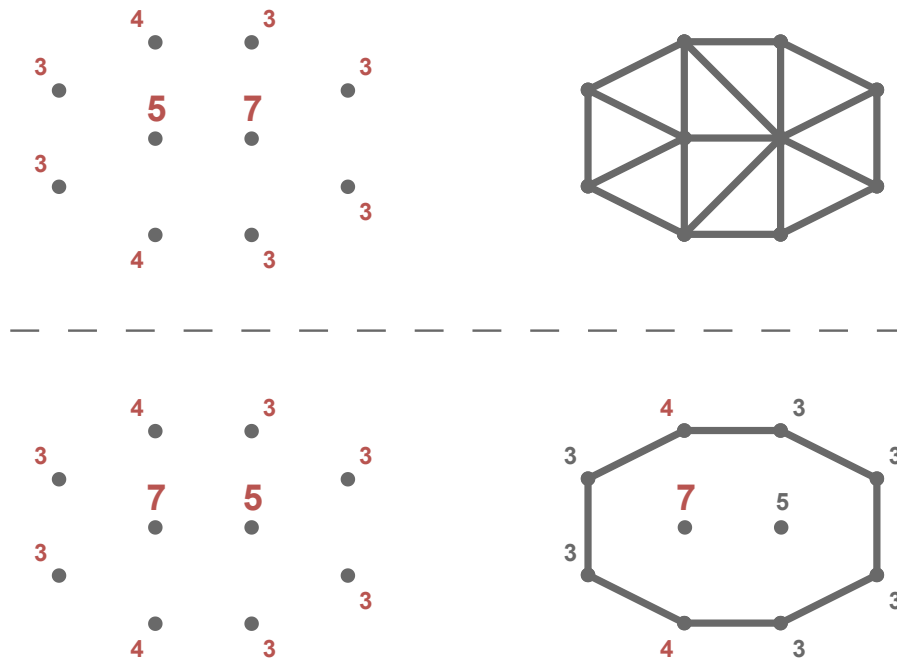


Abbildung 1.1.: Eine Ja-Instanz (oben) und eine Nein-Instanz (unten). Die Zahlen an den Knoten sind die Sollgrade. Bei der Nein-Instanz können die Sollgrade mit einer Triangulierung nicht eingehalten werden.

In Abbildung 1.1 sind zwei Instanzen mit Lösungen dargestellt. Eine Instanz ist hierbei eine Knotenmenge mit vorgegebenen Graden. Die Zahlen an den Knoten geben deren Grad an. Diese werden als Sollgrad bezeichnet. Die obere Instanz ist eine Ja-Instanz, da die Sollgrade mit der Triangulierung eingehalten werden können. Bei der unteren Instanz handelt es sich um eine Nein-Instanz, da mit den gegebenen Sollgraden keine Triangulierung möglich ist.

Eine praktische Anwendung dieses Problems könnte zum Beispiel in der Computergrafik, genauer in der Erzeugung von Netzen, für die Modellierung von Oberflächen genutzt werden. Durch den Sollgrad lässt sich die Dichte auf der Oberfläche in bestimmten Bereichen gezielt steuern. Analog kann in der Nachrichtentechnik ein DCT verwendet werden, um die Verbindungsichte zwischen Knoten in einem Sensornetzwerk zu regulieren. Auch beim Transport von Gütern kann DCT genutzt werden, wobei der Sollgrad angibt, wie viele Güter an jedem Ort vorhanden sind und weiterbewegt werden sollen.

1.2. Ergebnisse dieser Arbeit

In dieser Arbeit wurde das DCT praktisch untersucht. Die wichtigsten erzielten Ergebnisse lassen sich wie folgt zusammenfassen:

- Es konnte gezeigt werden das Ja- und Nein-Instanzen des DCT bis zu einer Knotengröße von 220 Gelöst werden konnten.
- Wird das DCT als Optimierungsproblem formuliert, konnte eine Nein-Instanzen zu 97 % gelöst werden.
- Für einen SAT-Solver wurde eine spezielle *Propagation* implementiert, die bei einer Instanz in 3 % der Aufrufe erfolgreich eine Kante ausschließen konnte.
- Die Analyse der Instanzen zeigt, dass sowohl die Gradverteilungen als auch die Verteilungen der Kantenlängen Hinweise auf die Schwierigkeit einer Instanz geben können.

1.3. Verwandte Arbeiten

In diesem Abschnitt werden verschiedene, wissenschaftliche Arbeiten aus dem Themenbereich betrachtet, um einen Überblick über den aktuellen Entwicklungsstand zu geben. Es werden drei Bereiche betrachtet. Der erste Bereich behandelt die *Delaunay Triangulierung*. Diese zählt zu den bekanntesten Triangulationen und bietet einen grundlegenden Einblick in das Thema. Der zweite Bereich umfasst *Flips in Triangulationen*. Hier wird untersucht, wie sich Triangulierungen durch lokale Änderungen gezielt modifizieren lassen. Der dritte Bereich befasst sich mit dem Problem des *degree constrained minimum spanning tree*. Dabei werden insbesondere Ansätze zur Einhaltung von Gradbeschränkungen betrachtet.

Erwähnenswert ist, dass Sharir et al. [19] den minimalen und maximalen Grad in einer Triangulierung untersucht haben. Darüber hinaus haben Datta et al. [6] und Gewali et al. [11] sich mit Triangulierungen beschäftigt, bei denen jeder Knoten einen geraden Grad besitzt. Da diese beiden Themenbereiche keinen direkten Mehrwert für die vorliegende Arbeit bieten, werden sie hier lediglich namentlich erwähnt.

1.3.1. Delaunay Triangulierung

Als Grundlage für den Algorithmus kann die *Delaunay-Triangulierung* betrachtet werden. Diese liefert eine Triangulierung, die möglichst gleichschenklige und gleichwinklige Dreiecke verwendet [17]. Dies wird erreicht, indem der minimale Winkel maximiert wird. Berg et al. [8] behandeln diese Variante der Triangulierung ausführlich.

Triangulierungen eignen sich durch ihre Struktur besonders gut zur Darstellung von Höhendaten im zweidimensionalen Raum (siehe Abbildung 1.2). Dabei können die unterschiedlichen Innenwinkel und Kantenlängen der verwendeten Dreiecke genutzt werden.

1. Einleitung

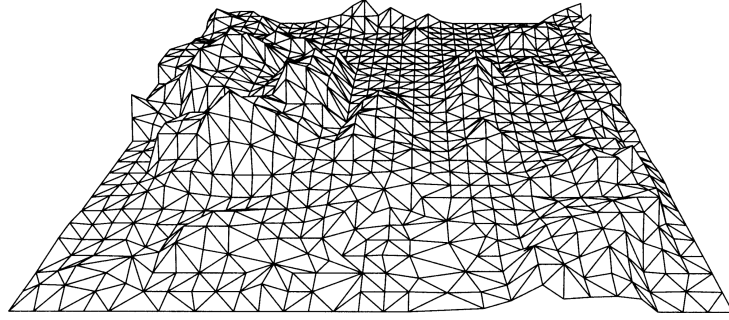


Abbildung 1.2.: Ein Beispiel für die Darstellung von Höhendaten durch eine Triangulierung aus [8, p. 183].

Das Grundproblem besteht darin, dass die Höhendaten nur an bestimmten Knoten bekannt sind und die restlichen Werte daher geschätzt werden müssen. Dies kann durch die Delaunay-Triangulierung gut gelöst werden, da die Dreiecke aufgrund ihrer Gleichschenkligkeit geeignet sind, die fehlenden Daten zu ergänzen.

Berg et al. geben zudem Formeln für die Anzahl der Kanten $|E|$ und Dreiecke $|F|$ in einer Triangulierung an. Für eine Knotenmenge mit n Knoten und k Knoten auf der konvexen Hülle gilt

$$|E| = 3n - 3 - k$$

$$|F| = 2n - 2 - k.$$

1.3.2. Kantenflips in Triangulations

Kantenflips, kurz Flips genannt, sind Möglichkeiten, um Kanten in einer Triangulierung auszutauschen. Triangulierungen besitzen per Definition eine feste Anzahl an Kanten [8]. Aus diesem Grund können keine Kanten hinzugefügt oder entfernt werden. Daraus folgt, dass für jede entfernte Kante eine andere Kante eingefügt werden muss. Eine besondere Art dieser Umstrukturierung von Kanten sind sogenannte Flips, diese werden im folgenden noch genauer erklärt. Hurtado et al. [12] zeigten, dass jede Triangulierung mindestens

$$\frac{n - 4}{2}$$

flippbare Kanten besitzt, wobei n die Anzahl der Knoten bezeichnet. Laut der wissenschaftlichen Arbeit *Flips in planar graphs* [5] lässt sich jede mögliche Triangulierung in jede beliebige andere Triangulierung mit Flips überführen. Ist somit eine DCT möglich, so kann diese durch Flips konstruiert werden. Dieser Ansatz wird in Abschnitt 3.2 nochmal näher betrachtet. In Triangulierungen gibt es nur eine Art von Flips, die sogenannten diagonalen Flips. Bei diesen wird ein konvexes Viereck betrachtet und die vorhandene Diagonale durch die andere Diagonale ersetzt, indem die beiden nicht verbundenen Knoten nun verbunden werden. In Definition 2.6 wird genauer auf diesen Prozess eingegangen.

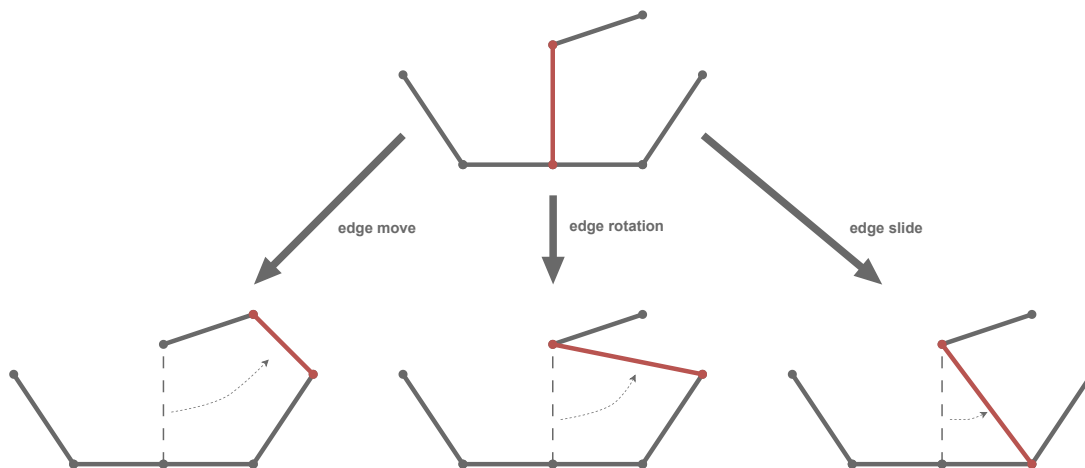


Abbildung 1.3.: Beispiel für *edge move*, *edge rotation* und *edge slide* aus [5, p. 70].

Wenn man Flips außerhalb von Triangulierungen betrachtet, gibt es verschiedene Arten von Flips. Drei beispielhafte Flips sind in Abbildung 1.3 dargestellt. Zum einen der *edge move*, hierbei handelt es sich um das einfache Entfernen und anschließende Einfügen einer Kante. Etwas stärker eingeschränkt ist die *edge rotation*, bei der ein Endknoten der Kante unverändert bleibt. Es erfolgt eine Rotation um einen Knoten. Die dritte Variante, der *edge slide*, schränkt die *edge rotation* weiter ein. Hier darf der veränderte Knoten nur ein Nachbar des ursprünglich veränderten Knotens sein.

Eine weitere Unterteilung von Flips sind die sog. *k*-Flips. Diese zeichnen sich durch eine Gruppierung der Anzahl der veränderten Kanten aus. Dabei werden *k* Kanten entfernt und durch *k* neue Kanten ersetzt. Die bisher genannten Flips entsprechen somit 1-Flips.

1.3.3. Degree constrained minimum spanning tree

Eine Gradbeschränkung gibt es auch beim Minimum Spanning Tree. Hier hat jede Kante ein vorgegebenes Gewicht, welches im Spanning Tree minimiert werden soll. Das Besondere am *degree constrained minimum spanning tree* ist, dass ein *minimum spanning tree* erzeugt werden soll, welcher an jedem Knoten einen maximalen Grad von *d* hat. Dabei wird *d* global für das Problem vorgegeben.

Es wurden verschiedene Ansätze von Joshua Knowles und David Corne [13] vorgestellt, die das Problem lösen sollen. Der erste Ansatz ignoriert vorerst den Aspekt der Minimalität des Problems. Es werden Kanten hinzugefügt, welche die Gradbeschränkung nicht verletzen. Im weiteren Verlauf wird versucht, die Anzahl der Kanten zu minimieren.

Ein weiterer Ansatz nutzt aus, dass der minimale Spannbaum ohne Gradbeschränkung in polynomialer Zeit gelöst werden kann. Dieser Ansatz erzeugt zunächst einen minimalen Spannbaum, welcher die Gradbeschränkung ignoriert. Wenn eine Lösung gefunden wurde, werden den Kanten neue Gewichte zugeteilt. Diese neuen Gewichte werden mit folgender

1. Einleitung

Formel berechnet,

$$weight + fault \cdot max \cdot \frac{(weight - min)}{(max - min)}$$

wobei *weight* das aktuelle Gewicht der Kante ist. Die Variable *fault* hat den Wert, $\{0, 1, 2\}$ abhängig davon, wie viele Knoten an der Kante gerade die Gradbeschränkung verletzen. Die Variablen *min* und *max* bezeichnen das kleinste, beziehungsweise größte Gewicht aller Kanten. Danach wird erneut ein minimaler Spannbaum gebildet, wobei durch das neue Gewicht nun Kanten bevorzugt werden, die keine Gradbeschränkung verletzen. Diese beiden Schritte können so lange wiederholt werden, bis eine valide Lösung gefunden wurde oder eine maximale Anzahl an Iterationen erreicht wurde. Der Nachteil bei diesem Ansatz ist, dass Lösungen aufgrund der maximalen Iterationen fälschlicherweise als *nicht möglich* bewertet werden können.

Ein weiterer Ansatz wird als *Hillclimbing* bezeichnet. Dieser löst den Nachteil des *degree constrained minimum spanning tree* nicht, wird jedoch zur Suche nach neuen Knoten in einem anderen Algorithmus verwendet. In diesem Ansatz werden lokal neue Knoten gesucht, die bessere Ergebnisse liefern. Dies könnte auf DCT angewendet werden, um neue Kanten zu finden. Der Ansatz erzeugt keine eigene Lösung, sondern verbessert eine Bestehende. Zunächst wird ein zufälliger Knoten aus einer Lösung ausgewählt, woraufhin weitere mögliche Knoten in der Nähe des ursprünglichen Knotens gesucht werden. Bei diesen Knoten wird überprüft, ob ein Knotentausch gleich gute oder bessere Ergebnisse liefert. Falls dies der Fall ist, wird der Knoten ausgetauscht, andernfalls wird der gefundene Knoten ignoriert. Dieser Ansatz soll lokal gute Ergebnisse liefern, bringt jedoch aufgrund seines greedy-Verhaltens keine globalen Verbesserungen. Eine mögliche Verbesserung der lokalen Problematik stellt *multistart hillclimbing* dar. Bei diesem Verfahren wird nur ein Knoten in der Nähe getestet. Ist dieser nicht gleich oder besser, wird ein neuer zufälliger Knoten ausgewählt.

Der nächste Ansatz wird *simulated annealing* genannt. Auch hier werden neue Knoten gesucht, um bestehende Lösungen zu verbessern. Das Ziel des Ansatzes ist es, das Problem mit einer globalen Verbesserung zu lösen, indem zu Beginn viele Veränderungen zugelassen werden und im Verlauf immer genauere Verbesserungen erfolgen. Auch hier wird ein zufälliger Knoten ausgewählt und es werden weitere mögliche Knoten in der Nähe gesucht. Bei diesem Ansatz werden jedoch neue Knoten gemäß der folgenden Formel übernommen:

$$p\{\text{accept } j\} = \begin{cases} 1 & \text{if } f(j) \leq f(i) \\ \exp\left(\frac{f(i)-f(j)}{c}\right) & \text{sonst} \end{cases}$$

Dabei beschreibt $p\{\text{accept } j\}$ die Wahrscheinlichkeit, dass ein Knoten übernommen wird. Die aktuelle Lösung ist *i* und die mögliche neue Lösung ist *j*. Die Funktion *f* gibt hierbei die Kosten der Lösung an. Der Parameter *c* wird zu Beginn sehr hoch gesetzt und mit der Zeit verringert. Mit *c* kann gesteuert werden, wie aggressiv der Ansatz ist. Also wie viel Veränderung er erzeugt. Der Ansatz endet, wenn ein finales c_f erreicht wird, also wenn $c = c_f$. Dieser Ansatz beschreibt das übliche Verfahren eines Evolutionsalgorithmus. Dabei werden verschiedene Lösungen erzeugt und bewertet.

Die besten Lösungen werden ausgewählt und auf Basis dieser Lösungen werden durch Mutation neue Lösungen generiert.

Krishnamoorthy et.al. [15] haben Evolutionsalgorithmen genauer betrachtet. Der erste Schritt ist eine Codierung zu wählen, die nur zulässige Spannbäume darstellen kann. Andernfalls könnten von den erstellten Lösungen viele nicht genutzt werden und diese wären somit nicht relevant. Wenn eine passende Codierung gewählt wurde, müssen dann nur noch der Grad und das Gewicht betrachtet werden.

Eine Möglichkeit wie Mutationen gebildet werden können, ist die *Crossover* Methode. Dabei existieren zwei *Eltern* Lösungen P_1 und P_2 . Aus diesen werden $2 \cdot (n-2)$ verschiedene *Kinder* erzeugt, da es $n-1$ Veränderungsmöglichkeiten gibt. Anschließend werden die *Kinder* bewertet und sortiert. Das beste *Kind*, das P_1 am ähnlichsten ist, wird ausgewählt. Ist dieses *Kind* besser als P_1 , ersetzt es P_1 . Für P_2 wird ein *Kind* nach dem gleichen Verfahren ausgewählt.

Eine weitere Möglichkeit ist die *Breeding* Methode. Dabei existiert ein Pool aus Lösungen. Zufällig werden Paare gebildet, die jeweils zwei *Kinder* erzeugen. Von diesen *Kindern* ersetzt das bessere *Kind* das schlechtere *Elternteil*. Die Größe des Pools kann so angepasst werden, dass alle möglichen Veränderungen berücksichtigt werden.

Beide Methoden sollen im Durchschnitt alle 10 Generationen die beste Lösung mit allen anderen Lösungen verglichen haben. Als mögliche Iterationsanzahl wird,

$$\frac{50 \cdot n}{\bar{b}}$$

verwendet, wobei mit $\bar{b}$ die durchschnittliche Gradbeschränkung bezeichnet wird. Diese Formel sorgt dafür, dass große Instanzen mehr Iterationen erhalten, während leichtere Instanzen (mit höherem Grad) schneller bearbeitet werden.

Sandor P. Fekete et al. [10] betrachten ebenfalls das *degree constrained minimum spanning tree* problem. Hierbei wird ein *Flow based Algorithmus* verwendet. Da dieser Ansatz nicht gut auf Triangulations übertragbar ist, wird er hier nur kurz behandelt.

Im praktischen Teil des Papers werden zwei Algorithmen vorgestellt. Diese erhalten einen minimalen Spannbaum und verändern ihn so, dass er eine Gradbeschränkung einhält. Dabei wird darauf geachtet, dass sich das Gewicht des Spannbaums nicht stark verschlechtert. Für diese Algorithmen werden Hauptoperationen namens *Adoptions* eingeführt. Diese funktionieren ähnlich wie Flips, nur in Spannbäumen. Es handelt sich also um Operationen, die eine zulässige Veränderung bewirken. Die beiden Algorithmen liefern eine Liste von Adoptionen, welche die gewünschte Änderung erzeugen. Der erste Algorithmus nutzt eine fraktionale Lösung und eine greedy Strategie, um die Adoptionen zu finden. Der andere Algorithmus beschränkt die betrachteten Kanten auf jene des gegebenen Spannbaums. So können die Algorithmen in polynomialer Zeit eine Lösung finden, die eine Gradbeschränkung einhält.

Eine Abwandlung des minimalen Spannbaums mit Gradbeschränkung wurde von Ana Maria de Almeida et al. [7] untersucht. Dabei erweitern sie das Problem um eine Funktion, welche für jeden Knoten einen Mindestgrad vorgibt. Dieses Problem wird als *Min-Degree Constrained Minimum Spanning Tree* bezeichnet.

1. Einleitung

Das *k-Dimensional Matching Problem* wurde verwendet, um zu zeigen, dass das *Min-Degree Constrained Minimum Spanning Tree* Problem NP-schwer ist. Es ist zu erwähnen, dass dies in der dritten Dimension von Ana Maria de Almeida et al. nicht gezeigt wurde. Allerdings soll das Problem schwieriger sein als das *Degree Constrained Minimum Spanning Tree*, obwohl auch dieses Problem NP-schwer ist.

Auch in diesem Fall wird das Problem mit einer *Flow based Formulierung* betrachtet. Dabei werden zwei Formulierungen angegeben. Eine gerichtete und eine ungerichtete Formulierung, wobei die gerichtete Formulierung hierbei effizientere Ergebnisse liefern soll. Die Formulierungen wurden mit einem *Branch and Bound* Algorithmus getestet. Dabei wurde festgestellt, dass bei der gerichteten Formulierung relevant ist, welcher Knoten als Wurzelknoten ausgewählt wird. Es konnte jedoch kein einheitlich optimaler Wurzelknoten für die Formulierungen festgestellt werden. Des Weiteren wurde häufig festgestellt, dass die Relaxierung keine guten Schranken lieferte. Dies konnte damit erklärt werden, dass die zugehörige Formel für den minimalen Spannbaum ohne Gradbeschränkung gleich gute Ergebnisse lieferte.

2. Definitionen

In diesem Kapitel werden die Definitionen vorgestellt, die für diese Arbeit benötigt werden. Dabei werden insbesondere grundlegende Begriffe erläutert, die in den folgenden Kapiteln verwendet werden. Zunächst werden Überschneidungen zwischen Kanten betrachtet.

Definition 2.1 (Überschneidung). Zwei Kanten gelten als überschneidend, wenn sie sich in der Ebene kreuzen. Im Folgenden bezeichnet die Funktion $I : (\mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2 \times \mathbb{Q}^2)$ die Menge aller Kanten, die eine gegebene Kante schneiden.

Im nächsten Schritt wird die Überschneidung von Dreiecken definiert, da diese in späteren Ansätzen relevant werden.

Definition 2.2 (Überschneidung Dreiecke). Zwei Dreiecke gelten als überschneidend, wenn sie mindestens einen inneren Knoten gemeinsam haben. Das bedeutet, dass sich zwei Außenkanten kreuzen und sich nicht nur berühren dürfen. Im Folgenden bezeichnet die Funktion $I : (\mathbb{Q}^2 \times \mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2 \times \mathbb{Q}^2 \times \mathbb{Q}^2)$ die Menge aller Dreiecke, die ein gegebenes Dreieck schneiden.

Auf Grundlage der Überschneidungen können nun die grundlegenden Begriffe der Triangulierung und DCT definiert werden.

Definition 2.3 (Triangulierung). Eine Triangulierung einer Knotenmenge $P \subset \mathbb{Q}^2$ ist ein maximaler kreuzungsfreier Graph auf P . Das bedeutet, es können keine weiteren Kanten hinzugefügt werden, ohne dass sich Kanten schneiden.

Für DCT wird zuerst der Grad eines Knotens definiert.

Definition 2.4 (Grad). Der Grad eines Knotens ist die Anzahl der anliegenden Kanten. Im Folgenden wird er durch die Funktion $\deg : \mathbb{Q}^2 \rightarrow \mathbb{N}$ angegeben.

Basierend auf dem Grad eines Knotens wird nun DCT eingeführt.

Definition 2.5 (Gradbeschränkte Triangulierung). Die gradbeschränkte Triangulierung (DCT) hat die gleichen Voraussetzungen wie eine Triangulierung. Zusätzlich gibt es eine Funktion $\deg^* : \mathbb{Q}^2 \rightarrow \mathbb{N}$. Diese gibt den Grad an, den ein Knoten haben muss.

Im Folgenden wird die Durchführung des Kantenflips beschrieben, die eine wichtige Rolle bei der Umstrukturierung von Triangulierungen spielt.

Definition 2.6 (Kantenflip). Ein Kantenflip oder kurz Flip ist eine Operation, bei der eine Kante entfernt und eine andere hinzugefügt wird, so dass die Eigenschaften einer Triangulierung erhalten bleiben. In dieser Arbeit wird ausschließlich mit Triangulierungen

2. Definitionen

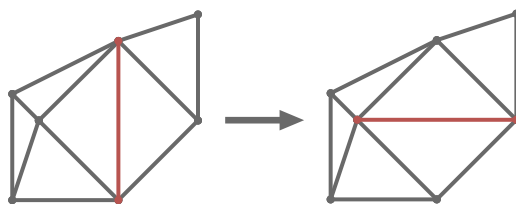


Abbildung 2.1.: Ein Kantenflip in einer Triangulation.

gearbeitet, dementsprechend wird nur der diagonale Flip betrachtet. Bei diesem wird die Diagonale eines konvexen Vierecks durch die andere Diagonale ersetzt. Ein beispielhafter Flip ist in Abbildung 2.1 abgebildet.

Im nächsten Schritt wird die konvexe Hülle mit Randknoten definiert. Da die konvexe Hülle in dieser Arbeit um alle Randknoten erweitert werden muss.

Definition 2.7 (Konvexe Hülle mit Randknoten). Die konvexe Hülle einer Knotenmenge ist die kleinste konvexe Menge, die alle Knoten enthält. Die Randknoten sind die Knoten, die auf dem Rand der konvexen Hülle liegen, einschließlich solcher, die nicht an einer Ecke liegen. In dieser Arbeit werden bei der konvexen Hülle immer die Randknoten mit einbezogen.

Abschließend wird die konjunktive Normalform eingeführt, die im weiteren Verlauf für die Benutzung der SAT-Solver benötigt wird.

Definition 2.8 (Konjunktive Normalform). Eine Formel in konjunktiver Normalform (KNF) ist eine logische Formel, die als Konjunktion (UND-Verknüpfung) von Klauseln dargestellt wird, wobei jede Klausel eine Disjunktion (ODER-Verknüpfung) von Literalen ist. Ein Literal ist entweder eine Variable oder ihre Negation. Ein Beispiel für eine KNF ist

$$(x_1 \vee \overline{x_2}) \wedge (x_2 \vee x_3)$$

wobei x_1 , x_2 und x_3 Variablen sind. Eine Negation wird durch das Symbol $\overline{x}$ dargestellt.

Da in einigen Ansätzen die Orientierung von Knoten relevant ist, werden im Folgenden die relevanten Definitionen vorgestellt. Zunächst wird das Kreuzprodukt definiert, das die Orientierung von Vektoren beschreibt.

Definition 2.9 (Kreuzprodukt). Das Kreuzprodukt zweier Vektoren $\vec{a} = (a_1, a_2)$ und $\vec{b} = (b_1, b_2)$ in der Ebene ist ein Skalarwert, der die Orientierung der beiden Vektoren zueinander beschreibt. Es wird definiert als:

$$\vec{a} \times \vec{b} = a_1 b_2 - a_2 b_1$$

Das Kreuzprodukt kann anschließend verwendet werden, um die Orientierung von drei Knoten zu bestimmen.

Definition 2.10 (Rechtsknick / Linksknick). Ein Rechtsknick ist eine Abfolge von drei Knoten p_1 , p_2 und p_3 , die im Uhrzeigersinn angeordnet sind. Ein Linksknick ist eine Abfolge von drei Knoten p_1 , p_2 und p_3 , die gegen den Uhrzeigersinn angeordnet sind. Die Orientierung der Knoten kann durch das Kreuzprodukt der Vektoren $\overrightarrow{p_1 p_2}$ und $\overrightarrow{p_2 p_3}$ bestimmt werden. Wenn das Kreuzprodukt positiv ist, handelt es sich um einen Linksknick, wenn es negativ ist, um einen Rechtsknick. Ist das Kreuzprodukt null, liegen die Knoten auf einer Geraden.

Der Rechts- und Linksknick wird verwendet, um zu bestimmen, ob ein Knoten links oder rechts von einer Kante liegt.

Definition 2.11 (Links/Rechts von einer Kante). Ein Knoten p liegt links von einer Kante (p_1, p_2) , wenn die Knoten p_1 , p_2 und p einen Linksknick bilden. Analog liegt ein Knoten p rechts von einer Kante (p_1, p_2) , wenn die Knoten p_1 , p_2 und p einen Rechtsknick bilden.

3. Lösungsansätze

In diesem Kapitel werden die verschiedenen Ansätze erklärt, mit denen das Problem gelöst werden kann. Der erste Abschnitt betrachtet das Problem als Entscheidungsproblem, bei dem entschieden wird, ob es sich um eine Ja- oder Nein-Instanz handelt. Im Anschluss daran wird das Problem als Optimierungsproblem betrachtet. Das bedeutet, unabhängig von der Instanz, wird versucht eine möglichst gute Lösung zu finden. Im letzten Abschnitt wird das Problem während des Lösungsprozesses optimiert.

3.1. Entscheidungsproblem

In diesem Abschnitt wird das DCT als Entscheidungsproblem betrachtet. Das bedeutet dem Solver wird eine Instanz übergeben und dieser soll entscheiden, ob eine Lösung existiert oder nicht.

Unabhängig vom Ansatz muss für das DCT überprüft werden, ob die Summe der Sollgrade zur Anzahl der Kanten passt. Mithilfe der Gleichung

$$|E| = 3n - 3 - k$$

kann die Anzahl der benötigten Kanten in einer Triangulierung berechnet werden. Diese Anzahl kann dann in der Gleichung

$$|E| \cdot 2 \stackrel{!}{=} \sum \deg^*(i)$$

genutzt werden, um zu überprüfen, ob die Sollgrade eine Ja-Instanz ermöglichen. Diese Überprüfung hat auch den Vorteil, dass die Maximalität nicht überprüft werden muss. So kann angenommen werden, dass wenn der Sollgrad möglich ist, dieser nur mit einem Maximalen Graphen erfüllt werden kann.

Im Folgenden werden zwei verschiedene Ansätze beschrieben. Zum einen ein Kantenansatz und zum anderen ein Dreiecksansatz.

3.1.1. Kantenansatz

Bei diesem Ansatz werden Kanten betrachtet. Das bedeutet die DCT wird erreicht, indem bestimmte Bedingungen an die Kanten gestellt werden. Dafür werden zuerst zwei notwendige Bedingungen erklärt und dann weitere optionale, die die Laufzeit verbessern könnten.

Notwendige Bedingungen

Die erste Bedingung betrachtet Überschneidungen. Da eine Triangulierung keine Überschneidungen enthalten darf, werden alle sich kreuzenden Kantenpaare berechnet. Von jedem dieser Paare darf dann nur eine Kante aktiv sein.

Die zweite Bedingung ist die Gradbeschränkung. Für jeden Knoten i wird die Summe der inzidenten Kanten auf den Sollgrad $\deg^*(i)$ beschränkt. Um dies zu erreichen, wird für jeden Knoten festgelegt, dass die Summe der ausgehenden Kanten dem Sollgrad entspricht.

Kanten fixieren/ausschließen

In diesem Abschnitt werden optionale Bedingungen betrachtet, die Kanten fixieren oder ausschließen. Die erste und naheliegendste Bedingung ist die *fixierte Hülle*. Hierbei werden die Kanten der konvexen Hülle immer hinzugefügt. Diese dürfen hinzugefügt werden, da sie keine Schnittkanten besitzen. Würde eine dieser Kanten nicht hinzugefügt werden, wäre der Graph nicht maximal und somit keine Triangulierung. Da die Kanten der Hüllen in jeder möglichen Triangulierung vorhanden sind, verletzen sie auch nicht die Gradbeschränkung.

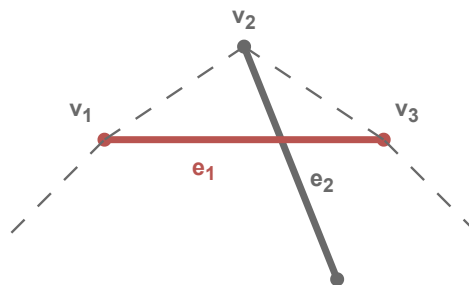


Abbildung 3.1.: Abhängig von dem Sollgrad von v_2 kann entschieden werden, ob e_1 aktiv oder nicht aktiv sein muss. Sollte $\deg^*(v_2) = 2$ sein, dann muss e_1 aktiv sein damit alle Kanten wie e_2 nicht aktiv sein können. Sollte $\deg^*(v_2) > 2$ sein, dann muss e_1 nicht aktiv sein damit der Sollgrad erreicht werden kann.

Eine weitere optionale Bedingung ist, zuvor berechnete Kanten zu fixieren oder auszuschließen. Eine Möglichkeit um Kanten zu fixieren ist immer drei aufeinander folgende Knoten der Hülle zu betrachten. Ein mögliches Beispiel ist in Abbildung 3.1 dargestellt. Sollte $\deg^*(v_2) = 2$ sein, muss die Kante e_1 in der Triangulierung genutzt werden. Der Grund dafür ist, dass alle Schnittkanten von e_1 nicht aktiv sein dürfen, da sie den Grad von v_2 erhöhen würden. Analog dazu kann die Kante e_1 ausgeschlossen werden, sollte $\deg^*(v_2) > 2$ sein. Sonst kann der Sollgrad nicht erreicht werden. Ein $\deg^*(v_2) < 2$ ist in einer Triangulierung nicht möglich.

3.1. Entscheidungsproblem

Zwei weitere Bedingungen, um Kanten auszuschließen, basieren auf der Teilung der Knotenmenge. Es werden zunächst alle Kanten betrachtet, bei denen beide Endknoten auf der Hülle liegen. Die folgenden beiden Bedingungen können genutzt werden, um diese Kanten auszuschließen.

$$|E| \cdot 2 > \sum_{v \in V} \deg^*(v) \quad (3.1)$$

$$\deg^*(v) > |V| - 1 \quad \forall v \in (V \setminus V_H) \quad (3.2)$$

Dabei sei V die Menge aller Knoten in der betrachteten Hälfte. Analog sei $V_H \subseteq V$ die Menge der Knoten auf der Hülle. Sollte eine der Bedingungen wahr werden, so kann die Kante ausgeschlossen werden. Die Bedingungen werden für beide Hälften getestet, die durch die Teilung entstehen.

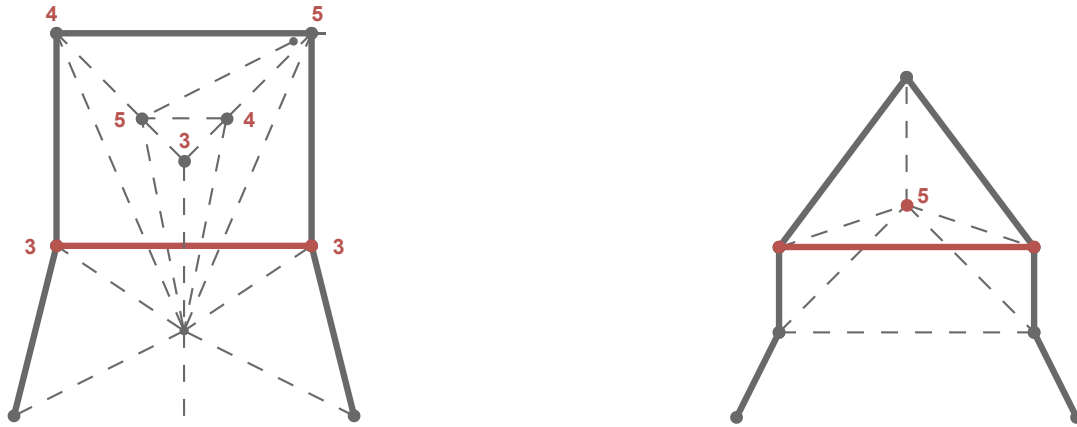


Abbildung 3.2.: Zwei Teilinstanzen, in denen die rote Kante ausgeschlossen werden kann. Das bedeutet, die Instanz wird an der roten Kante halbiert und auf diesen Hälften werden die Bedingungen überprüft. Links kann aufgrund von Formel (3.1) die Kante ausgeschlossen werden da die Gradsumme die Anzahl der Kanten unterschreitet. Rechts kann aufgrund von Formel (3.2) die Kante ausgeschlossen werden, weil der mittlere rote Knoten nicht genügend Kanten erzeugen könnte.

Zwei Beispiele sind in Abbildung 3.2 abgebildet. Dabei ist links eine Teilinstanz, in der die Anzahl der benötigten Kanten (28) größer als die Summe der Grade (27) ist. Und rechts eine Teilinstanz, bei der der rote Knoten mit dem Sollgrad 5 nur 3 mögliche Kanten erzeugen kann.

Eine weitere Möglichkeit, Kanten auszuschließen, besteht darin, ihre Schnittkanten zu betrachten. Wenn eine Kante e_1 zu viele Kanten eines Knotens v_1 blockiert, sodass dieser seinen Sollgrad nicht mehr erreichen kann, kann e_1 ausgeschlossen werden. In Abbildung 3.3 ist zu erkennen, dass e_2 , e_3 und e_4 durch e_1 blockiert werden. Da v_1 jedoch

3. Lösungsansätze

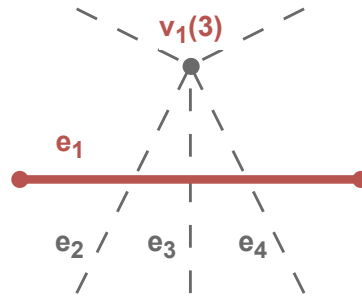


Abbildung 3.3.: In der Abbildung hat der Knoten v_1 einen Sollgrad von 3. Da die Kante e_1 drei von fünf möglichen Kanten blockiert, kann v_1 seinen Sollgrad nicht erreichen. Daher kann e_1 ausgeschlossen werden.

einen Sollgrad von 3 besitzt und nur 5 mögliche Kanten existieren, kann v_1 mit der aktiven Kante e_1 seinen Sollgrad nicht erreichen.

Alternative Kantenbeschränkung

Eine andere mögliche Bedingung ist die *alternative Kantenbeschränkung*. Diese wurde von Sándor P. Fekete et.al. [9] übernommen.

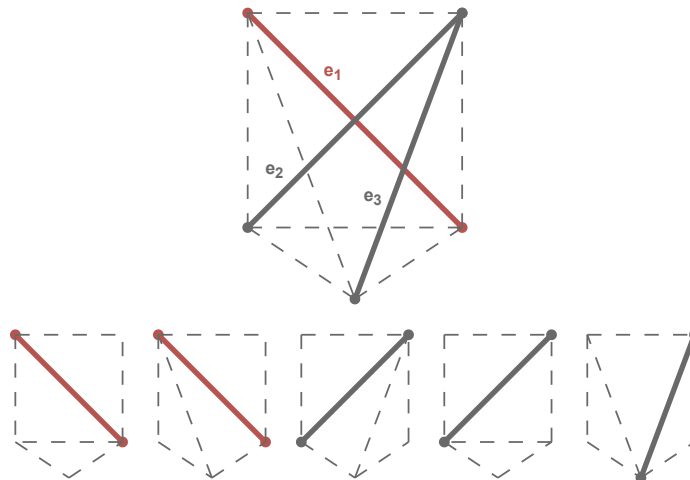


Abbildung 3.4.: Ein Vollständiger Graph mit allen möglichen Triangulierung. Die Kanten e_2 und e_3 sind Schnittkanten von e_1 . Es ist zu erkennen, dass entweder e_1 oder eine der Schnittkanten aktiv sein muss, damit eine Triangulierung möglich ist.

Der gesuchte Graph muss für diese Bedingung maximal sein. Damit dies der Fall sein kann, muss für jede Kante gelten, dass entweder die Kante oder eine der Schnittkanten

aktiv ist. Wie in Abbildung 3.4 abgebildet, ist e_1 die ausgewählte Kante und e_2 und e_3 die Schnittkanten. Weiter unten ist zu sehen, dass alle möglichen Triangulierungen mindestens eine der drei Kanten verwenden. Zu beachten ist allerdings, dass die Anzahl der Schnittkanten nicht gleich eins gesetzt werden darf, sondern mindestens auf eins. Wie man in Abbildung 3.4 sieht, können die Kanten e_2 und e_3 gleichzeitig aktiv sein.

Formulierungen

Im Folgenden werden zwei verschiedene Formulierungen beschrieben, die den Kantenansatz umsetzen. Zum einen eine mathematische Formulierung, die Formeln erstellt, und zum anderen eine Formulierung in KNF. Für diese Formulierung wird angenommen, dass V die Menge aller Knoten und E die Menge aller möglicher Kanten sei. E enthält also die Kanten, die keine Knoten schneiden. Für jede Kante $e \in E$ wird eine Bool Variable x_e erstellt. Diese Variable ist wahr, wenn die Kante e in der Triangulierung enthalten ist.

Mathematische Formulierung

Für die Überschneidungsbeschränkung darf für jedes Variablenpaar

$$(x_{e_1}, x_{e_2}) \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

nur eine Variable aktiv sein. Dies kann mit der Formel

$$x_{e_1} + x_{e_2} \leq 1 \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

umgesetzt werden.

Für die Gradbeschränkung wird die Summe der booleschen Variablen x_e für alle Kanten eines Knotens i auf den Sollgrad $\deg^*(i)$ beschränkt.

Dafür kann die Formel

$$\sum x_{e_i} = \deg^*(i) \quad \forall i \in V$$

genutzt werden, wobei e_i alle Kanten für den Knoten i darstellen.

Für die *alternative Kantenbeschränkung* kann die folgende Gleichung

$$\sum_{j \in I(e)} x_j + x_e \geq 1 \quad \forall e \in E$$

genutzt werden. Für die Fixierung bzw. Ausschließung der Kanten werden die zugehörigen Variablen auf *Wahr* oder *Falsch* gesetzt. Dies kann mit $x = 1$ bzw. $x = 0$ erfolgen.

KNF Formulierung

Die Überschneidungen können sehr ähnlich mit der Formel

$$\overline{x_{e_1}} \vee \overline{x_{e_2}} \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

3. Lösungsansätze

umgesetzt werden. Bei der Gradbeschränkung besteht das Problem, dass in KNF für eine Menge von Variablen nicht direkt festgelegt werden kann, wie viele von ihnen wahr sein müssen. Im Folgenden werden zwei verschiedene Ansätze beschrieben, um dies erreichen zu können. Der erste Ansatz nutzt Kardinalitätscodierungen aus einer Sammlung von Codierungen. Diese Sammlung wird von der Python Bibliothek PySAT bereitgestellt. Von diesen Codierungen kommen nur *sequential counters* [20], *sorting networks* [3], *cardinality networks* [1], *totalizer* [2], *modulo totalizer* [18], *modulo totalizer for k-cardinality* [16] und eine native Codierung in Frage. Die restlichen Codierungen bieten keine Möglichkeit um Kardinalitätsbeschränkungen größer als eins zu erzeugen.

In Abschnitt 6.4 wird sich zeigen, dass *sequential counters* am besten geeignet ist. Daher wird diese Codierung im Folgenden genauer erläutert. Der Ansatz der *sequential counters* wurde von Carsten Sinz [20] entwickelt. Er basiert auf einer Idee aus der Computerhardwareentwicklung. Zur Berechnung der Kardinalität k für die Klausel $(x_1 \vee x_2 \vee \dots \vee x_n)$ werden $n - 1$ Variablen s_i der Länge k in der unären Darstellung benötigt. Im Folgenden gibt für die Zahl $s_{x,y}$ das y die Stelle in s_x an. Zusätzlich werden n Hilfsvariablen v_i verwendet. Für $i = 1$ werden s_1 und v_1 wie folgt definiert

$$\begin{aligned} s_{1,1} &\Leftrightarrow x_1 \\ s_{1,j} &\Leftrightarrow 0 && \text{für } 1 < j \leq k \\ v_1 &\Leftrightarrow 0. \end{aligned}$$

Für $i = n$ wird v_n wie folgt gesetzt

$$v_n \iff x_n \wedge s_{n-1,k}.$$

Iterativ werden dann s_i und v_i für $i \in \{2, \dots, n - 1\}$ wie folgt definiert

$$\begin{aligned} s_{i,1} &\Leftrightarrow x_i \vee s_{i-1,1} \\ s_{i,j} &\Leftrightarrow x_i \wedge s_{i-1,j-1} \vee s_{i-1,j} && \text{für } 1 < j \leq k \\ v_i &\Leftrightarrow x_i \wedge s_{i-1,k}. \end{aligned}$$

Mit diesen Variablen können die folgenden Klauseln erstellt werden

$$\begin{aligned} &(\overline{x_1} \vee s_{1,1}) \\ &(\overline{s_{1,j}}) \quad \text{für } 1 < j \leq k \\ &\left. \begin{aligned} &(\overline{x_i} \vee s_{i,1}) \\ &(\overline{s_{i-1,1}} \vee s_{i,1}) \\ &(\overline{x_i} \vee \overline{s_{i-1,j-1}} \vee s_{i,j}) \\ &(\overline{s_{i-1,j}} \vee s_{i,j}) \\ &(\overline{x_i} \vee \overline{s_{i-1,k}}) \end{aligned} \right\} \quad \text{für } 1 < j \leq k \quad \left. \vphantom{\begin{aligned} &(\overline{x_i} \vee s_{i,1}) \\ &(\overline{s_{i-1,1}} \vee s_{i,1}) \\ &(\overline{x_i} \vee \overline{s_{i-1,j-1}} \vee s_{i,j}) \\ &(\overline{s_{i-1,j}} \vee s_{i,j}) \\ &(\overline{x_i} \vee \overline{s_{i-1,k}}) \end{aligned}} \right\} \quad \text{für } 1 < i < n \\ &(\overline{x_n} \vee \overline{s_{n-1,k}}). \end{aligned}$$

3.1. Entscheidungsproblem

Diese Klauseln stellen sicher, dass genau k Variablen von $(x_1 \vee x_2 \vee \dots \vee x_n)$ wahr sind. Alle Literale, die auf $s_{i,j}$ basieren, müssen mit Hilfsvariablen verknüpft werden.

Bei den Codierungen besteht die Möglichkeit, die Kardinalität auf den exakten Sollgrad oder den mindestens Sollgrad zu setzen. Insgesamt erzielen beide Bedingungen denselben Effekt, da der maximale Grad durch die Anzahl der Kanten begrenzt ist. Wenn jedoch nur gefordert wird, dass der Knoten mindestens den Sollgrad erreicht, müssen weniger Hilfsvariablen eingeführt werden. Bei der exakten Methode könnte der Solver allerdings schneller eine Lösung finden, da diese Bedingung stärker ist, da sie mehr Informationen liefert.

Die andere Methode ist die sogenannte *subset* Methode. Bei dieser Methode wird für jeden Knoten i eine Menge von Klauseln erstellt. Diese Klauseln umfassen alle Kombinationen der Kanten E_i der Größe

$$|E_i| - (\deg^*(i) - 1),$$

wobei E_i die Menge aller Kanten für den Knoten i darstellt. Angenommen, der Knoten v_1 besitzt die Kanten e_1, e_2, e_3, e_4 und den Sollgrad 2, dann werden folgende Klauseln erstellt

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4) \wedge (x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee x_4)$$

Damit diese vier Klauseln erfüllt sind, müssen mindestens zwei der Variablen wahr sein. Die Idee basiert darauf, dass für jede Klausel angenommen wird, dass nur noch eine Kante zum Erreichen des Sollgrads fehlt und die in der Klausel genannten Kanten noch nicht aktiv sind. Insgesamt wird so eine Kardinalitäts Codierung erzeugt. Diese Methode hat den Vorteil, dass keine Hilfsvariablen benötigt werden. Allerdings entstehen pro Knoten

$$\binom{|E_i|}{|E_i| - (\deg^*(i) - 1)}$$

Klauseln. Das wären in O-Notation

$$\mathcal{O}\left(n \cdot \binom{|E_{\max}|}{|E_{\max}| - d_{\min}}\right)$$

viele Klauseln. Wobei $E_{\max}$ die maximale Kantenanzahl pro Knoten und $d_{\min}$ der minimale Sollgrad ist. Da dies nicht polynomial ist, werden voraussichtlich zu viele Klauseln erzeugt.

Für die *alternative Kantenbeschränkung* können folgende Klauseln genutzt werden:

$$x_e \vee \bigvee_{e_j \in I(e)} x_{e_j} \quad \forall e \in E$$

Die Fixierung von Kanten kann ähnlich zur mathematischen Formulierung umgesetzt werden. Hierbei werden anstatt die Variablen auf *Wahr* und *Falsch* zu setzen, die Klauseln (x_e) und $(\overline{x_e})$ hinzugefügt.

3. Lösungsansätze

3.1.2. Dreiecksansatz

Bei dem Dreiecksansatz wird nicht jeder Kante eine Variable zugeordnet. Stattdessen erhält jedes mögliche leere Dreieck eine eigene Variable.

Die Gradbeschränkung kann aus der Kantenformulierung übernommen werden. Lediglich die Knoten auf der Hülle müssen gesondert betrachtet werden. Für diese Knoten muss der Sollgrad um eins reduziert werden, da ein Dreieck an der Hülle zwei Kanten für den Grad stellt.

Für die Überschneidungsbeschränkung können zwei verschiedene Ansätze genutzt werden. Zum einen können sich überschneidende Dreiecke ausgeschlossen werden. Die andere Möglichkeit ist es, Kanten zu betrachten. Wenn man die Kanten auf der Hülle gegen den Uhrzeigersinn betrachtet, muss auf der linken Seite immer genau ein Dreieck aktiv sein. Für alle anderen inneren Kanten müssen auf der linken sowie auf der rechten Seite immer genau gleich viele Dreiecke aktiv sein. Da es in einer Triangulierung keine Überschneidungen gibt, bestehen nur zwei Möglichkeiten. Entweder ist auf der linken Seite und auf der rechten Seite kein Dreieck aktiv oder auf beiden Seiten genau ein Dreieck aktiv.

Für den Dreiecksansatz gibt es als einzige optionale Bedingung das Ausschließen von Dreiecken. Dafür können die gleichen Bedingungen wie beim Ausschluss von Kanten verwendet werden. Es werden dann alle Dreiecke ausgeschlossen, die eine ausgeschlossene Kante enthalten.

Formulierungen

Für diese Formulierungen wird ebenfalls angenommen, dass V die Menge aller Knoten und E die Menge aller möglicher Kanten sei, also der Kanten die keine Knoten schneiden. Zusätzlich ist F die Menge leerer Dreiecke, da nur diese in einer Triangulierung vorkommen können. Für jedes Dreieck $f \in F$ wird eine Boolesche Variable x_f erstellt. Diese Variable ist wahr, wenn das Dreieck f in der Triangulierung enthalten ist.

Mathematische Formulierung

Die Gradbeschränkung kann analog zum Kantenansatz übernommen werden. Für die Knoten auf der Hülle muss der Sollgrad um eins reduziert werden. In Formeln wird dies als

$$\begin{aligned} \sum x_{f_i} &= \deg^*(i) & \forall i \in V \setminus V_H \\ \sum x_{f_i} &= \deg^*(i) - 1 & \forall i \in V_H \end{aligned}$$

definiert, wobei V_H die Menge der Knoten auf der Hülle und f_i alle Dreiecke für den Knoten i sind.

Der Ansatz ohne die Kantenüberschneidungen eignet sich besser für die mathematische

3.2. Optimierungsproblem

Formulierung. Er kann mit den folgenden Formeln umgesetzt werden:

$$\begin{aligned} \sum x_{f_e} &= 1 & \forall e \in E_H \\ \sum x_{f_{e,l}} &= \sum x_{f_{e,r}} & \forall e \in E \setminus E_H. \end{aligned}$$

Die Menge der Kanten auf der Hülle wird mit E_H bezeichnet und $f_{e,l}$ und $f_{e,r}$ alle Dreiecke auf der linken und rechten Seite der Kante e . Bei der Menge E_H ist zu beachten, dass die Kanten gegen den Uhrzeigersinn betrachtet werden.

KNF Formulierung

Für die Gradbeschränkung müssen auch hier die Sollgrade der Knoten auf der Hülle um eins reduziert werden. Diese Kardinalitäten können anschließend mit den Codierungen aus Abschnitt 3.1.1 umgesetzt werden. Für die Überschneidungsbeschränkungen werden in diesem Fall sich überschneidende Dreiecke ausgeschlossen. Dies kann mit der Formel

$$\overline{x_{f_1}} \vee \overline{x_{f_2}} \quad \forall f_1, f_2 \in F : f_2 \in I(f_1)$$

umgesetzt werden. Der Ansatz ohne Kantenüberschneidungen eignet sich auf Grund der fehlenden direkten Kardinalitätsunterstützung nicht für die KNF Formulierung.

3.2. Optimierungsproblem

Im Gegensatz zu den vorherigen Ansätzen wird DCT in diesem Abschnitt nicht als Entscheidungsproblem, sondern als Optimierungsproblem betrachtet. Dies hat zum einen den Vorteil, dass bei einem *Timeout* des Solvers nicht der gesamte Fortschritt verloren geht. Ein weiterer Vorteil ist, dass bei Nein-Instanzen versucht wird, dennoch eine Lösung zu finden. Das bedeutet, dass zumindest eine möglichst nahe Triangulierung bestimmt werden kann, sollte es in bestimmten Bereichen nicht möglich sein, eine DCT zu finden.

Für die Bewertung der Lösungen wurde folgende Formel

$$\text{diff}_i = \left| \deg^*(i) - \deg(i) \right| \tag{3.3}$$

$$q_i = \begin{cases} \frac{\deg^*(i) - \text{diff}_i}{\deg^*(i)} & \deg^*(i) \geq \text{diff}_i \\ 0 & \text{sonst} \end{cases} \tag{3.4}$$

$$Q = \sum_{i \in V} \frac{q_i}{n} \tag{3.5}$$

eingeführt. Sie berechnet den Gesamtdurchschnitt aller prozentualen Gradabweichungen und liefert eine Bewertung (Q) für eine Triangulierung mit n Knoten. In Formel (3.3) wird die Differenz zwischen dem gewünschten und aktuellen Grad berechnet. Formel (3.4) bewertet diese Abweichung prozentual, woraufhin in Formel (3.5) ein Durchschnitt über alle prozentualen Werte gebildet wird.

3. Lösungsansätze

Ansätze

Es werden drei Ansätze vorgestellt. Der erste ist ein naiver Ansatz der versucht mit Flips die Gradbeschränkung zu erreichen. Die anderen beiden Ansätze verwenden einen mathematischen Ansatz. Ein KNF-Ansatz ist nicht möglich, da KNF-Formeln nur für Entscheidungsprobleme geeignet sind.

Flips

In dieser Heuristik werden die in Definition 2.6 definierten Kantenflips verwendet. Ziel ist es, die Gradbeschränkung zu erreichen. Dabei wird ausgenutzt, dass Flips den Grad eines Knotens verändern können, die Triangulationseigenschaft dabei aber erhalten bleibt.

Das Verfahren beginnt mit einer initialen Triangulierung. Diese wird durch ein klassisches Triangulationsverfahren erzeugt. Anschließend werden maximal k viele Flips durchgeführt. Ziel ist es, die Gradbeschränkung zu erfüllen.

Algorithm 1 CHOOSE EDGE(v, p)

```
1:  $E \leftarrow \{e \mid e \text{ ist ausgehende Kante von } v\}$ 
2: while  $E \neq \emptyset$  do
3:    $e \leftarrow$  zufällige Kante aus  $E$ 
4:    $E \leftarrow E \setminus \{e\}$ 
5:    $u \leftarrow$  Knoten von  $e$  and  $v \neq u$ 
6:   if  $\deg(u) \neq \deg^*(u)$  then
7:     return  $e$ 
8:   else if zu  $p$  % then
9:     return  $e$ 
10:  end if
11: end while
12: return  $e$ 
```

Zunächst wird mit Algorithmus 1 eine Kante ausgewählt die geflippt werden soll. Der Algorithmus versucht, eine Kante zu finden, bei der die Knoten den Sollgrad nicht erfüllen. Zusätzlich wird über den Parameter p eine Wahrscheinlichkeit eingeführt. Dadurch kann auch eine Kante ausgewählt werden, deren Knoten den Sollgrad erfüllen. Dies verhindert, dass das Verfahren in einem lokalen Optimum stecken bleibt.

Die ausgewählte Kante e_0 wird anschließend wie folgt geflippt. Für die Kante werden zunächst alle Dreiecke gesucht, die diese Kante enthalten. Dies geschieht, indem für die beiden Knoten $x_{0,0}$ und $x_{0,1}$ der Kante e_0 alle ausgehenden Kanten verglichen werden. Sollten zwei dieser Kanten den gleichen Knoten v haben, muss das Tripel $(x_{0,0}, x_{0,1}, v)$ ein Dreieck bilden, das die Kante e_0 enthält. Sollten mehr als zwei Dreiecke gefunden werden, so müssen die beiden leeren Dreiecke ausgewählt werden. Da es in einer Triangulierung nicht zu Überschneidungen kommen darf, kann es nur zwei leere Dreiecke pro Kante geben. Aus diesen beiden Dreiecken $(x_{0,0}, x_{0,1}, v_0)$ und $(x_{0,0}, x_{0,1}, v_1)$ wird dann die Kante e_0 entfernt und durch die neue Kante, bestehend aus v_0 und v_1 , ersetzt.

3.3. Conflict-driven clause learning

Nach jedem Flip wird überprüft, ob die Gradbeschränkung für alle Knoten erfüllt ist. Sollte dies der Fall sein, wird das Verfahren beendet.

Mathematische Ansätze

In diesem Abschnitt werden zwei Zielfunktionen beschrieben, die das DCT als Optimierungsproblem lösen. Für beide Ansätze können die Bedingung aus Abschnitt 3.1.1 übernommen werden. Nur die Gradbeschränkung wird durch eine Zielfunktion ersetzt. Zum einen wird als Zielfunktion die Formel (3.5) genutzt. Für diese wird eine angepasste Version der Formel (3.5) maximiert. Die Teiler in (3.5) und (3.4) werden entfernt. Diese sind für die Maximierung nicht relevant.

Eine Fallunterscheidung für (3.4) ist für eine Zielfunktion nicht sinnvoll. Die Variablen q_i werden daher mit folgender Formel berechnet.

$$q_i = \deg^*(i) - \min(\text{diff}_i, \deg^*(i))$$

Zusammengefasst ergibt das folgende Zielfunktion.

$$\max \left(\sum_{i \in V} q_i \right)$$

Die andere Zielfunktion minimiert die maximale Gradabweichung. Dafür wird eine Hilfsvariable max_min eingeführt. Mit folgenden Formeln kann die Zielfunktion definiert werden

$$\begin{aligned} \text{max_min} &\geq \text{diff}_i && \forall i \in V \\ \text{max_min} &\geq 0 \end{aligned}$$

Die Variable max_min wird als Zielfunktion minimiert.

3.3. Conflict-driven clause learning

In diesem Kapitel werden SAT-Solver genauer betrachtet. SAT-Solver werden genutzt um KNF-Formulierungen zu lösen. In diesem Abschnitt sollen keine weiteren Klauseln gefunden werden, stattdessen wird der Solver während der Lösung optimiert. Dafür wird *Conflict-driven clause learning* (CDCL) betrachtet. Die in diesem Kapitel vorgestellten Informationen stammen aus dem Buch *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability* [14]. Im Allgemeinen funktionieren SAT-Solver so, dass sie jede Möglichkeit ausprobieren. Dabei werden zuerst Möglichkeiten ausgetestet, die aufgrund der aktuellen Belegung und Klauseln relevanter sind. Dies lässt sich, wie in Abbildung 3.5 gezeigt, als Baum darstellen.

Eine Möglichkeit die relevanten Äste effektiv zu untersuchen, ist der CDCL-Ansatz. Das Besondere am CDCL-Ansatz ist, dass es einen Pfad gibt, der eine aktuelle Teilbelegung der Literale speichert. Zusätzlich muss für jedes belegte Literal eine Begründung in Form

3. Lösungsansätze

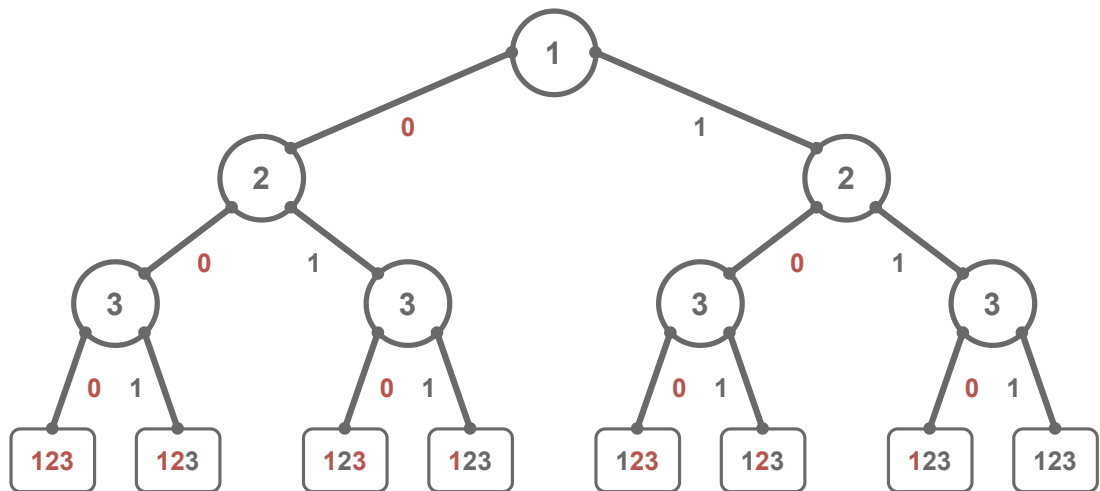


Abbildung 3.5.: Ein Baum, der die Möglichen Entscheidungen eines SAT-Solvers darstellt. Jeder Ast entspricht einer Belegung der Literale. Die Blätter sind entweder erfüllende Belegungen oder Konflikte. Wobei eine rote Zahl für eine nicht belegtes Literal steht und eine schwarze Zahl für ein belegtes Literal.

einer Klausel angegeben werden. Diese Begründungen werden bei Konflikten relevant. Sollte es dazu kommen das der Solver keine weiteren Literale durch Begründungen setzen kann, wird ein Literal ohne Begründung gesetzt. Dieser Schritt wird als neue Entscheidungsebene bezeichnet.

3.3.1. Unit-Klauseln

Der Grundansatz zur Erweiterung des Pfades besteht darin, *Unit-Klauseln* zu finden. Dabei handelt es sich um Klauseln, in denen nur ein Literal noch nicht belegt ist und die Klausel nicht erfüllt ist. Zum Beispiel kann durch die Klausel $(\bar{x}_1 \vee x_2 \vee \bar{x}_3)$ und dem Pfad $(\bar{x}_2, x_3)$ das Literal $\bar{x}_1$ hinzugefügt werden. Die Klausel dient hierbei als Begründung.

Sollten keine weiteren *Unit-Klauseln* existieren, wird basierend auf Heuristiken ein Literal belegt. Bei dieser Art der Belegung wird intern vermerkt, dass eine neue Entscheidungsebene erreicht wurde. Die sind auch bei Konflikten wichtig. Es ist zu beachten, dass jeder Abschnitt des Pfades einer Entscheidungsebene zugeordnet werden kann. Ein Literal, das weiter rechts im Pfad steht, wird einer höheren Entscheidungsebene zugeordnet.

3.3.2. Konflikte

Ein Konflikt tritt auf, wenn ein Literal dem Pfad hinzugefügt werden soll, es aber bereits eine andere Belegung besitzt. Hier zeigt sich der besondere Ansatz von CDCL. Kommt es zu einem Konflikt, wird nicht einfach zu einer vorherigen Entscheidungsebene zurückgekehrt. Stattdessen werden die Informationen des Konflikts zur Lösung des

Problems genutzt. Sei

$$c = (\bar{l} \vee \bar{a}_1 \vee \dots \vee \bar{a}_k)$$

eine Klausel, die die Begründung für einen Konflikt darstellt. Das bedeutet aufgrund dieser Klausel wurde versucht ein Literal l zu belegen, das schon eine andere Belegung besitzt. Es kann angenommen werden, dass l und ein weiteres Literal a_i aus c auf der höchsten Entscheidungsebene d liegen. Andernfalls wäre c eine *Unit-Klausel* und somit wäre l bereits auf $d - 1$ belegt worden. Weiterhin kann angenommen werden, dass das Literal l das aus c am weitesten rechts im Pfad stehende ist. Also das Literal aus c , das zuletzt belegt wurde. Daraus folgt, dass l keine neue Entscheidungsebene erzeugt hat, da zumindest a_i vor l auf der Entscheidungsebene d belegt wurde.

Es existiert somit eine Begründung

$$(\bar{l} \vee \bar{r}_1 \vee \dots \vee \bar{r}_k)$$

für die erste Belegung von l . Damit kann die Klausel

$$c' = (\bar{a}_1 \vee \dots \vee \bar{a}_k \vee \bar{r}_1 \vee \dots \vee \bar{r}_k)$$

erzeugt werden, wobei c' mindestens eine Belegung aus der Entscheidungsebene d enthält. Sollte es ein weiteres Literal außer a_i geben, das auf der Entscheidungsebene d belegt wurde, ist c' ebenfalls eine Konfliktklausel. In diesem Fall wird c' so lange rekursiv erzeugt, bis es genau ein Literal $\bar{x}$ gibt, das auf Entscheidungsebene d belegt wurde. Der rekursive Aufruf fügt dabei die Begründung des zweiten Literals aus Entscheidungsebene d ohne dieses Literal zu c' hinzu. Von den restlichen Literalen in c' wird dann die höchste Entscheidungsebene bestimmt und zu dieser zurückgekehrt. Anschließend wird $\bar{x}$ dem aktuellen Pfad hinzugefügt und c' als Begründung für die Belegung verwendet.

4. Instanzgenerierung

In diesem Kapitel werden die verschiedenen Instanzentypen und ihre Generierung dargestellt. Auf diesen Instanzen werden im späteren Verlauf die Experimente ausgeführt.

Die Knotenmengen werden hierbei durch die von Python bereit gestellte Funktion *randint* generiert. Mit dieser werden so lange verschiedene Koordinaten berechnet, bis die gewünschte Anzahl erreicht wurde.

4.1. Ja - Instanzen

Bei den Ja-Instanzen handelt es sich um Knotenmengen, bei denen die $\deg^*$ -Funktion Grade liefert, die DCT ermöglichen. Hierbei ist nicht bekannt, wie viele mögliche Lösungen existieren, nur dass mindestens eine vorhanden ist.

Der generelle Ablauf zur Erzeugung von Ja-Instanzen beginnt mit einer zufälligen Knotenmenge. Dabei wird lediglich die Anzahl und Position der Knoten vorgegeben. Anschließend wird mit einem klassischen Triangulationsverfahren eine initiale Triangulierung erstellt. Von dieser werden dann die Grade gespeichert, um sie als Eingabe für die zu testenden Algorithmen zu verwenden.

Alle fünf Verfahren zur Erzeugung von Ja-Instanzen werden in Abbildung 4.1 visualisiert.

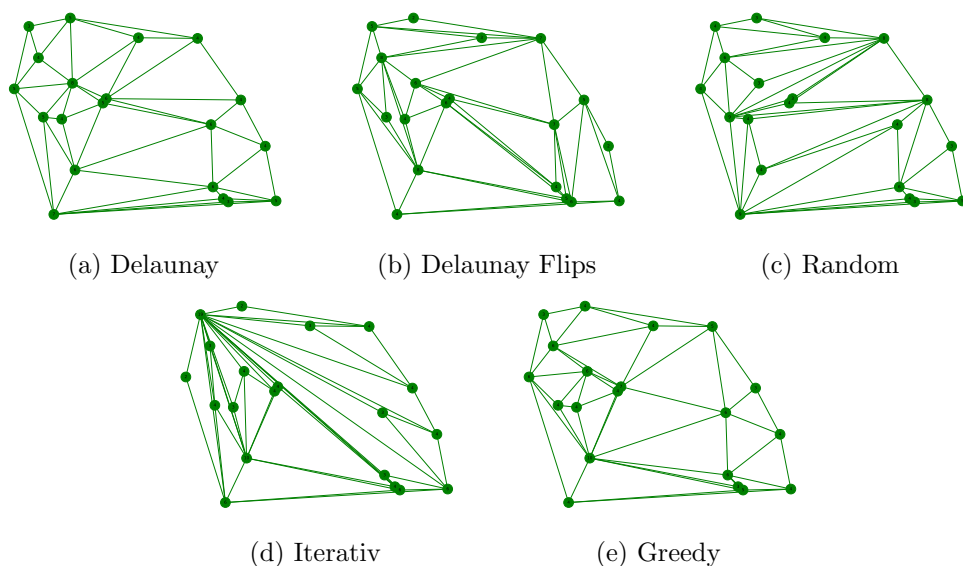


Abbildung 4.1.: Visualisierung der fünf verschiedenen Instanztypen.

4. Instanzgenerierung

4.1.1. Delaunay

Das erste klassische Verfahren zur Gradbestimmung verwendet die Delaunay-Triangulierung. Die Delaunay-Triangulierung wurde bereits im Kapitel Abschnitt 1.3.1 vorgestellt. Die Besonderheit bei dieser Instanz ist das viele gleichschenklige Dreiecke erstellt werden. Dies ergibt viele Kanten der gleichen Länge und gleichmäßiger verteilte Grade. Für die Erstellung der Delaunay-Triangulierung wird die Bibliothek *scipy* genutzt.

4.1.2. Delaunay mit zufälligen Flips

Das zweite Verfahren zur Erzeugung von Ja-Instanzen erweitert das Delaunay Verfahren, indem zufällige Flips durchgeführt werden. Zunächst wird eine Delaunay-Triangulierung der Knotenmenge erstellt. Auf dieser Triangulierung werden anschließend k zufällige Flips gemäß Definition 2.6 durchgeführt. Das genaue Verfahren wurde in Abschnitt 3.2 schon beschrieben. Diese Instanzen haben ähnliche Eigenschaften wie die Delaunay Instanzen. Allerdings kann sich die Instanz stark unterscheiden, was abhängig von der Größe von k ist. Generell gibt es höhere Grade und längere Kanten, als bei Delaunay.

4.1.3. Zufällige Kantenerzeugung

Das nächste Verfahren basiert auf der zufälligen Auswahl von Kanten. Hierbei werden zunächst alle möglichen Kanten zwischen den Knoten identifiziert. Im ersten Schritt werden Kanten ausgeschlossen, die andere Knoten schneiden würden. Die verbleibenden gültigen Kanten werden dann in einer zufälligen Reihenfolge betrachtet. Für die betrachteten Kanten wird geprüft, ob sie eine der bereits hinzugefügten Kanten schneidet. Nur wenn keine Schnittkanten entstehen, wird die Kante der Triangulierung hinzugefügt. Dieses Verfahren ist maximal, da alle möglichen Kanten iterativ betrachtet wurden.

Generell kann aufgrund der Zufälligkeit keine Aussage über die Struktur der Instanz getroffen werden. Allerdings werden meistens längere Kanten und Knoten mit höheren Graden hinzugefügt.

4.1.4. Iteration

Ein weiteres Verfahren zur Erzeugung von Ja-Instanzen basiert auf der Iteration. Hierbei werden zunächst alle möglichen Kanten zwischen den Knoten identifiziert. Diese werden nun nacheinander betrachtet und wenn sie keine andere Kante schneiden, hinzugefügt. Dieses Verfahren ist ebenfalls maximal, da alle möglichen Kanten betrachtet wurden. Beim iterativen Verfahren ist zu beachten das Triangulationen erzeugt werden, die einen Knoten mit einem sehr hohen Sollgrad sowie langen Kanten haben. Dies basiert darauf, dass die Kanten in der Reihenfolge erstellt werden, in der sie betrachtet werden. Dadurch können alle Kanten des ersten Knotens hinzugefügt werden, da es noch keine anderen Kanten gibt, die diese schneiden könnten.

4.1.5. Greedy

Das letzte Verfahren ist das Greedy Verfahren. Hierbei werden auch alle möglichen Kanten zwischen den Knoten identifiziert. Allerdings werden sie im Gegensatz zum iterativen Verfahren zunächst nach der Länge aufsteigend sortiert. Anschließend wird wie beim Iterativen Verfahren jede mögliche Kante nacheinander hinzugefügt. Aufgrund des Verfahrens werden vor allem kurze Kanten hinzugefügt das hat zur Folge das es eher kleinere Sollgrade gibt. Generell haben diese Instanzen die größte Ähnlichkeit mit den Delaunay Instanzen.

4.2. Nein-Instanzen

Nein-Instanzen sind Knotenmengen, bei denen die $\deg^*$ Funktion Gradwerte liefert, die DCT nicht ermöglichen. Diese Instanzen entstehen nicht direkt, sondern werden aus Ja-Instanzen abgeleitet. Nach der Generierung muss zusätzlich geprüft werden, ob die Instanz nicht möglich ist. Da die beschriebenen Verfahren mit hoher Wahrscheinlichkeit Nein-Instanzen erzeugen, allerdings nicht zu 100 Prozent. Es ist zu beachten, dass die Summe der Sollgrade mithilfe der Polyederformel einfach berechnet und überprüft werden kann. Daher besitzen die abgeleiteten Nein-Instanzen insgesamt die gleichen Sollgrade wie die Ja-Instanzen. Zusätzlich darf kein Sollgrad größer als die Anzahl der Knoten abzüglich eins sein, da sonst der Sollgrad nicht erfüllt werden kann, da nicht genügend Kanten erzeugt werden können.

Bei der ersten Methode werden zwei Knoten ausgewählt. Vom ersten Knoten wird ein Wert abgezogen und beim zweiten Knoten derselbe Wert hinzugefügt. Zunächst wird eine zufällige Zahl r zwischen null und c gewählt. Hierbei ist c eine konstante Zahl, die beim Generieren festgelegt wird. Anschließend werden zwei Knoten v_1 und v_2 ausgewählt. Dabei müssen die Bedingungen

$$\begin{aligned} v_1 &\neq v_2 \\ \deg^*(v_1) &> r + 2 \\ \deg^*(v_2) + r &< n - 1 \end{aligned}$$

erfüllt sein, wobei n die Anzahl der Knoten ist. Wenn beide Knoten die Bedingungen erfüllen, wird der Sollgrad von v_1 um r verringert. Der Sollgrad von v_2 wird um r erhöht.

Die zweite Methode besteht darin, zwei Knoten auszuwählen und deren Sollgrad zu tauschen. Hierbei muss lediglich beachtet werden, dass es sich um zwei verschiedene Knoten handeln muss.

Beide Ansätze werden mehrfach angewendet, um die Wahrscheinlichkeit zu erhöhen, dass die Instanzen nicht mehr möglich sind. Knoten werden dabei allerdings nicht mehrfach betrachtet.

Die erste Methode wird im Folgenden *move* genannte. Die andere Methode wird *change* genannt.

4.3. Theorie mehrerer Lösungen

Bei Ja-Instanzen stellt sich die Frage, ob sie eindeutig sind. Das bedeutet, ob es pro Ja-Instanz nur eine Lösung gibt und ob die Instanz nicht mehr erfüllbar ist, sobald eine Kante dieser Lösung verboten wird. Es lässt sich zeigen, dass Instanzen existieren, bei denen mehrere Lösungen möglich sind, weshalb Ja-Instanzen nicht eindeutig sein müssen. In Abbildung 4.2 ist ein Beispiel für eine solche Instanz zu erkennen, bei der zwei verschiedene Lösungen für die gleiche Ja-Instanz existieren. Die Zahlen an den Knoten geben hierbei den Sollgrad an.

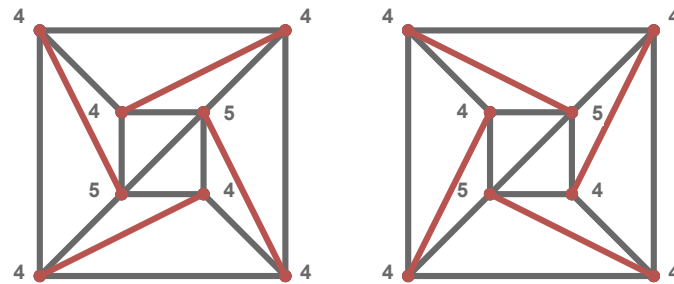


Abbildung 4.2.: Eine Ja-Instanz mit zwei möglichen Lösungen. Die Zahlen an den Knoten geben hierbei den Sollgrad an. Die roten Kanten sind jene die in den Lösungen unterschiedlich sind.

Diese Eigenschaft wird in Abschnitt 6.12.1 zusätzlich im Zusammenhang mit der Schwierigkeit einer Instanz untersucht. Diese Art von Konstruktion kann theoretisch unendlich fortgesetzt werden. Dies könnte genutzt werden, um auf andere Probleme zu reduzieren. Die Anzahl der Lösungen einer Instanz kann somit als eine Eigenschaft dieser betrachtet werden.

5. Solver

In dieser Arbeit werden vier verschiedene Solver verwendet PySAT, OR-Tools, Gurobi und Simplified CDCL Solve. Dieses Kapitel liefert eine Übersicht über diese und eine Implementierung der in Kapitel 3 beschriebenen Ansätze.

5.1. Generelle Implementierungen

In diesem Kapitel werden generelle Implementierungen gegeben, die in allen Solvern verwendet werden. Zum einen wird betrachtet, wie der Graph verwaltet wird. Danach wird erklärt, wie die Schnittkanten bestimmt werden. Im letzten Abschnitt wird eine Implementierung für das in Abschnitt 5.1 beschriebene Ausschließen von Kanten anhand von Schnittkanten erläutert.

Graphoperationen

Alle Kanten werden mit der Bibliothek *Networkx* verwaltet. Mit dieser können auch die inzidenten Kanten eines Knotens abgefragt werden. Beim Erstellen der Kanten muss getestet werden, ob die Kanten keinen Knoten schneiden. Dies wird mit der Bibliothek *Shapely* getestet. Bei der Berechnung der Hülle muss beachtet werden, dass alle Knoten, die auf der Hülle liegen, dazu gehören. Die *convex_hull* Funktion von *Shapely* liefert nur die Knoten auf den Ecken der Hülle. Damit auch Knoten auf einer Geraden zwischen zwei Knoten an den Ecken betrachtet werden, muss nach der Berechnung der konvexen Hülle noch der Schnitt zwischen konvexer Hülle und allen Knoten berechnet werden. In diesem Schnitt sind dann alle Knoten enthalten, die auf der Hülle liegen.

Schnittkanten

Für die Bestimmung der Schnittkanten wird ein Algorithmus verwendet, der auf dem Konzept des *Rechtsknicks* beziehungsweise *Linksknicks* basiert. Dafür ist es notwendig, dass ein vollständiger Graph betrachtet wird, da angenommen wird, dass zwischen allen Knoten eine Kante existiert. Dementsprechend müssen die Schnittkanten danach auf mögliche Kanten gefiltert werden. Es wird jede Kante $e_i \in E$, bestehend aus den Knoten $v_{i,1}$ und $v_{i,2}$, nacheinander betrachtet. Anschließend werden zwei Knoten v_3 und v_4 gesucht, die auf unterschiedlichen Seiten der Kante liegen. Dabei gilt das Tripel $(v_{i,1}, v_{i,2}, v_3)$ bildet einen Rechtsknick und das Tripel $(v_{i,1}, v_{i,2}, v_4)$ einen Linksknick. Danach muss überprüft werden, ob $v_{i,1}$ und $v_{i,2}$ auf verschiedenen Seiten der Kante liegen, die durch v_3 und v_4 gebildet wird. Sind diese Bedingungen erfüllt, so ist die Kante (v_3, v_4) eine Schnittkante von e_i . Der Vorteil dieses Algorithmus liegt darin, dass alle vier Knicküberprüfungen

5. Solver

notwendige Eigenschaften darstellen. In der Praxis werden bereits dadurch viele Knoten ausgeschlossen, da sie die ersten Knickbedingungen nicht erfüllen.

Schnittkanten ausschließen

Um die Kanten zu finden, bei denen zu viele Schnittkanten ausgeschlossen werden, wird jede Kante iterativ in einem vollständigen Graphen betrachtet. Für jede betrachtete Kante werden zunächst Zähler für jeden Knoten erstellt. Danach werden nacheinander alle Schnittkanten der betrachteten Kante aufgerufen. Für alle Schnittkanten wird der Zähler der beiden Endknoten der Schnittkante um eins erhöht. Am Ende werden alle Knoten überprüft. Sollte ein Knoten i folgende Formel erfüllen, wird die Kante ausgeschlossen

$$\text{deg}(i) - \text{node_count}[i] > \text{deg}^*(i).$$

Die Formel basiert darauf, dass wenn der aktuelle Grad ohne die Schnittkanten größer als der Sollgrad ist, der Sollgrad nicht erfüllt werden kann.

5.2. PySAT

PySAT ist eine Python-Bibliothek, die eine Sammlung von SAT-Solvern bereitstellt. Diese SAT-Solver entscheiden, ob eine KNF-Formel erfüllbar ist. Dementsprechend kann PySAT nur Entscheidungsprobleme, aber keine Optimierungsprobleme lösen. Für alle SAT-Solver wird eine einheitliche Schnittstelle in Python bereitgestellt. Zusätzlich werden verschiedene Codierungen für Kardinalitäten angeboten. Das bedeutet, dass für eine Menge von Variablen angegeben wird, wie viele von ihnen wahr sein sollen und erhält eine KNF-Formel, die diese Bedingung abbildet. PySAT stellt hierfür verschiedene Codierungen mit Implementierung bereit. PySAT ist open source und kostenlos.

Für PySAT wird die KNF-Formulierung verwendet. Es wurden alle Bedingungen aus Abschnitt 3.1.1 und Abschnitt 3.1.2 umgesetzt.

5.3. OR-Tools

Der zweite Solver ist OR-Tools (Google Optimization Tools) von Google. Dieser unterstützt verschiedene Modellierungsarten wie lineare Programmierung (LP), gemischt-ganzzahlige Programmierung (MIP), Constraint Programming (CP) und Routing-Algorithmen. OR-Tools ist in C++ implementiert und es wird eine Python-Schnittstelle bereitgestellt. Mit OR-Tools können sowohl Entscheidungsprobleme als auch Optimierungsprobleme gelöst werden. Kardinalitätsbeschränkungen und andere lineare Bedingungen werden direkt unterstützt und müssen nicht codiert werden. OR-Tools ist open source und kostenlos unter der Apache License 2.0.

Für OR-Tools wird die mathematische Formulierung verwendet. OR-Tools würde auch die KNF-Formulierung unterstützen, jedoch ist die mathematische Formulierung durch die direkte Unterstützung von Kardinalitäten deutlich einfacher umzusetzen. Die

Dreiecksformulierungen aus Abschnitt 3.1.2 können direkt an den Solver übergeben werden.

Die Kantenformulierung aus Abschnitt 3.1.1 kann ebenfalls direkt übergeben werden. Allerdings können die Paare bei den Überschneidungen auch anders behandelt werden. So kann für jedes Paar die Funktion $AddAtMostOne(x_{e_1}, x_{e_2})$ oder $AddBoolOr(\overline{x_{e_1}}, \overline{x_{e_2}})$ verwendet werden.

Für OR-Tools werden zusätzlich die Zielfunktionen aus Abschnitt 3.2 implementiert. Für die Durchschnitts-Zielfunktionen ist folgende Formel gegeben.

$$\text{diff}_i = \left| \deg^*(i) - \deg(i) \right| \quad (5.1)$$

$$\max \left(\sum_{i \in V} \deg^*(i) - \min(\text{diff}_i, \deg^*(i)) \right). \quad (5.2)$$

Da OR-Tools keine Betragsfunktion in Formeln erlaubt, muss die Funktion *AddAbsEquality* verwendet werden. Diese erzeugt den Betrag mit den folgenden Bedingungen

$$\begin{aligned} \text{diff}_i &\geq \deg^*(i) - \deg(i) \\ \text{diff}_i &\geq \deg(i) - \deg^*(i) \\ \text{diff}_i &= (\deg^*(i) - \deg(i)) \vee (\deg(i) - \deg^*(i)). \end{aligned}$$

Die Variable diff_i ist hierbei eine Integer-Hilfsvariable, die den Betrag der Differenz erhält. Diese Variable kann dann für die Formel (5.1) genutzt werden.

OR-Tools erlaubt zusätzlich keine *min*-Funktion in Formeln. Deshalb wird die Funktion *AddMinEquality* verwendet. Diese nutzt, dass eine Minimierung der Formel

$$\min(a, b) = -\max(-a, -b)$$

in eine Maximierung umgewandelt werden kann. Eine Betragsfunktion kann auf eine Maximierungsfunktion reduziert werden. Da dies im Beispiel oben der Fall ist, können hier die gleichen angepassten Bedingungen genutzt werden.

Alle angepassten Formeln zusammengefasst sehen wie folgt aus:

$$\begin{aligned} \text{diff}_i &\geq \deg^*(i) - \deg(i) \\ \text{diff}_i &\geq \deg(i) - \deg^*(i) \\ \text{diff}_i &= (\deg^*(i) - \deg(i)) \vee (\deg(i) - \deg^*(i)) \\ \min_i &\geq -\text{diff}_i \\ \min_i &\geq -\deg^*(i) \\ \min_i &= (-\text{diff}_i) \vee (-\deg^*(i)) \\ q_i &= \deg^*(i) - (-\min_i) \\ Q &= \sum_{i \in V} q_i. \end{aligned}$$

Als Zielfunktion wird dann Q maximiert.

Die Max-Min-Zielfunktion kann mit dem angepassten diff_i direkt übergeben werden.

5.4. Gurobi

Den vorletzte Solver stellt Gurobi dar. Es werden ebenfalls Modellierungen wie LP, MIP und quadratische Programmierung (QP) unterstützt. Gurobi ist in C implementiert und bietet eine Python-Schnittstelle an. Gurobi ist kostenpflichtig, bietet jedoch eine kostenlose Lizenz für die Forschung an.

Für Gurobi kann die mathematische Formulierung genutzt werden. Umgesetzt wurde zum einem der Kantenansatz aus Abschnitt 3.1.1 und der Dreiecksansatz aus Abschnitt 3.1.2. Gurobi bietet durch die *addConstr* Funktion direkt die Möglichkeit, alle Formeln hinzuzufügen.

5.5. Simplified CDCL Solve

Der CDCL-Solver *Simplified CDCL Solve (CaDiCaL)* [4] von Armin Biere ermöglicht es, eigene Funktionen in den Lösungsprozess einzubinden. Diese Funktion erlaubt es, Literale mit Begründung zu setzen. Im Folgenden wird diese Funktion *Propagation* genannt. Die Idee bei dieser *Propagation* ist es, die Bedingungen aus Abschnitt 3.1.1 auf Teilinstanzen anzuwenden.

Es wird angenommen, dass die Hülle fixiert wurde und alle Kanten, die die gesamte Instanz halbieren, bereits getestet wurden. Der Grund hierfür ist, dass die Hülle eine geeignete Fläche bietet, mit der alle Knoten betrachtet werden können.

Zur Umsetzung wird mit Hilfe eines Arrangements verfolgt, welche Kanten aktuell aktiv sind. Es ist darauf zu achten, dass bei jeder neuen Entscheidungsebene das Arrangement gespeichert und eindeutig einer Ebene zugeordnet wird. Tritt ein Konflikt auf, wird das Arrangement auf die entsprechende Entscheidungsebene zurückgesetzt.

Wird die *Propagation* aufgerufen, können alle Flächen des Arrangements betrachtet werden. Relevant sind dabei nur Flächen, die mehr als drei Kanten besitzen. Zusätzlich kann die unbegrenzte Fläche ausgeschlossen werden, da alle Knoten durch die Hülle eingeschlossen sind. Für diese Flächen können dann alle teilenden Kanten erzeugt werden, indem sämtliche Zweierkombinationen der Randknoten betrachtet werden. Dadurch wird die Fläche in zwei Hälften geteilt und die Kanten können getestet werden.

Vor dem Testen können alle Kanten übersprungen werden, die bereits festgelegt wurden oder keine gültigen Kanten sind. Für die Knotenzuordnung kann zu Beginn jeder *Propagation* jeder Knoten einer Fläche zugeordnet werden. Sollte eine Kante getestet werden, können alle Knoten der zugehörigen Fläche betrachtet und ähnlich wie in Abschnitt 5.1 mit Hilfe von *Rechtsknicks* und *Linksknicks* einer Teilhälfte zugeordnet werden. So können alle Kanten der Hülle der Teilhälfte betrachtet werden und der Knoten der Hälfte zugewiesen werden, bei der nur *Rechts-* oder *Linksknicks* vorhanden sind.

Führt das Testen einer Kante zu einem Konflikt, kann das zugehörige Literal l auf *falsch* gesetzt werden. Als Begründung kann die folgende Klausel

$$(\bar{l} \vee \bigvee_{e \in E_A} \bar{x}_e)$$

verwendet werden, wobei E_A die Menge der aktiven Kanten bezeichnet.

6. Auswertung

In diesem Kapitel werden die Ansätze aus Kapitel 3 ausgewertet. Die ersten beiden Experimente testen, ob die Solver bei gleichen Bedingungen die gleichen Ergebnisse liefern. Die beiden folgenden Experimente werden genutzt, um den besten SAT-Solver für DCT aus Abschnitt 5.2 zu finden. Danach folgt in Abschnitt 6.6 ein Experiment, das die verschiedenen Solver mit den verschiedenen Beschränkungen vergleicht, um den besten Solver zu bestimmen. Dieser wird genutzt, um in den folgenden Experimenten zum einen die theoretische Ausschließung von Kanten zu testen, möglichst große Instanzen zu lösen und Ja-Instanzen bzw. Nein-Instanzen getrennt voneinander zu betrachten. Anschließend werden in Abschnitt 6.10 die Zielfunktionen betrachtet. In dem darauf folgenden Experiment wird der CDCL-Ansatz aus Abschnitt 3.3 genauer betrachtet, um während des Lösens Einfluss auf den Solver zu nehmen. Zum Abschluss wird in drei verschiedenen Experimenten versucht, die Schwierigkeit der Instanzen zu bestimmen.

6.1. Versuchsumgebung

In diesem Abschnitt werden die Bedingungen für die Experimente beschrieben. Alle Experimente, die eine Laufzeit beinhalten, wurden entweder auf *AMD Ryzen 7 5800X @ 8x 3.8GHz-4.7GHz* mit *128GB* Arbeitsspeicher oder auf *AMD Ryzen 9 7900 @ 12x 3.7GHz-5.4GHz* mit *96GB* Arbeitsspeicher durchgeführt. Im Folgenden werden in dem Experiment zwischen *Ryzen 7* und *9* unterschieden. Die Ergebnisse werden mit der Python-Bibliothek *Algbench* gespeichert. Dabei wurden für alle Experimente mit Ausnahme von Abschnitt 6.8 ein Timeout von 5 Minuten (300 Sekunden) gesetzt. Die komplette Laufzeit inklusive der Vorbereitungszeit musste in der Timeoutzeit liegen.

Für die Instanzgenerierung wurden die in Kapitel 4 beschriebenen Methoden verwendet. Es wurden Instanzen mit 30 bis 230 Knoten in 10er Schritten generiert, wobei immer zwei Ja- und zwei Nein-Instanzen pro Knotengröße generiert wurden. Bei den Nein-Instanzen wurde jeweils eine *move* und eine *change* Instanz generiert. Im Folgenden werden alle Instanzen nach der Knotengröße aufsteigend sortiert.

Alle Experimente wurden in Python 3.13.5 durchgeführt. Der Algorithmus zum Finden von Überschneidungen wurde in C++ implementiert und mit *pybind11* in Python eingebunden. Ebenso wurde der CDCL-Ansatz in C++ implementiert.

In vielen Experimenten werden sogenannte Kaktus-Diagramme verwendet. Bei diesen wird auf der X-Achse die Zeit und auf der Y-Achse die Anzahl der gelösten Instanzen angezeigt. Bei diesen Diagrammen werden alle Solver mit allen Instanzen *gleichzeitig* gestartet. Wenn ein Solver eine Instanz löst, wird die Linie vertikal um eine Einheit nach oben versetzt. So ergibt sich eine Treppelinie. Bei diesen Diagrammen zeigt die Höhe die

6. Auswertung

Anzahl der gelösten Instanzen an. Analog die Breite die benötigte Zeit. Dementsprechend ist es positiv, wenn eine Linie höher und weiter links ist. Wenn im Folgenden keine Kaktus-Diagramme verwendet werden, werden die genutzten Diagramme genauer erklärt.

Für eine bessere Lesbarkeit sind einige Diagramme nur im Anhang enthalten. Diese sind am Buchstaben im Namen zu erkennen.

6.2. Permutation

In diesem Experiment wird untersucht, ob die Reihenfolge einen Einfluss hat, in der die Knoten den Solvern übergeben werden. Es wurden Instanzen mit 30 bis 100 Knoten betrachtet und auf dem Ryzen 7 getestet. Als Beschränkungen kamen die Überschneidungsbeschränkung und die Gradbeschränkung zum Einsatz. Wie in Abbildung A.1, Abbildung A.2 und Abbildung A.3 zu erkennen ist, unterscheiden sich die Laufzeiten bei jedem Solver in mindestens einer Instanz. Daraus folgt, dass die Permutation der Knotenmenge einen Einfluss auf die Laufzeit hat. In den folgenden Experimenten wird für jeden Durchlauf die Knotenmenge fünfmal in unterschiedlichen Permutationen übergeben und der Durchschnitt berechnet.

6.3. Mehrmalige Durchläufe

In diesem Experiment wird untersucht, ob sich bei mehrfachen Durchläufen mit den selben Instanzen, mit dem selben Solver und gleichen Parametern die Laufzeit verändert. Auch hier wurden Instanzen mit 30 bis 100 Knoten betrachtet. Allerdings wird auf dem Ryzen 9 getestet. Als Beschränkungen kamen die Überschneidungsbeschränkung und die Gradbeschränkung zum Einsatz. Bei allen Solvern variieren die Laufzeiten bei den meisten Durchläufen. Dies ist in Abbildung A.4, Abbildung A.5 und Abbildung A.6 zu erkennen. Da aufgrund von Abschnitt 6.2 schon mehrere Durchläufe durchgeführt wurden, werden die Experimente nicht angepasst.

6.4. SAT Gradbeschränkung

In diesem Experiment werden verschiedene Methoden der Gradbeschränkung in einem SAT-Solver verglichen. Hierbei wurden für alle drei Methoden *subset*, Kardinalität mit Exaktem-Sollgrad und Kardinalität mit Mindest-Sollgrad die Laufzeiten gemessen. Bei den Kardinalitätscodierungen von PySat wurden auch alle möglichen Codierungen getestet. Das Ziel dieser Optimierung ist es die Codierung mit der besten Laufzeit zu finden, die dann in den folgenden Experimenten verwendet wird. Für die Knotengröße wurden Instanzen der Größe 30 bis 90 verwendet und auf dem Ryzen 7 getestet. Bei der *subset* Methode wurde nur die Instanz mit 30 Knoten verwendet, da bei größeren Knotenmengen der zur Verfügung stehende Arbeitsspeicher zu klein war. Als zusätzliche Bedingung wurde die Überschneidungsbeschränkung verwendet.

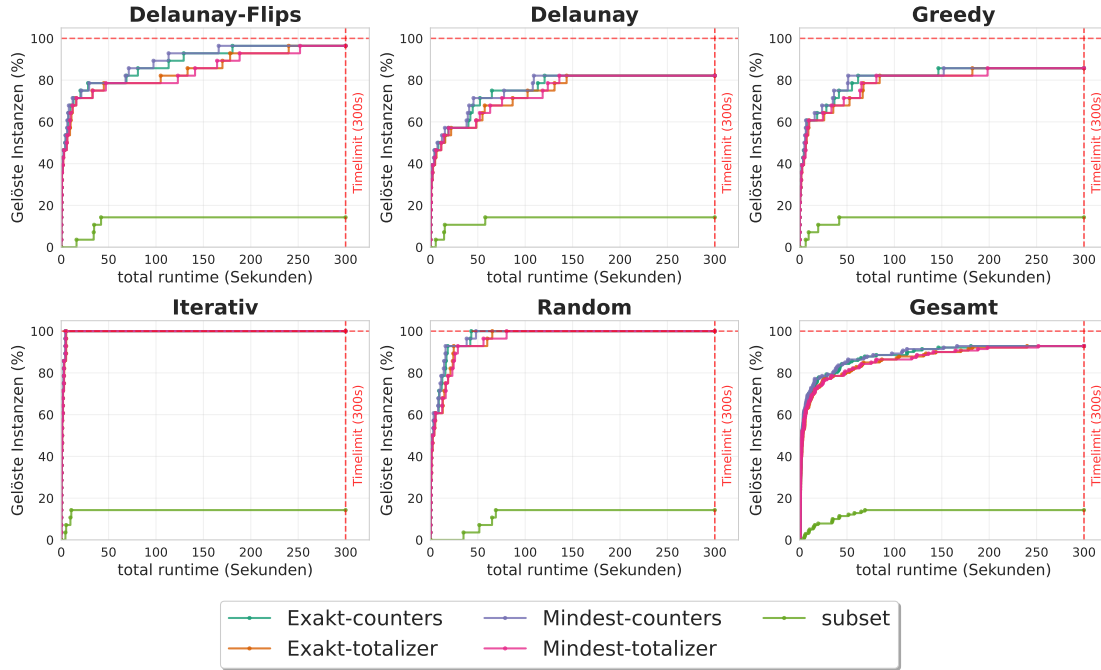


Abbildung 6.1.: In der Abbildung werden für alle Instanzenarten die drei Ansätze verglichen. Dabei werden für die Exakte-Methode und die Mindest-Methode *sequential counters* und *totalizer* genutzt. Die *subset* Methode ist die schlechteste, da sie nur für kleine Instanzen funktioniert. Es zeigt sich, dass die Exakte-Methode sowie die Mindest-Methode bei *sequential counters* und *totalizer* sehr ähnliche Ergebnisse erzielen. *Sequential counters* ist dabei eindeutig schneller als *totalizer*. Im Gesamtüberblick lässt sich erkennen, dass die Mindest-Methode mit *sequential counters* minimal schneller ist als die Exakte-Methode mit *sequential counters*.

In Abbildung A.7 ist zu erkennen, dass *sequential counters* und *totalizer* bei der Exakten-Methode sowie bei der Mindest-Methode die besten Ergebnisse erzielen. Diese werden in Abbildung 6.1 erneut verglichen. Hier ist zu sehen das *sequential counters* die besten Ergebnisse erzielt. Da die Exakte-Methode etwas besser ist, wird diese in den folgenden Experimenten verwendet.

6.5. SAT-Solver Vergleich

In diesem Experiment werden die verschiedenen SAT-Solver verglichen, die von PySat bereitgestellt werden.

PySat stellt *Cadical195*, *Gluecard4*, *Glucose42*, *Lingeling*, *MapleChrono*, *MapleCM*, *Maplesat*, *Mergesat3*, *Minicard* und *Minisat22* zur Verfügung. Von diesen wurden jeweils die neuesten verfügbaren Versionen verwendet. Fast alle Solver nutzen die *sequential*

6. Auswertung

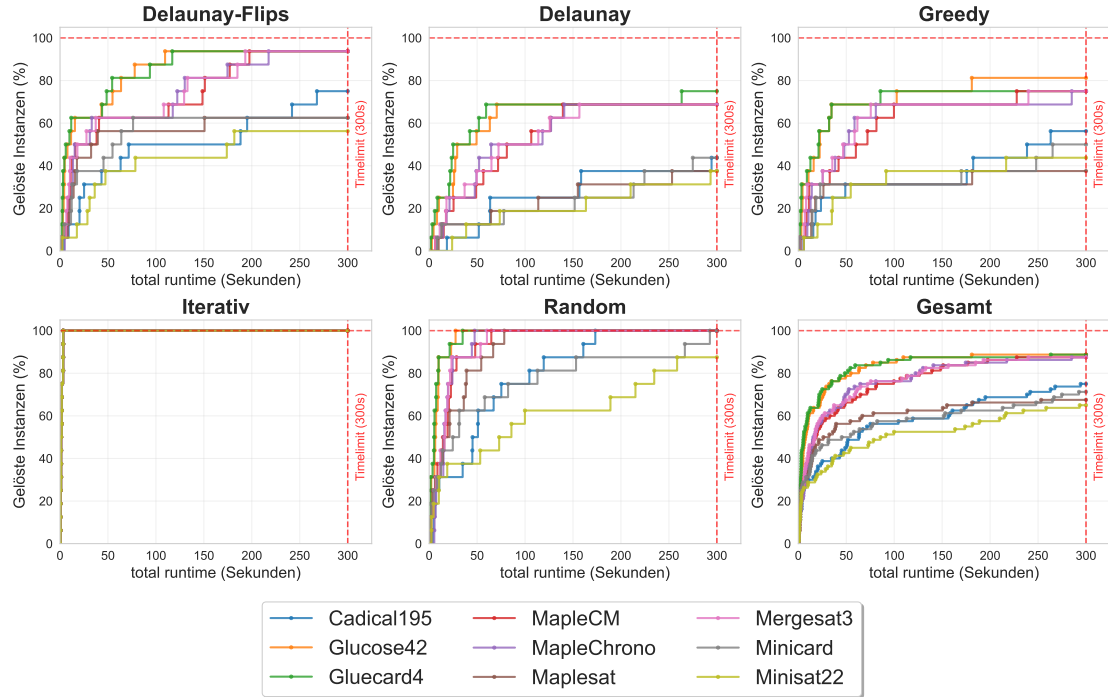


Abbildung 6.2.: In dieser Abbildung sind die Laufzeiten der verschiedenen SAT-Solver aus PySAT zu sehen. Hierbei ist klar erkennbar, dass *Gluecard4* und *Glucose42* die besten Ergebnisse erzielen. Dass diese beiden Solver ähnliche Ergebnisse liefern ist nicht verwunderlich, da sie beide auf dem Glucose-Algorithmus basieren. Es zeigt sich aber, dass bei diesem Problem weder der native noch der *sequential counters* Ansatz einen signifikanten Unterschied machen. Jedoch ist zu erkennen, dass *Glucose42* bei Greedy eine Instanz in guter Zeit lösen konnte, was *Gluecard4* nicht geschafft hat.

counters Codierung, sowie die Überschneidungsbeschränkung. Bei *Gluecard4* und *Minicard* kam anstelle von *sequential counters* eine native Codierung der Gradbeschränkung zum Einsatz. Diese wird nur für diese beiden Solver angeboten und bietet eine direkte Codierung ohne zusätzliche Variablen. Für die Instanzgröße wurden 60 bis 90 Knoten gewählt, die auf dem Ryzen 9 getestet wurde. Das Ziel besteht darin, den Solver zu identifizieren, der die besten Ergebnisse liefert, um diesen in zukünftigen Experimenten einzusetzen. In Abbildung 6.2 ist zu erkennen, dass *Gluecard4* und *Glucose42* die besten Ergebnisse erzielen. Für den weiteren Verlauf wird *Glucose42* verwendet, da dieser bei Greedy etwas besser war. Der *Lingeling*-Solver wurde in großen Experimenten nicht berücksichtigt, da er kein Timeout unterstützt. Er erzielt jedoch schlechtere Ergebnisse als *Glucose42*.

6.6. Vergleich der Modellierungstechniken

In diesem Experiment werden alle möglichen Bedingungen an die Solver übergeben und einzeln getestet. Das Ziel ist es, den besten Solver zu finden.

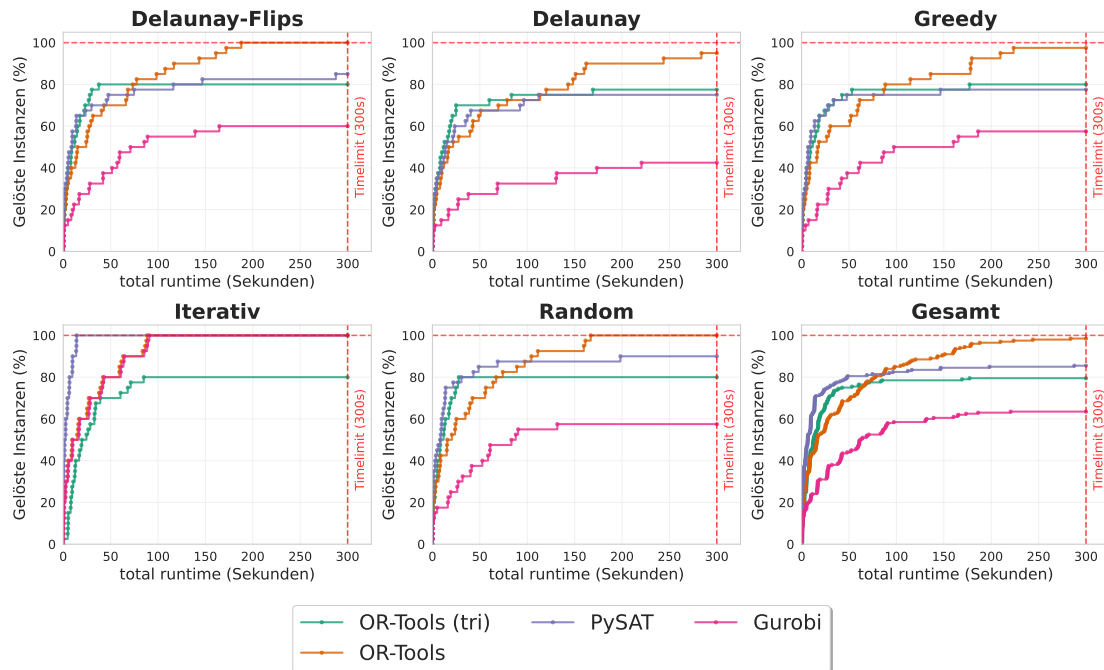


Abbildung 6.3.: In dieser Abbildung werden für die vier verschiedenen Ansätze die besten Solver verglichen. Für PySAT und Gurobi ist die beste Methode das Fixieren der Kanten bei der Kantenformulierung. Bei OR-Tools hingegen ist für die Kantenformulierung das fixieren der Kanten und der Hülle sowie die alternative Kanten Bedingung am besten. Für die Dreiecksformulierung wurde OR-Tools (tri) ohne weiteren Bedingungen ausgewählt. Insgesamt erzielt OR-Tools die besten Ergebnisse, gefolgt von PySAT. Gurobi ist in allen Fällen mit Abstand am schlechtesten. OR-Tools (tri) ist besser als Gurobi aber schlechter als die andern beiden. PySAT ist in Instanzen wie Iterativ allerdings teilweise besser als OR-Tools. Auch erkennbar ist das OR-Tools (tri) in allen Instanzen ähnliche Werte erzielt. Generell lässt sich sagen das OR-Tools die besten Ergebnisse erzielt. Allerdings sind bei kleineren Knotengrößen SAT und OR-Tools (tri) besser.

Untersucht werden die Solver PySAT, OR-Tools und Gurobi jeweils mit dem Kantenansatz sowie dem Dreiecksansatz. Im Folgenden werden Solver mit dem Dreiecksansatz ein (tri) hinter dem Namen haben. Für alle Durchläufe wurden die Überschneidungsbeschränkung und die Gradbeschränkung genutzt. Bei dem Kantenansatz wurden das

6. Auswertung

Fixieren und Ausschließen von Kanten, sowie die alternative Kantenformulierung getestet. Bei dem Dreiecksansatz wurde nur das Ausschließen von Kanten getestet. Die Instanzgröße liegt bei 30 bis 120 Knoten. Diese wurde auf dem Ryzen 7 getestet.

In Abbildung A.11 und Abbildung A.9 ist zu erkennen, dass Gurobi und PySAT am besten mit dem Fixieren der Kanten optimiert werden. Bei OR-Tools hingegen erzielt die alternative Kantenformulierung im Zusammenhang mit dem Fixieren der Kanten und der Hülle die besten Ergebnisse, wie in Abbildung A.10 zu sehen ist. Interessant hierbei ist, dass bei OR-Tools die Kombination aus drei Bedingungen am besten ist und bei PySAT und Gurobi die gleiche Kombination schlechter ist, als nur das Fixieren der Kanten.

In Abbildung A.8 ist zu sehen, dass OR-Tools (tri) ohne weitere Bedingungen die besten Ergebnisse erzielt. Auch lässt sich erkennen, dass bei OR-Tools (tri) bzw. Gurobi (tri) alle Instanzen gleich schwierig wirken. So sind alle Instanzen bis ca. 75 % bzw. 65 % relativ schnell gelöst, aber danach werden nicht mehr viele Instanzen gelöst.

In Abbildung 6.3 ist zu sehen das OR-Tools die besten Ergebnisse erzielt, gefolgt von PySAT. Nur bei Iterative ist PySAT besser. Interessant bei der Dreiecksformulierung ist, dass bei allen Instanzarten die gleichen Knotenmengen in der gleichen Zeit geschafft werden. Dies könnte daran liegen, dass die Vorbereitungszeit ab einer bestimmten Knotengröße zu lange dauert. Um dies genauer zu untersuchen, wurden für die besten Solver nur die Solverzeiten ohne die Vorbereitungszeiten betrachtet. In Abbildung A.12 ist erkennen, dass die gleichen Ergebnisse erzielt werden. Dementsprechend hat die Vorbereitungszeit keinen Einfluss auf das Ergebnis. Im Folgenden wird OR-Tools mit fixierten Kanten, sowie Hülle und der alternativen Kanten Bedingung als bester Solver verwendet.

6.7. Kanten vorab berechnen

In Abschnitt 6.6 wurde festgestellt, dass das Ausschließen und Fixieren von Kanten nur schlechte Ergebnis liefert. Die Hypothese dazu ist, dass zu wenig Kanten ausgeschlossen bzw. fixiert wurden. Um dies zu überprüfen, wird vor dem Experiment eine Menge von Kanten bestimmt, die sicher ausgeschlossen oder fixiert werden können. Somit können während des Lösungsprozess beliebig viele Kanten ausgeschlossen oder fixiert werden. In diesem Experiment wurde OR-Tools mit der Überschneidungsbeschränkung und Gradbeschränkung ausgewählt. Für die Knotengröße wurden Instanzen mit 30 bis 80 Knoten gewählt und das Experiment wurden auf dem Ryzen 9 durchgeführt. Um das Experiment zu vereinfachen, wurden nur Instanzen betrachtet, die genau eine Lösung besitzen. Somit konnte eine Lösung der Instanz genommen und mit dieser die Kanten bestimmt werden, die fixiert oder ausgeschlossen werden sollen. Anhand von Abbildung A.13 und Abbildung 6.4 lässt sich bestätigen, dass das Ausschließen und Fixieren der Kanten in Abschnitt 6.6 schlechter war, weil zu wenig Kanten ausgeschlossen bzw. fixiert wurden. Es ist überraschend, dass es beim Fixieren keinen Unterschied macht, ob 50 oder 80 % der Kanten fixiert wurden. Dass dies beim Ausschließen nicht der Fall ist, lässt sich damit erklären, dass zum einen mehr Kanten theoretisch ausgeschlossen werden können und somit der absolute Unterschied größer ist. Zum anderen, dass beim Fixieren so viele Kanten durch den Solver indirekt durch andere Bedingungen ausgeschlossen

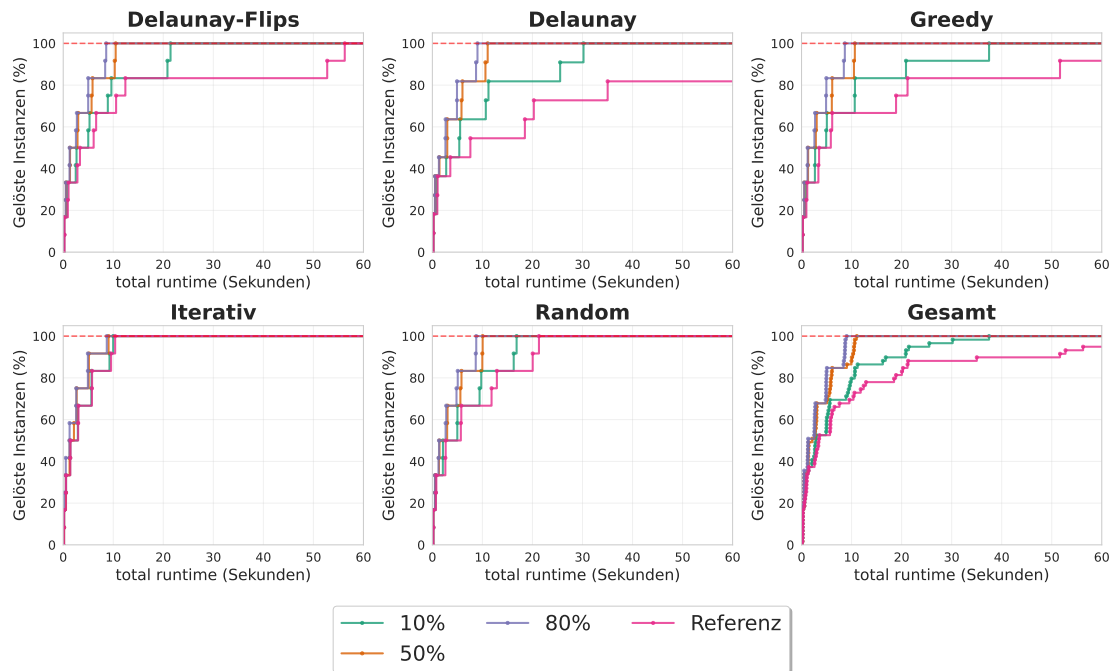


Abbildung 6.4.: In diesem Experiment wird die theoretische Ausschließung von Kanten untersucht. Für das Experiment wurde OR-Tools mit Grad- und Überschneidungsbeschränkung als Referenz und zusätzlich OR-Tools mit 10 %, 50 % und 80 % der benötigten Kanten ausgeschlossen verwendet. In dem Diagramm wurde der betrachtete Zeitraum auf 60 Sekunden begrenzt, da hier alle relevanten Informationen zu sehen sind. Es ist klar zu erkennen, dass der Referenz-Solver die schlechtesten Ergebnisse erzielt. Auch die anderen Solver sind in der erwarteten Reihenfolge, also nach der Menge der ausgeschlossenen Kanten sortiert. Nur 50 % und 80 % sind deutlich besser als die anderen beiden Ansätze.

werden können, dass ab einer bestimmten Menge an fixierten Kanten die resultierenden Teilinstanzen gleich groß sind.

6.8. Größerer Timeout

In diesem Experiment wird geprüft, welche Instanzen der Solver lösen kann, wenn das Zeitlimit von 5 Minuten auf 30 Minuten erhöht wird. Dafür wurden Instanzen mit zunehmender Größe generiert, bis der Solver diese nicht mehr lösen konnte. Die Knotenzahl der Instanz lag am Ende zwischen 80 und 230 und wurde auf dem Ryzen 9 getestet. Es wird erneut der beste Solver aus Abschnitt 6.6 verwendet. In Abbildung A.14 ist zu erkennen, dass der Solver Instanzen mit bis zu 220 Knoten lösen konnte. Allerdings handelt es sich hierbei um iterative Instanzen. Alle Instanzen konnten bis mindestens 170

6. Auswertung

Knoten gelöst werden. Zusätzlich ist interessant, dass die meisten großen Instanzen nicht wegen eines Timeouts fehlgeschlagen sind, sondern wegen fehlendem Arbeitsspeicher.

6.9. Ja/Nein getrennt

In diesem Experiment wird untersucht, ob der Solver bei Ja-Instanzen oder bei Nein-Instanzen schneller ist. Dafür werden die Daten aus Abschnitt 6.8 wiederverwendet. In Abbildung A.15 ist zu erkennen, dass beide Instanzenarten bis zur Knotengröße 220 gelöst werden konnten. Allerdings werden die Nein-Instanzen generell schneller als die Ja-Instanzen gelöst. Interessant ist, dass die Nein-Instanzen bis 220 Knoten gelöst wurden, aber für 200 Knoten keine Lösung gefunden wurde.

6.10. Zielfunktion

In diesem Experiment werden die beiden Zielfunktionen aus Abschnitt 3.2 verglichen. Zusätzlich wird ein Referenz Solver verwendet, mit diesem kann verglichen werden, ob die Zielfunktionen schneller eine Entscheidung treffen können. Es sei noch zu erwähnen, dass der Flips-Ansatz vorab getestet wurde. Die größte Instanz, die dieser Ansatz lösen konnte, bestand aus maximal 8 Knoten. Da die anderen beiden Zielfunktionen auf Instanzen mit 120 Knoten getestet werden, ist der Flips Ansatz nicht mehr relevant. Das Experiment wurde auf dem Ryzen 9 durchgeführt.

Im Folgenden werden Ja-Instanzen und Nein-Instanzen getrennt betrachtet. Bei Ja-Instanzen ist interessant, wie schnell die Instanz gelöst werden kann und ab wann gute Ergebnisse erzielt werden. Bei Nein-Instanzen ist hingegen interessant, wie nah der Solver an eine gültige Lösung kommen kann. Für dieses Experiment wurde OR-Tools mit den in Abschnitt 6.6 gefundenen Bedingungen ausgewählt. Der Referenz Solver wird hierbei komplett aus Abschnitt 6.6 übernommen. Für die Zielfunktionen wird die Gradbeschränkung durch die Zielfunktion ersetzt.

In Abbildung A.16 zeigt sich, dass bei Ja-Instanzen der Min-Max-Ansatz besser abschneidet. Bei Nein-Instanzen erzielt der Durchschnittsansatz die besseren Ergebnisse, was in Abbildung 6.5 dargestellt ist. Es ist nicht erkennbar, dass die Zielfunktionen oder der Referenz-Solver in einer der Instanzen eindeutig besser sind. Es ist auch anzumerken, dass nur auf zwei ausgewählten Instanzen getestet wurde und die Unterschiede nicht sehr groß sind. Abschließend lässt sich festhalten, dass die Zielfunktionen keine schnelleren Entscheidungen treffen als der Referenz Solver. Für die beiden Zielfunktionen lässt sich festhalten, dass für Ja-Instanzen der Min-Max-Ansatz besser ist. Für Nein-Instanzen schafft der Durchschnittsansatz bei Greedy 97 % zu erreichen und ist auch generell besser.

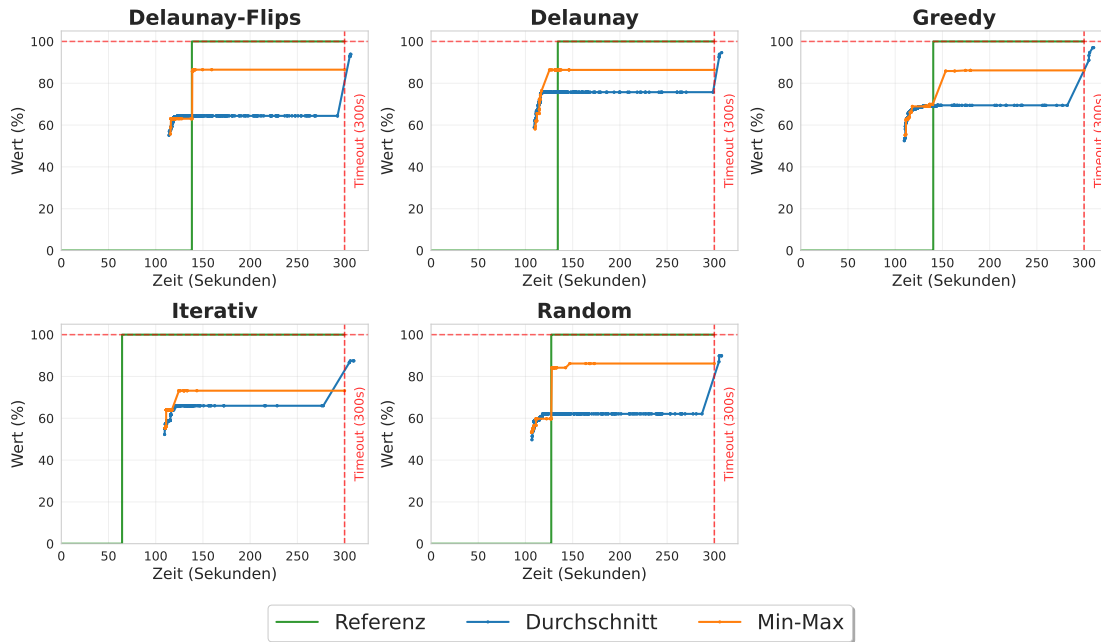


Abbildung 6.5.: Eine Nein-Instanz mit 120 Knoten. Hier werden zwei verschiedene Zielfunktionen mit einem Referenzsolver aus Abschnitt 6.6 verglichen. Es ist zu erkennen, dass der Durchschnittsansatz in den meisten Fällen bessere Ergebnisse liefert, als der Min-Max-Ansatz. Oft findet er erst beim Timeout eine Verbesserung, diese ist jedoch besser als die des Min-Max-Ansatzes. In den meisten Fällen stagniert der Min-Max-Ansatz bei etwas über 80 %. Der Min-Max-Ansatz bricht vermutlich früher ab, da er aus seiner Sicht ein Optimum gefunden hat. Der Referenzsolver erkennt nur bei Greedy und Iterativ früher, dass es sich um eine Nein-Instanz handelt. In allen anderen Fällen benötigt er mindestens genauso lange wie der Min-Max-Ansatz. Abschließend lässt sich festhalten, dass der Durchschnittsansatz bei Greedy eine Lösung mit einer Bewertung von 97 % schafft.

6.11. Propagation CDCL

In diesem Experiment wird der CDCL Ansatz genauer betrachtet. Dafür wird zum einen die Belegung der Literale bei den *Propagation* Aufrufen visualisiert. Zum anderen werden die Ansätze aus Abschnitt 3.3 genauer betrachtet.

6.11.1. Visualisierung

Der erste Schritt war es, die Teilinstanzen, mit denen die *Propagation* aufgerufen wird, zu visualisieren. In Abbildung 6.6 ist eine beispielhafte Teilinstanz dargestellt. Bei

6. Auswertung

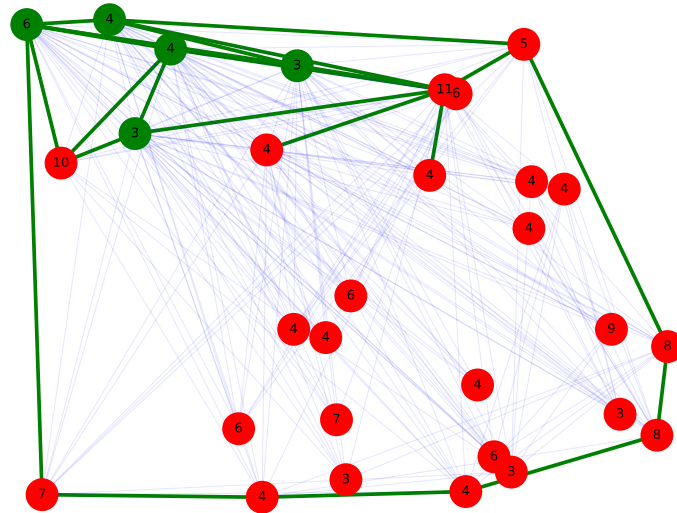


Abbildung 6.6.: Eine Teilinstanz bei einem *Propagation* Aufruf. Blaue Kanten wurden auf *False* gesetzt, grüne Kanten auf *True*. Die Zahlen in den Knoten geben den Sollgrad an. Sollte ein Knoten grün sein, hat er seinen Sollgrad erreicht, bei rot noch nicht. Es ist zu erkennen, dass sich ein Konstrukt von oben links nach unten rechts vorarbeitet. Hier ist der grüne Teil bereits fertig trianguliert und viele Kanten wurden ausgeschlossen.

der Visualisierung wurde festgestellt, dass es häufig zwei Arten von Teilinstanzen gibt. Die erste Variante bildet ein Konstrukt, das eine Verbindung zum Rand besitzt. In diesem Konstrukt ist meist eine gültige DCT vorhanden. Die zweite Variante besteht aus einzelnen Kanten, die isoliert in einer Fläche liegen. Es lässt sich also sagen, dass der Ansatz aus Abbildung 6.6 besser für Variante eins ist. Hier kann ausgenutzt werden, dass durch das Konstrukt, das in die Mitte der Instanz hineinragt, die Instanz leicht geteilt werden kann. Für die zweite Variante müsste der Ansatz erweitert werden, sodass zwei Kanten eingefügt werden, damit die Instanzen halbiert werden können.

6.11.2. Relative Anzahl erfolgreicher Propagation

In diesem Experiment wird betrachtet, wie viele erfolgreiche *Propagations*, also die, die eine Kante ausschließen können, in einer Instanz möglich sind. Dafür werden alle erfolgreichen Aufrufe gezählt und diese Zahl durch die Anzahl der *Propagation* Aufrufe geteilt. Dabei wird in diesem Experiment nicht die Laufzeit gemessen, da durch die vielen

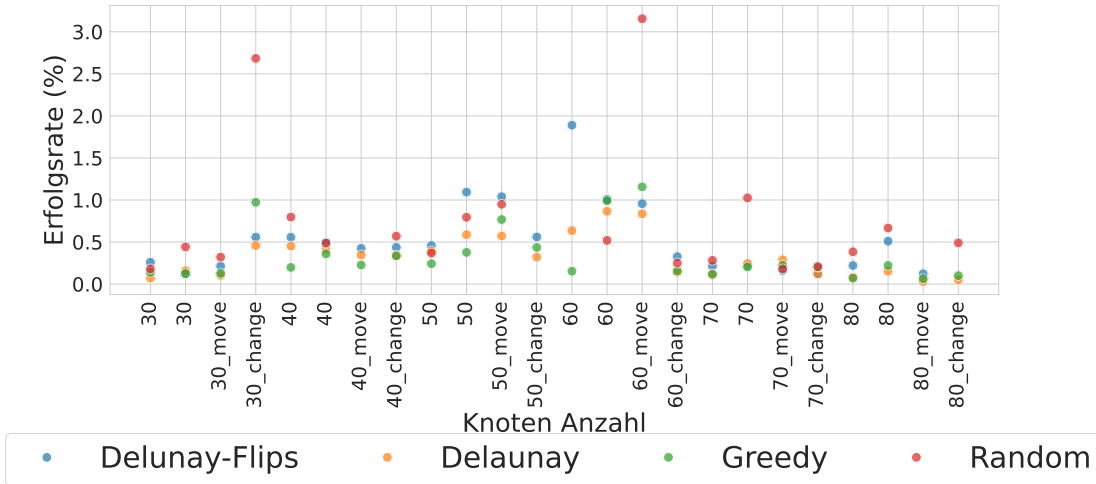


Abbildung 6.7.: In der Abbildung ist die relative Anzahl der erfolgreichen *Propagations* dargestellt. Dafür wird auf der Y-Achse die Anzahl der erfolgreichen *Propagations* geteilt durch die Anzahl der *Propagation* Aufrufe mal 100 dargestellt. Auf der X-Achse sind die Knotengrößen aufsteigend dargestellt. Dabei gibt es immer zwei Ja-Instanzen und zwei Nein-Instanzen pro Knotengröße. Bei den Nein-Instanzen wurde jeweils eine *move* und eine *change* Instanz genutzt. Es fällt auf, dass vor allem Random Instanzen sehr viele erfolgreiche *Propagations* haben. Greedy und Delaunay hingegen haben nur wenige erfolgreiche *Propagations*. Iterativ hatte keine erfolgreichen *Propagations*, da die Instanzen anscheinend zu einfach sind. Delaunay-Flips ist zwischen Random und den anderen beiden einzuordnen. Es fällt auf, dass vor allem bei den kleinen Instanzen erfolgreiche *Propagations* stattfinden. Die beste Erfolgsrate hat mit etwa 3 % Random bei einer Nein-Instanz mit 60 Knoten.

getesteten Kanten, die Laufzeit stark negativ beeinflusst wird. Eine *Propagation* soll im Normalfall sehr schnell entscheiden, welche Literale mit der aktuellen Belegung noch gesetzt werden können. In diesem Experiment geht es nur darum zu bestimmen, ob es sinnvoll scheint, eine *Propagation* mit diesem Ansatz zu nutzen. Bei diesem Experiment wurden Instanzen der Größe 30 bis 80 Knoten betrachtet. Das Experiment wurde auf dem Ryzen 9 durchgeführt.

In Abbildung 6.7 ist die relative Anzahl pro Instanz zu sehen. Dabei fällt auf, dass vor allem Random Instanzen sehr viele erfolgreiche *Propagations* haben. Delunay-Flips liegt in der Mitte vor Greedy und Delaunay. Iterativ hat keine erfolgreiche *Propagation*. Abschließend kann keine feste Aussage getroffen werden, wie sinnvoll die *Propagation* ist. Dafür müsste ein laufzeitenbasiertes Experiment genutzt werden. Dies ist aber aufgrund der langsamen *Propagation* nicht sinnvoll. Es lässt sich festhalten, dass Kanten damit ausgeschlossen werden können und vor allem für Random der Ansatz Erfolge erzielt.

6.12. Schwierigkeit der Instanzen

In diesem Abschnitt wird versucht die Schwierigkeit der Instanzen zu bestimmen. Aus den vorherigen Experimenten ist bekannt, dass für die Kantenformulierung *Delaunay* und *Greedy* die schwierigeren Instanzen, *Delaunay Flips* mittlere und *Iterativ* nach *Random* die einfachsten Instanzen sind. Im ersten Teil wird untersucht, ob die Anzahl der Lösungen pro Instanz einen Einfluss auf die Schwierigkeit hat. Danach wird im zweiten Teil bewertet, wie der Grad verteilt ist, was bedeutet, wie oft jeder Grad vertreten wird. Im letzten Teil wird untersucht, ob ein Zusammenhang zu den Kantenlängen besteht. Für alle drei Teile werden Instanzen der Größe 30 bis 120 Knoten betrachtet.

6.12.1. Anzahl Lösungen

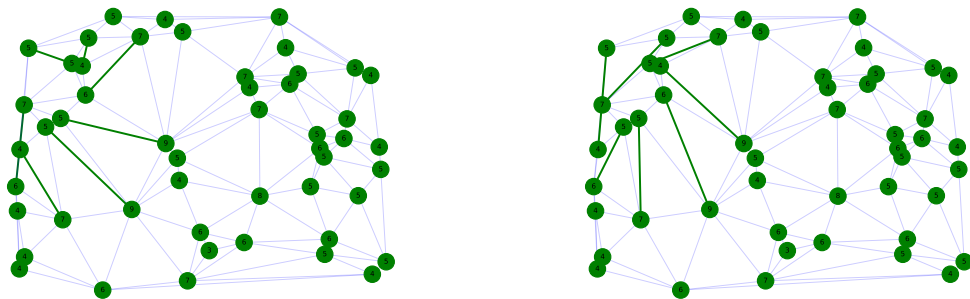


Abbildung 6.8.: In dem Bild sind zwei verschiedene Lösungen für eine Instanz dargestellt. In grün sind die Kanten zu sehen, die in der Instanz unterschiedlich sind. In blau die gleichen Kanten. Die Zahlen in den Knoten geben den Sollgrad an.

In Tabelle A.1 ist zu sehen, dass fast alle Instanzen nur eine Lösung haben. Vor allem ist aber zu sehen, dass es keinen Zusammenhang zwischen der Anzahl der Lösungen und der Instanzart gibt. Es kann also davon ausgegangen werden, dass die Anzahl der Lösungen keinen Einfluss auf die Schwierigkeit der Instanzen hat. Interessant ist dabei, dass die Kanten, die in einer Instanz unterschiedlich sind, klar einem Bereich zugeordnet werden können. Wie in Abbildung 6.8 zu erkennen ist, sind die Kanten in der linken Hälfte der Instanz unterschiedlich, während die rechte Hälfte gleich bleibt.

6.12.2. Gradverteilung

In diesem Experiment wird anhand der Gradverteilung der Knoten untersucht, ob es einen Zusammenhang zwischen der Schwierigkeit und dem Grad gibt. Dafür werden die Ja-Instanzen der Größe 30 bis 120 Knoten betrachtet. In Abbildung 6.9 ist die Gradverteilung der Knoten zu sehen. Es lässt sich vermuten, dass es einen Zusammenhang zwischen

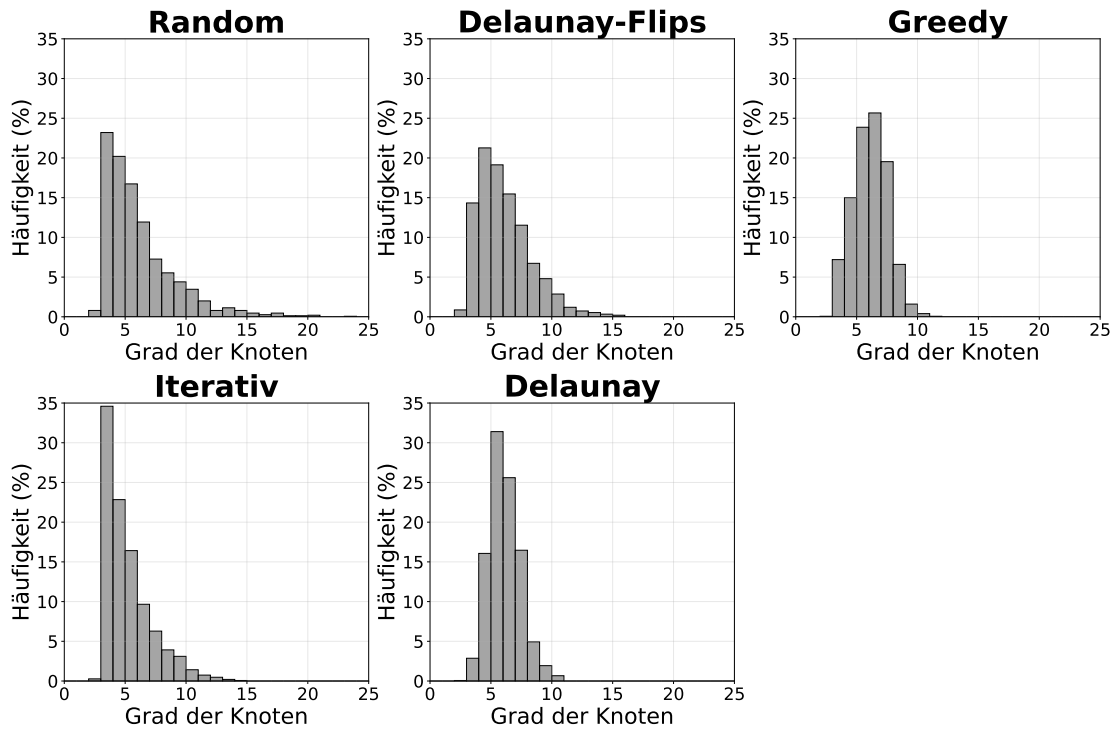


Abbildung 6.9.: In dieser Abbildung ist die Gradverteilung der Knoten pro Instanz zu sehen. Dabei wird auf der X-Achse der Grad und auf der Y-Achse die Häufigkeit der Knoten mit diesem Grad dargestellt.

der Schwierigkeit und dem Grad gibt. Es ist zu erkennen, dass vor allem Random und Iterativ eine rechtsschiefe Verteilung haben. Dazu passend haben Delaunay und Greedy eher eine zentrierte Verteilung. Auch das Delaunay-Flips rechtsschiefer ist, als Delaunay und Greedy ist würde passen, da Delaunay-Flips einfacher als Delaunay ist. Es lässt sich also anhand dieser Beispiele vermuten, dass einfachere Instanzen eher kleinere Grade und schwierigere Instanzen mittlere Grade haben.

6.12.3. Verteilung der Kantenlängen

In diesem Experiment soll ein Zusammenhang zwischen der Schwierigkeit und einer Eigenschaft gefunden werden. In diesem Fall werden die Kantenlängen betrachtet. Allerdings müssen diese normiert werden, da sonst weit auseinander liegende Knoten die Verteilung verzerren würden. Die Normierung erfolgt durch das Teilen durch die maximale Kantenlänge. Das heißt, dass für jede Instanz die maximale Kantenlänge bestimmt und alle Kantenlängen durch diese geteilt werden. Für dieses Experiment wurden nur Ja-Instanzen der Größe 30 bis 80 Knoten betrachtet. Auch hier lässt sich eine Verbindung zwischen der Schwierigkeit und der Verteilung in Abbildung 6.10 erkennen. Leichte Instanzen weisen eher lange Kanten auf, als schwere Instanzen. So sind bei schweren Instanzen wie Greedy

6. Auswertung

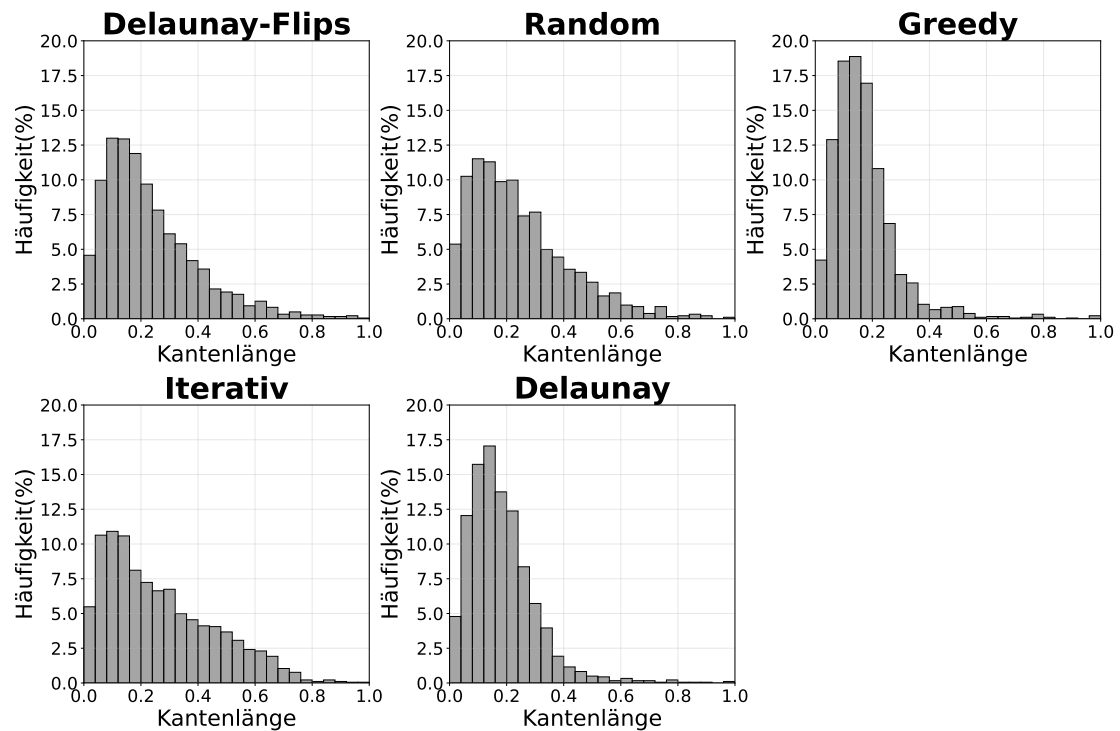


Abbildung 6.10.: In dieser Abbildung ist die Verteilung der Kantenlängen pro Instanz zu sehen. Dabei sind die Kantenlängen normiert, sodass sie zwischen 0 und 1 liegen. Auf der X-Achse ist die normierte Kantenlänge und auf der Y-Achse die Häufigkeit der Kanten mit dieser Länge zu sehen.

und Delaunay die Kantenlängen eher links, was bedeutet, dass diese eher kurze Kanten besitzen. Leichte Instanzen wie Iterativ hingegen haben deutlich weniger kurze Kanten und dafür mehr längere Kanten.

6.12.4. Zusammenfassung

In diesem Abschnitt wurde festgestellt, dass die Anzahl der Lösungen keinen Einfluss auf die Schwierigkeit hat. Die Gradverteilung und die Kantenlängenverteilung hingegen scheinen einen Einfluss auf die Schwierigkeit für die Kantenformulierung zu haben. Hierbei deuten viele kurze Kanten und breit verteilte Grade auf eine schwere Instanz hin. Wohingegen verteilte längere Kanten und wenige hohe Grade auf eine leichte Instanz hinweisen.

7. Fazit und Ausblick

In dieser Arbeit wurde das neuartige Problem DCT untersucht. Dabei erfolgte die Betrachtung sowohl als Entscheidungsproblem als auch Optimierungsproblem. Für das Entscheidungsproblem wurden zwei Modellierungen entwickelt, eine kantenbasierte und eine dreiecksbasierte Formulierung. Darüber hinaus wurden fünf verschiedene Instanztypen konstruiert, um sowohl lösbare als auch unlösbare Fälle zu erzeugen. Bei der kantenbasierten Modellierung zeigte sich, dass OR-Tools in Kombination mit der Fixierung der Hülle sowie ausgewählter möglicher Kanten und unter Einbezug der alternativen Kantenbeschränkung die besten Ergebnisse erzielte. Mit diesem Ansatz konnten Instanzen mit einer Knotenanzahl von bis zu 220 gelöst werden. Für jede getestete Instanzart konnten mindestens 170 Knoten erfolgreich gelöst werden.

Auch bei der dreiecksbasierenden Modellierung lieferte OR-Tools die besten Resultate. Es konnte festgestellt werden, dass diese Modellierung für alle Instanztypen, unabhängig von ihrer vermeintlichen Schwierigkeit, vergleichbare Ergebnisse erzielte. Zusätzlich wurde mit dem CDCL-Ansatz eine theoretische Möglichkeit aufgezeigt, den Lösungsprozess weiter zu optimieren. Dennoch bleibt die Skalierbarkeit auf große Instanzen eine zentrale Herausforderung, da die vorgestellten Verfahren insbesondere für kleinere Knotenmengen praktikabel sind. Darüber hinaus zeigte sich, dass die Struktur der Instanzen, insbesondere die Verteilung der Kantenlängen und Knotengrade, bei der kantenbasierten Formulierung einen erheblichen Einfluss auf die Schwierigkeit der Instanz hat.

Ein vielversprechender Ausblick besteht darin, die vorgestellten Verfahren im Hinblick auf ihre Skalierbarkeit zu verbessern. Eine interessante Möglichkeit wäre, den Arbeitsspeicherverbrauch der Ansätze systematisch zu vergleichen, da dieser insbesondere bei großen Instanzen zu Problemen geführt hat. Ebenso könnte die dreiecksbasierte Modellierung genauer untersucht werden, um zu verstehen, weshalb alle Instanztypen als ähnlich schwierig erschienen. Genauso interessant wäre es zu untersuchen, warum bei der Kantenbedingung die Optimierungen bei OR-Tools in Kombination geholfen haben und bei PySAT und Gurobi nur einzeln. Darüber hinaus bietet der *Propagation*-Ansatz weiteres Potenzial, da hierbei die Eigenschaft genutzt werden könnte, dass Kanten isoliert in Teilinstanzen auftreten. So können diese Kanten genutzt werden, um mit zwei weiteren die Instanz zu halbieren. Schließlich wäre es interessant, bei der Instanzgenerierung normalverteilte Triangulierungen zu berücksichtigen, um einheitlichere und realistischere Szenarien abzubilden.

Ein interessante Abwandlung des Problems wäre es, wenn in der Triangulierung Kanten vorgegeben sind, die immer genutzt werden müssen. Dies wurde in Abschnitt 6.7 teilweise betrachtet, aber nicht als eigenes Problem. Dieses Problem könnte man noch weiter abwandeln, indem man keine Knotenmengen, sondern Polygone mit und ohne Löcher nutzt. Auch könnte man das in Abschnitt 1.3.3 genannte *Min-Degree Constrained*

7. Fazit und Ausblick

Minimum Spanning Tree abwandeln, sodass der Sollgrad exakt und nicht als Mindestgrad betrachtet wird.

Literaturverzeichnis

- [1] Roberto Asín, Robert Nieuwenhuis, Albert Oliveras, and Enric Rodríguez-Carbonell. Cardinality networks and their applications. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 167–180. Springer, 2009.
- [2] Olivier Bailleux and Yacine Boufkhad. Efficient cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 108–122. Springer, 2003.
- [3] Kenneth E Batchner. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, pages 307–314, 1968.
- [4] Armin Biere. Arminbiere/cadical: Cadical sat solver, 2019. URL: <https://github.com/arminbiere/cadical>.
- [5] Prosenjit Bose and Ferran Hurtado. Flips in planar graphs. *Computational Geometry*, 42(1):60–80, 2009. URL: <https://www.sciencedirect.com/science/article/pii/S0925772108000370>, doi:10.1016/j.comgeo.2008.04.001.
- [6] Basudeb Datta and Subhojoy Gupta. Degree-regular triangulations of surfaces. *arXiv preprint arXiv:1711.01247*, 2017.
- [7] Ana Maria de Almeida, Pedro Martins, and Maurício C de Souza. Min-degree constrained minimum spanning tree problem: complexity, properties, and formulations. *International Transactions in Operational Research*, 19(3):323–352, 2012.
- [8] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Cheong Schwarzkopf. *Delaunay Triangulations*, pages 183–210. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. doi:10.1007/978-3-662-04245-8_9.
- [9] Sándor P Fekete, Phillip Keldenich, and Michael Perk. Exact algorithms for minimum dilation triangulation. *arXiv preprint arXiv:2502.18189*, 2025.
- [10] Sándor P Fekete, Samir Khuller, Monika Klemmstein, Balaji Raghavachari, and Neal Young. A network-flow technique for finding low-weight bounded-degree spanning trees. *Journal of Algorithms*, 24(2):310–324, 1997.
- [11] Laxmi P Gewali and Bhaikaji Gurung. Degree aware triangulation of annular regions. In *Information Technology-New Generations: 15th International Conference on Information Technology*, pages 683–690. Springer, 2018.

- [12] F. Hurtado, M. Noy, and J. Urrutia. Flipping edges in triangulations. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry*, SCG '96, page 214–223, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237218.237367.
- [13] J. Knowles and D. Corne. A new evolutionary approach to the degree-constrained minimum spanning tree problem. *IEEE Transactions on Evolutionary Computation*, 4(2):125–134, 2000. doi:10.1109/4235.850653.
- [14] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 6: Satisfiability*. Addison-Wesley Professional, 1st edition, 2015.
- [15] Mohan Krishnamoorthy, Andreas T. Ernst, and Yazid M. Sharaiha. Comparison of algorithms for the degree constrained minimum spanning tree. *Journal of Heuristics*, 7(6):587–611, 2001. doi:10.1023/A:1011977126230.
- [16] Antonio Morgado, Alexey Ignatiev, and Joao Marques-Silva. Mscg: Robust core-guided maxsat solving: System description. *Journal on Satisfiability, Boolean Modelling and Computation*, 9(1):129–134, 2014.
- [17] Oleg R Musin. Properties of the delaunay triangulation. In *Proceedings of the thirteenth annual symposium on Computational geometry*, pages 424–426, 1997.
- [18] Toru Ogawa, Yangyang Liu, Ryuzo Hasegawa, Miyuki Koshimura, and Hiroshi Fujita. Modulo based cnf encoding of cardinality constraints and its application to maxsat solvers. In *2013 IEEE 25th International Conference on Tools with Artificial Intelligence*, pages 9–17. IEEE, 2013.
- [19] Micha Sharir, Adam Sheffer, and Emo Welzl. On degrees in random triangulations of point sets. In *Proceedings of the twenty-sixth annual symposium on Computational geometry*, pages 297–306, 2010.
- [20] Carsten Sinz. Towards an optimal cnf encoding of boolean cardinality constraints. In *International conference on principles and practice of constraint programming*, pages 827–831. Springer, 2005.

A. Anhang

In diesem Kapitel werden alle Diagramme und Tabellen abgebildet, die für eine bessere Lesbarkeit im Text ausgelassen wurden.

A.1. Permutation

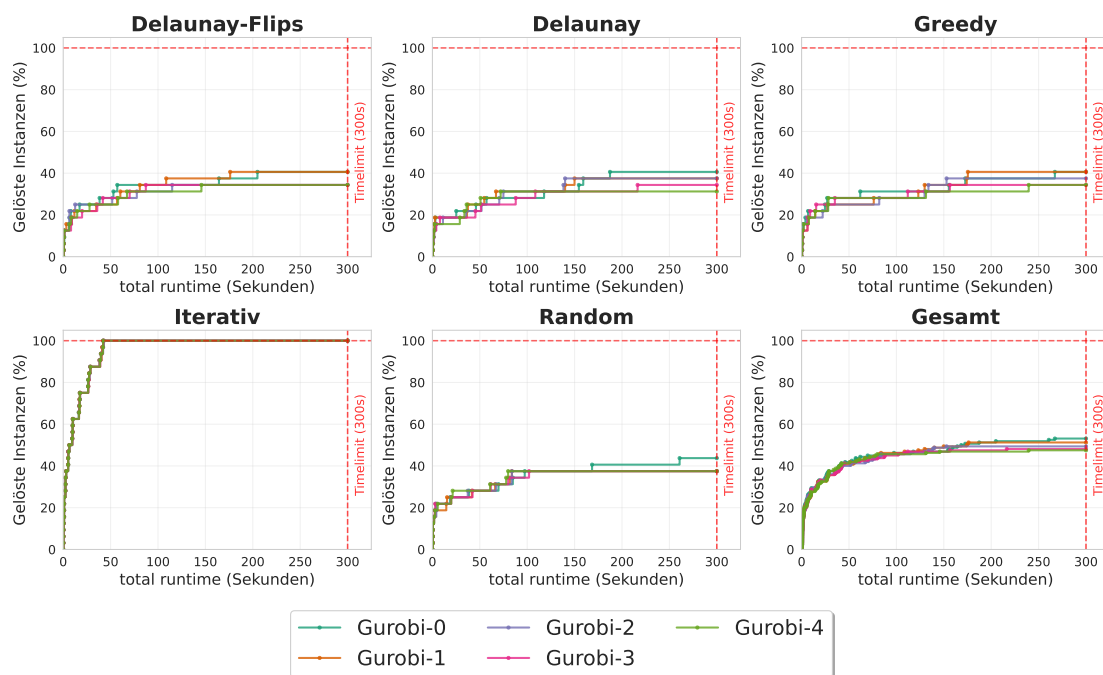


Abbildung A.1.: In diesem Diagramm wird untersucht, ob Gurobi bei verschiedenen Permutationen unterschiedliche Laufzeiten hat. Dazu wird jede Permutation als eine Zahl dargestellt. Es ist zu erkennen, dass vor allem bei Delaunay-Flips, Delaunay und Greedy die Linien verschiedene Laufzeiten haben, dementsprechend hat Gurobi bei verschiedenen Permutationen unterschiedliche Laufzeiten.

A. Anhang

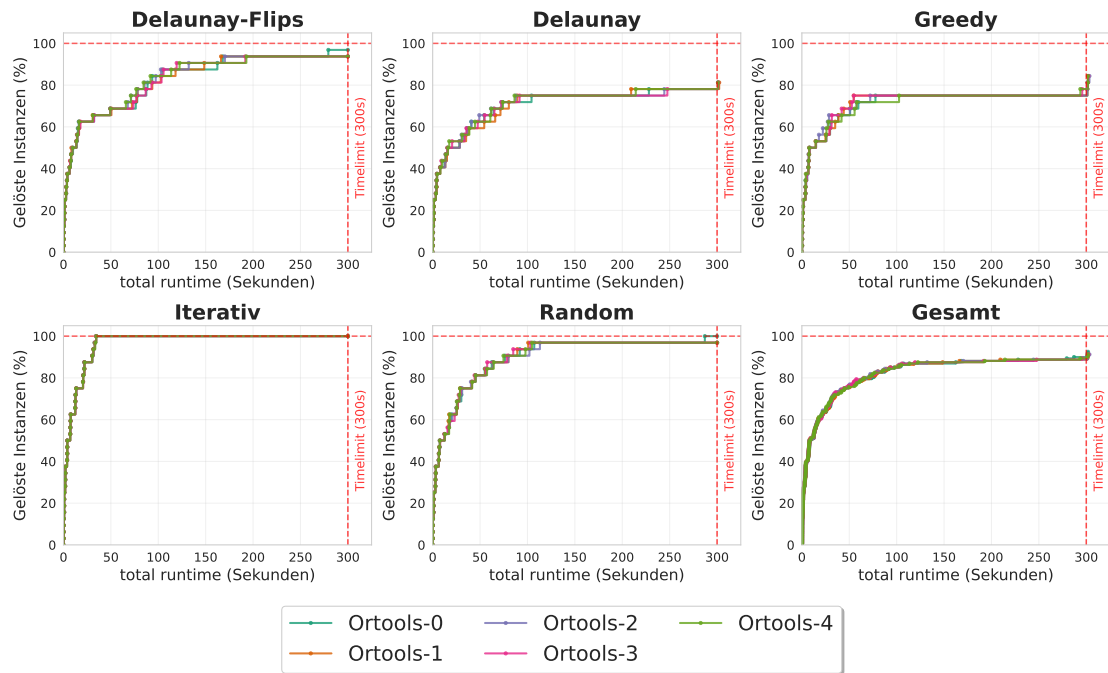


Abbildung A.2.: In diesem Diagramm wird untersucht, ob OR-Tools bei verschiedenen Permutationen unterschiedliche Laufzeiten hat. Dazu wird jede Permutation als eine Zahl dargestellt. Vor allem Delaunay-Flips hat verschiedene Laufzeiten bei den Permutationen. Dementsprechend hat OR-Tools bei verschiedenen Permutationen unterschiedliche Laufzeiten.

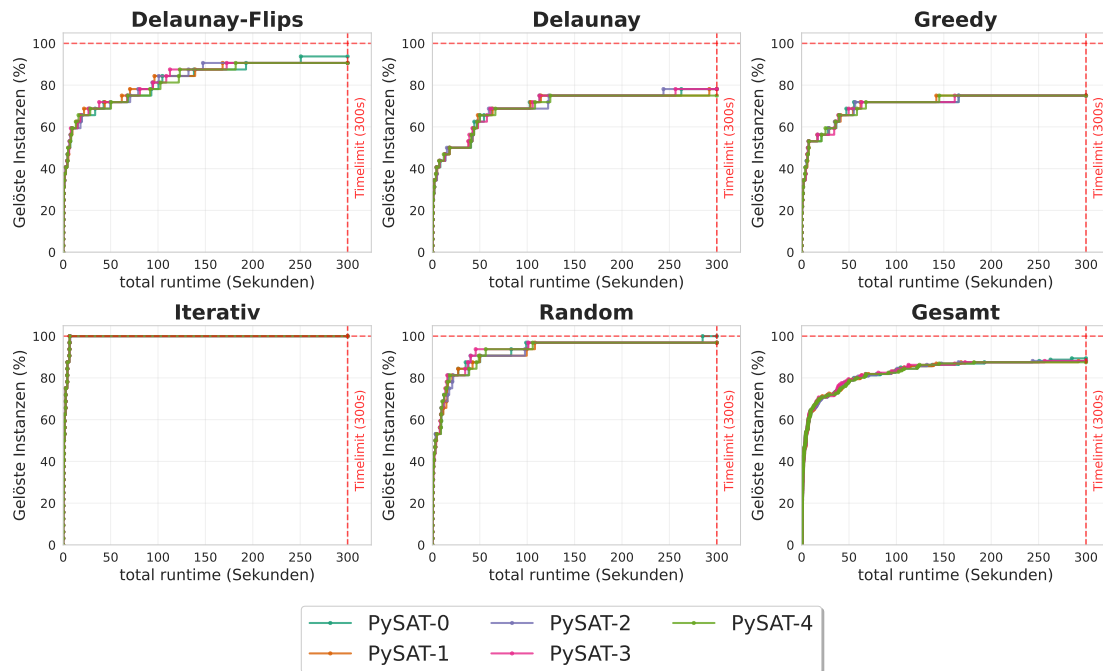


Abbildung A.3.: In diesem Diagramm wird untersucht, ob PySAT bei verschiedenen Permutationen unterschiedliche Laufzeiten hat. Dazu wird jede Permutation als eine Zahl dargestellt. Vor allem Delaunay-Flips hat verschiedene Laufzeiten bei den Permutationen. Dementsprechend hat PySAT bei verschiedenen Permutationen unterschiedliche Laufzeiten.

A.2. Mehrmalige Durchläufe

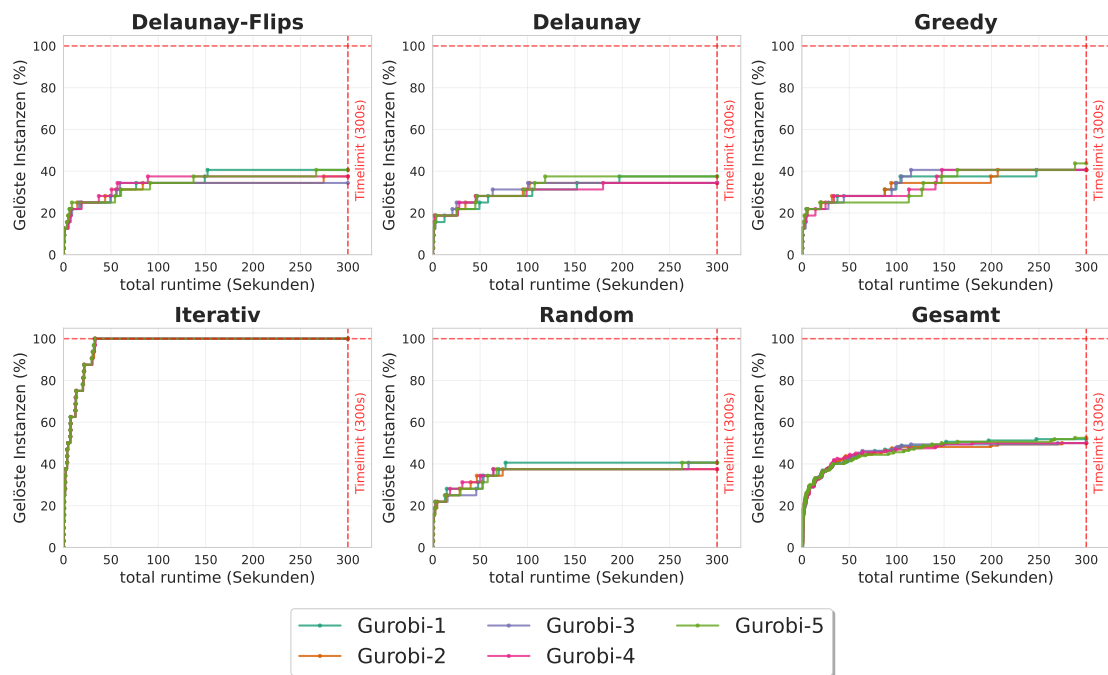


Abbildung A.4.: In diesem Diagramm wird untersucht, ob Gurobi bei den gleichen Argumenten und den gleichen unveränderten Instanzen unterschiedliche Laufzeiten hat. Dazu wird jeder Durchgang als eine Zahl dargestellt. Es ist zu erkennen, dass alle Instanzen außer Iterativ verschiedene Linien haben. Daraus lässt sich folgern, dass Gurobi bei gleichen Bedingungen unterschiedliche Ergebnisse liefert.

A.2. Mehrmalige Durchläufe

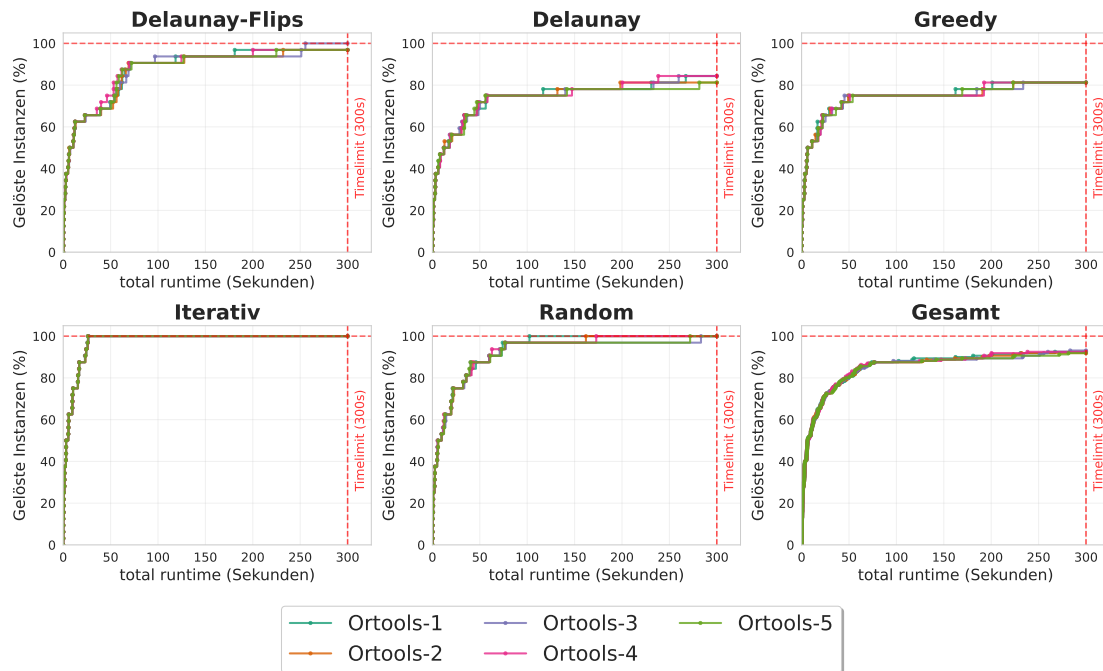


Abbildung A.5.: In diesem Diagramm wird untersucht ob OR-Tools bei den gleichen Argumenten und den gleichen unveränderten Instanzen unterschiedliche Laufzeiten hat. Dazu wird jeder Durchgang als eine Zahl dargestellt. Es fällt auf, dass vor allem bei Delaunay-Flips leicht unterschiedliche Linien sind. Daraus lässt sich folgern, dass OR-Tools bei gleichen Bedingungen unterschiedliche Ergebnisse liefert.

A. Anhang

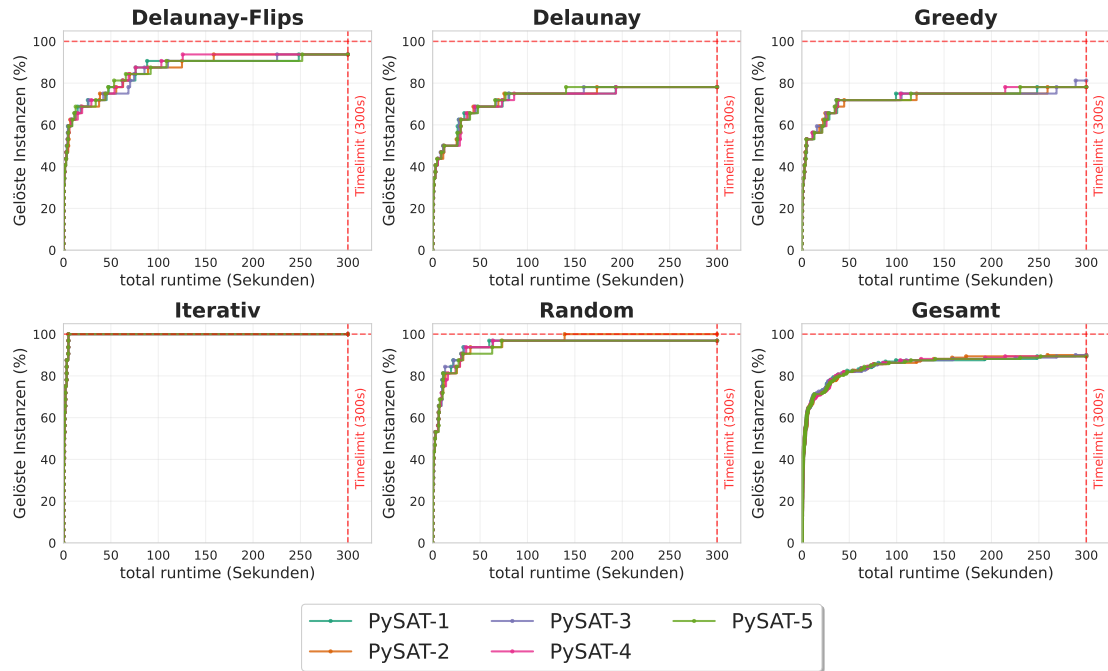
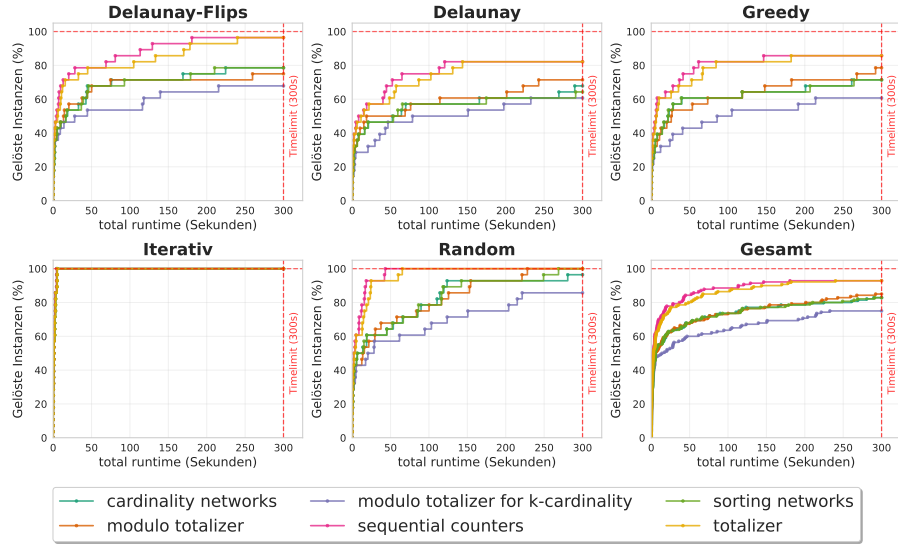


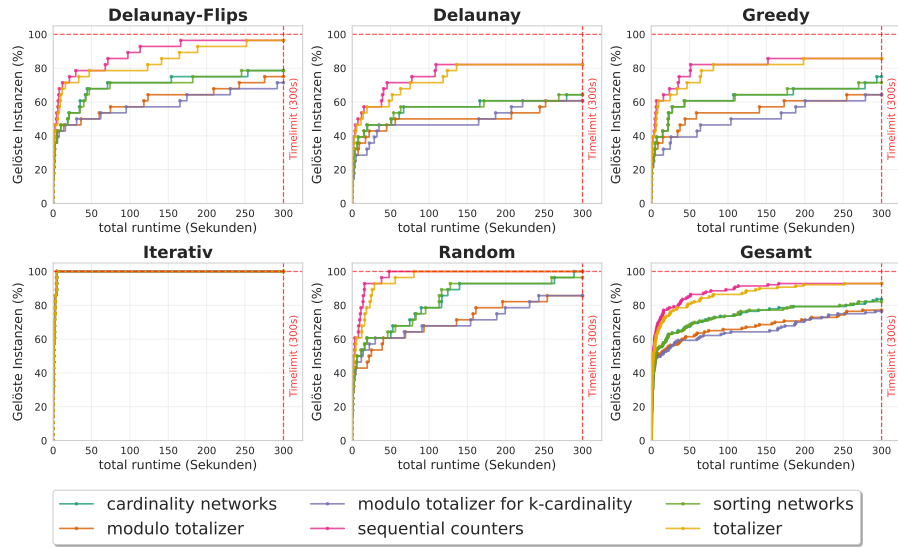
Abbildung A.6.: In diesem Diagramm wird untersucht ob PySAT bei den gleichen Argumenten und den gleichen unveränderten Instanzen unterschiedliche Laufzeiten hat. Dazu wird jeder Durchgang als eine Zahl dargestellt. Es ist zu sehen, dass vor allem bei Delaunay-Flips leicht unterschiedliche Linien sind. Daraus lässt sich folgern, dass PySAT bei gleichen Bedingungen unterschiedliche Ergebnisse liefert.

A.2. Mehrmalige Durchläufe

A.3. SAT Grad Auswertung



(a) Exakte Sollgrad: $\sum x_{e_i} = \deg(i) \forall i \in V$.



(b) Mindest Sollgrad: $\sum x_{e_i} \geq \deg(i) \forall i \in V$

Abbildung A.7.: In diesen Diagrammen werden verschiedene Codierungen für Kardinalitäten in einer KNF verglichen. Diese Kardinalitäten werden dazu genutzt, dass in einer KNF für jeden Knoten alle inzidenten Kanten gleich bzw. mindestens dem Sollgrad entsprechen müssen. Dabei wird zum einen der exakte Sollgrad (a) und zum anderen der minimale Sollgrad (b) betrachtet. In beiden Diagrammen ist klar zu erkennen, dass die *sequential counters* Methode am besten funktioniert, gefolgt von *totalizer*.

A.4. Vergleich der Modellierungstechniken

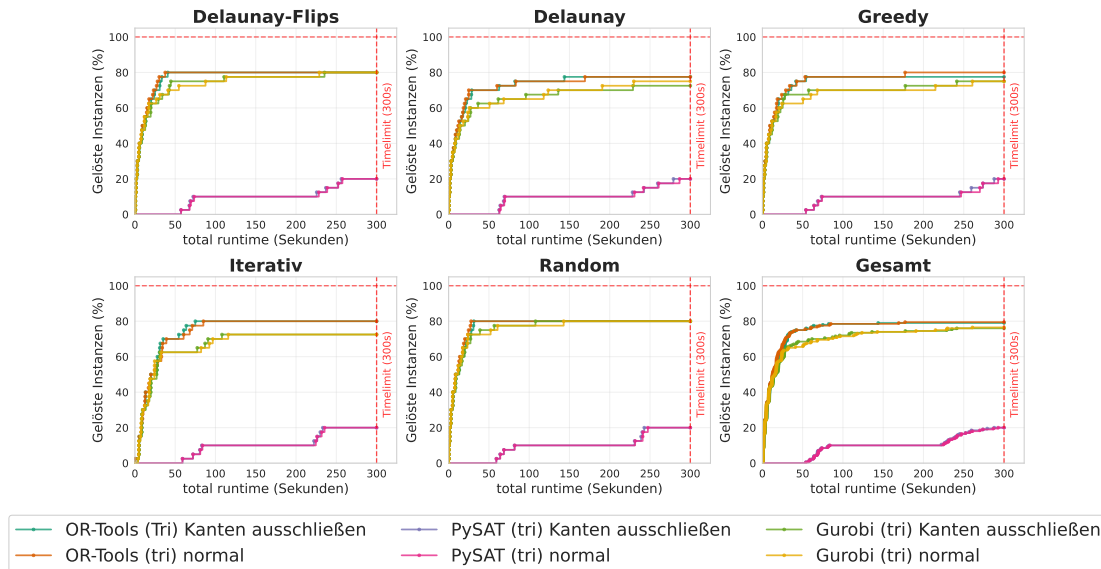


Abbildung A.8.: In diesem Experiment werden die drei Solver mit dem Dreiecks-Ansatz verglichen. Als einzige Verbesserung wurden Dreiecke ausgeschlossen. Es ist klar erkennbar, dass PySAT deutlich schlechter ist, als die anderen beiden. Das könnte darauf zurück führen sein, dass ein anderes Verfahren für die Überschneidungen genutzt wurde. Auch ist erkennbar, dass OR-Tools bessere Ergebnisse als Gurobi erzielt. Hingegen ist nicht erkennbar, ob das Ausschließen der Kanten eine Verbesserung bei den Solvern erzeugt. Bei Gurobi und OR-Tools sieht es eher so aus, dass keine Verbesserung erzeugt wird. Zudem ist erkennbar, dass OR-Tools bzw. Gurobi ab 75 % bzw. 65 % sehr schnell abflachen. Als bester Solver wird hier OR-Tools ohne Kantenausschließung ausgewählt.

A. Anhang

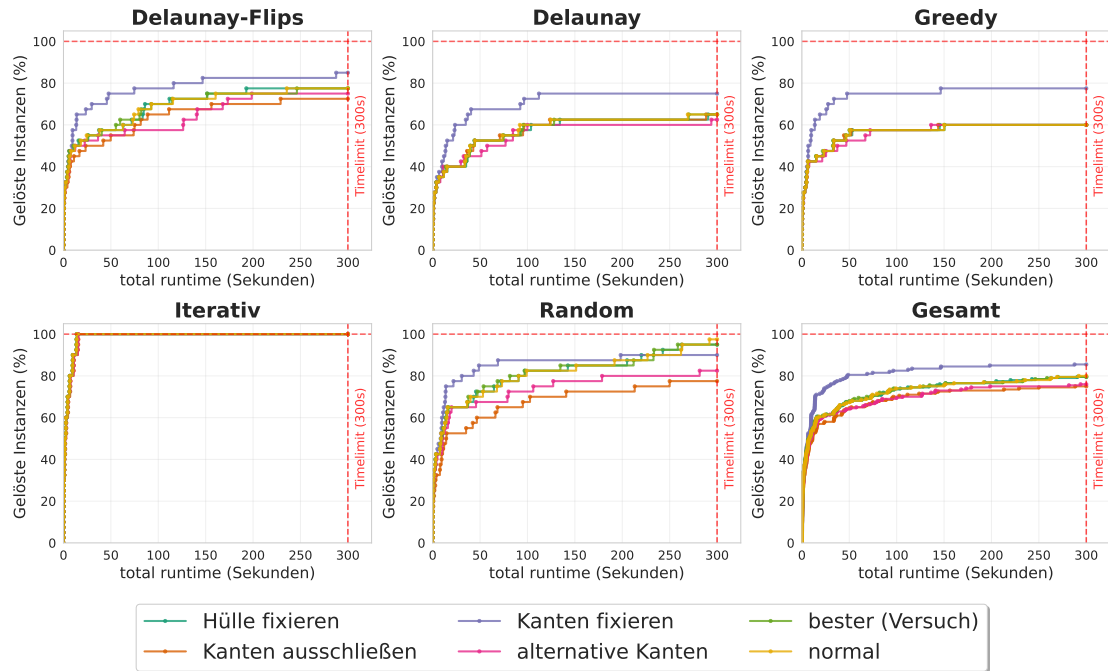


Abbildung A.9.: In diesem Experiment werden fünf verschiedene Optimierungsansätze für PySAT getestet. Hierbei wird untersucht, welche Optimierungen besser als der normale Solver sind. Es ist eindeutig erkennbar, dass das Fixieren der Kanten die größte Verbesserung bringt. Nur bei Random sind bei den höheren Instanzen der normale Solver und das Fixieren der Hülle besser. Es wurde probiert mit dem Fixieren der Hülle und der anderen Kanten einen besseren Solver zu erstellen. Wie zu sehen ist, ist allerdings das alleinige Fixieren der Kanten besser als dieser Versuch. Dabei fällt auf, dass der Versuch sehr ähnlich zum alleinigen fixieren der Hülle ist nur immer etwas länger benötigt. Dementsprechend wird für PySAT nur die Fixierung der Kanten, nicht allerdings die Fixierung der Hülle genutzt.

A.4. Vergleich der Modellierungstechniken

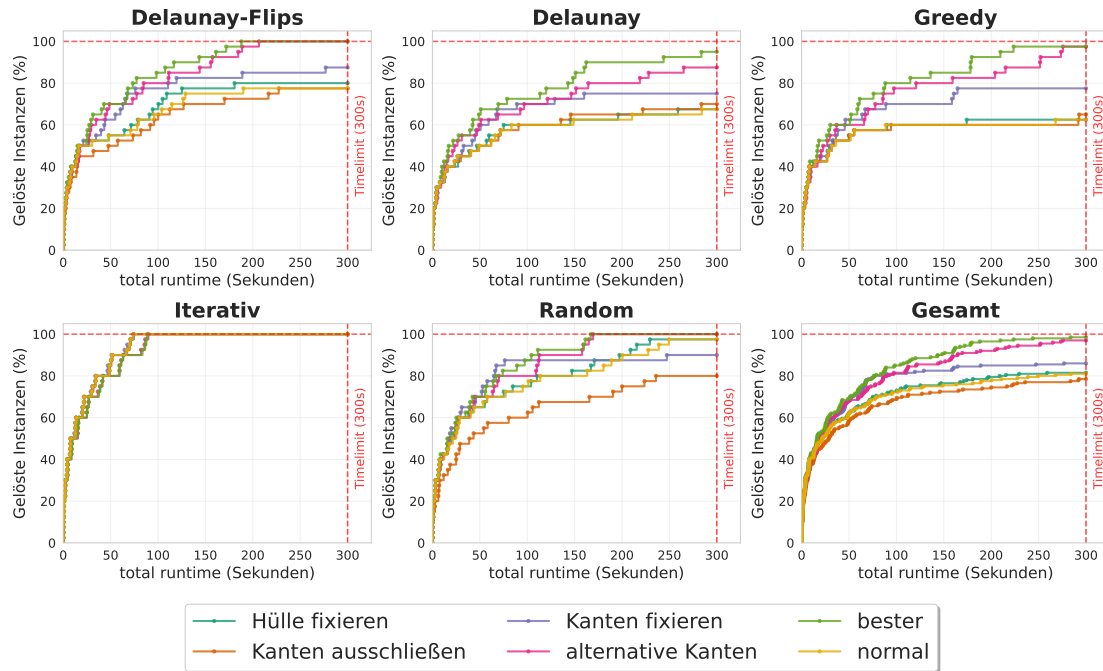


Abbildung A.10.: In diesem Experiment werden fünf verschiedene Optimierungsansätze für OR-Tools getestet. Hierbei wird untersucht, welche Optimierungen besser sind als der normale Ansatz. Es zeigt sich, dass Kanten auszuschließen deutlich langsamer als der normale Ansatz ist. Das Fixieren der Hülle liefert leichte Verbesserung gegenüber dem normalen Ansatz. Das Fixieren der Kanten und die alternative Kantenbedingung bringen einen deutlichen Vorteil gegenüber dem normalen Ansatz. Auffällig ist, dass bei Random das Fixieren der Kanten nach ungefähr 200 Sekunden wieder vom normalen Solver überholt wird. Um den besten Solver zu finden wurde eine Kombination aus Kanten und Hülle Fixieren sowie die alternative Kantenbedingung hinzugefügt. Wie im Bild zu erkennen, ist er abgesehen von der iterativen Instanz der beste Solver. Dementsprechend werden im Folgenden für OR-Tools als bester Solver alle drei Verbesserungen genutzt.

A. Anhang

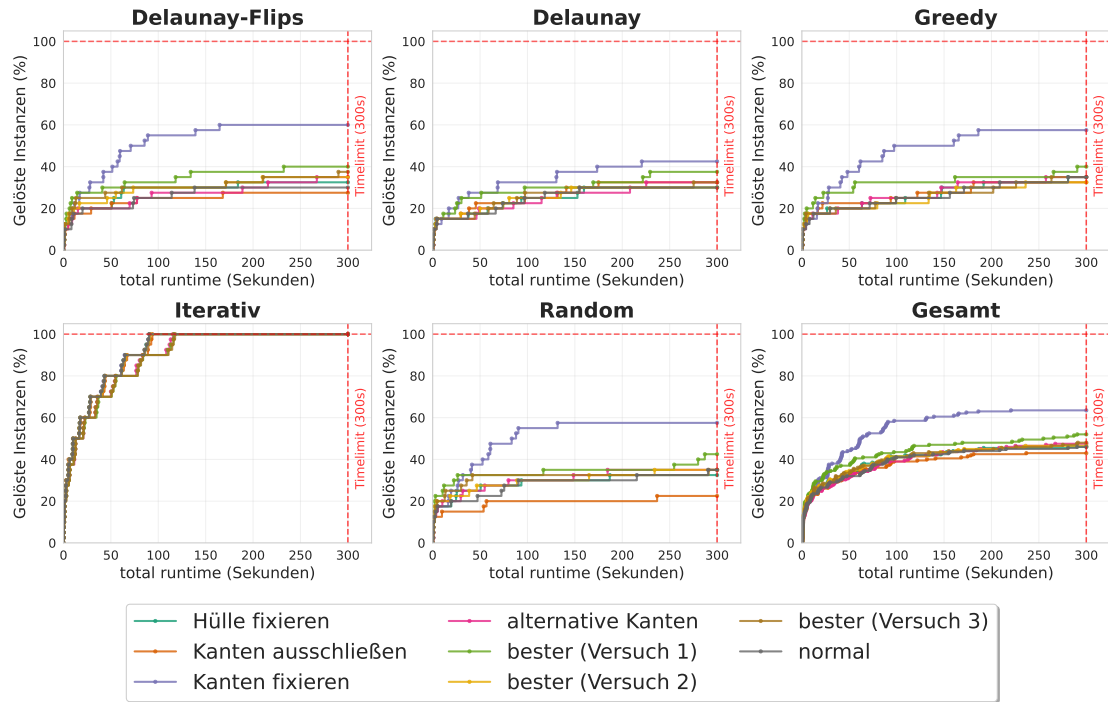


Abbildung A.11.: In diesem Experiment werden fünf verschiedene Optimierungsansätze für Gurobi getestet. Hierbei wird untersucht, welche Optimierungen besser sind als der normale Ansatz. Es ist zu erkennen, dass das Ausschließen von Kanten schlechter als der normale Ansatz ist. Das Fixieren der Hülle sowie die alternative Kantenbedingung liefern ähnliche Ergebnisse. Das Fixieren anderer Kanten bringt eine deutliche Verbesserung. Basierend auf dem Fixieren der Kanten wurde probiert mit den drei besseren Bedingungen einen besten Solver zu erstellen. Dafür wurden drei verschiedene Versuche erstellt. Der erste Versucht nutzt alle drei Bedingungen. Der zweite nur das Fixieren der Hülle sowie der Kanten. Der letzte und dritte Versuch fixiert Kanten und nutzt die alternative Kantenbedingung. Es ist zu erkennen, dass von allen drei Versuchen der erste die meiste Verbesserung bringt. Dieser ist besser als die meisten Solver. Die anderen beiden Versuche sind in der Nähe des normalen Solvers einzuordnen. Da alle versuchten Verbesserungen keine Globale Verbesserung gebracht haben, wird im Folgenden für Gurobi nur das alleinige Fixieren der Kanten genutzt. Es ist aber klar erkennbar, dass Gurobi keine guten Ergebnisse liefert. Bis auf bei den iterativen Instanzen werden meistens nicht die 50 % erreicht.

A.4. Vergleich der Modellierungstechniken

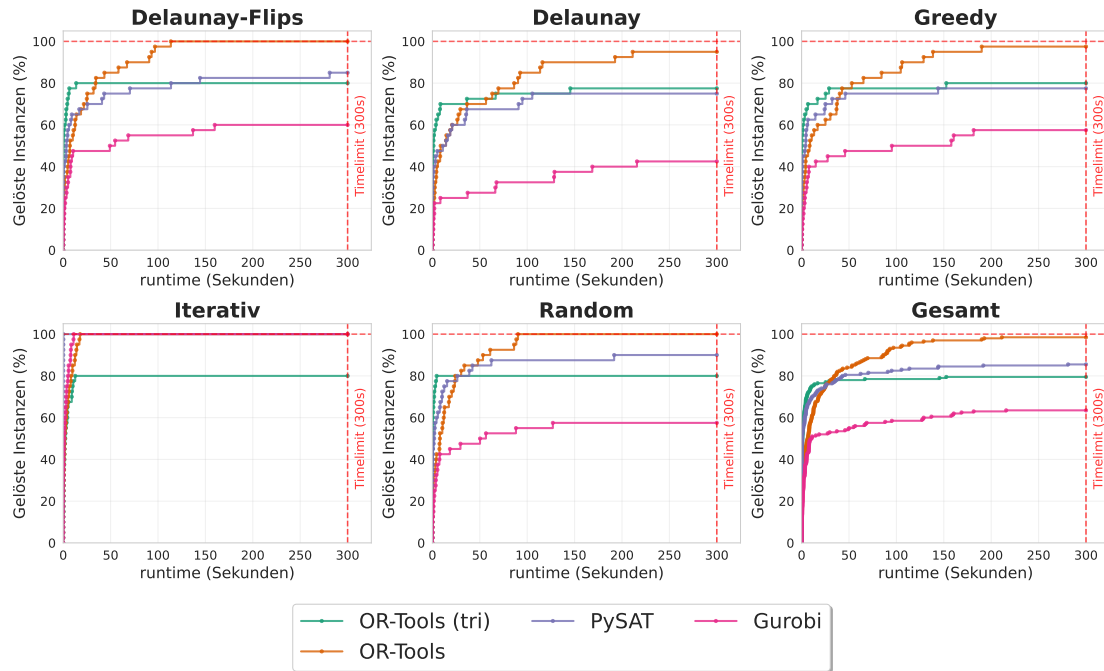


Abbildung A.12.: In diesem Experiment werden die Solverzeiten ohne die Vorbereitungszeiten betrachtet. Die restlichen Bedingungen wurden aus Abbildung 6.3 übernommen. Im Vergleich fällt auf, dass es keine großen Unterschiede macht. Es sind immer noch die gleichen Ergebnisse sichtbar. Der Unterschied liegt allein darin, dass alle Graphen etwas nach links verschoben wurden. Dementsprechend kann davon ausgegangen werden, dass die Vorbereitungszeit keinen Einfluss auf die Ergebnisse hat.

A.5. Theoretische Fixierung von Kanten

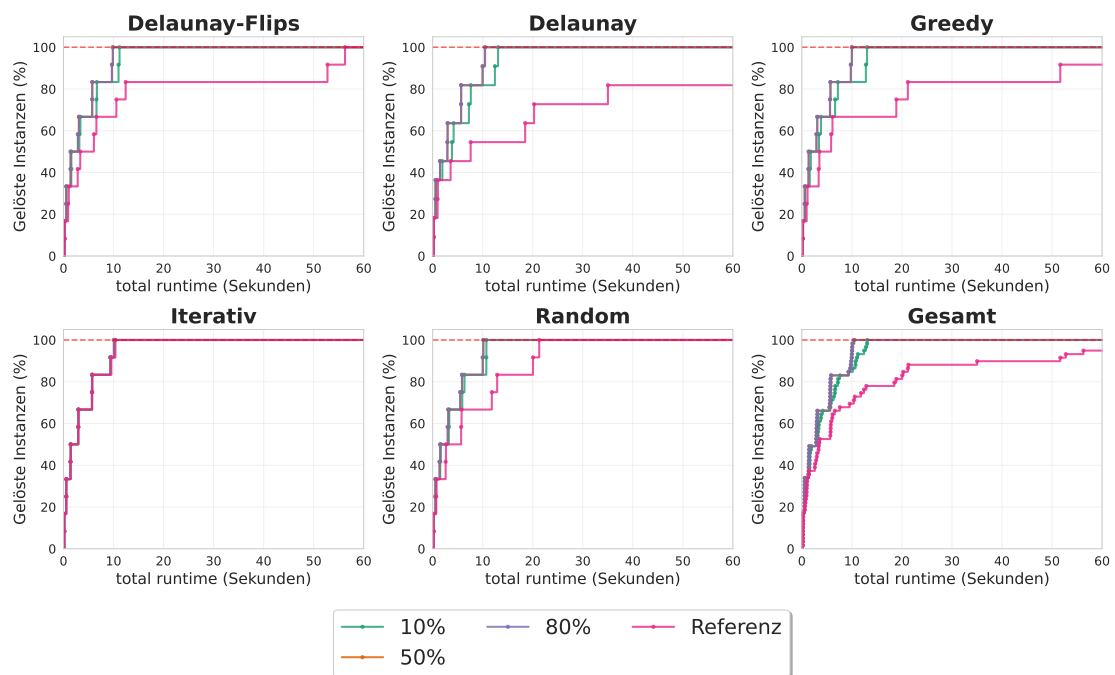


Abbildung A.13.: In diesem Experiment wird die theoretische Fixierung von Kanten untersucht. Für das Experiment wurde OR-Tools mit Grad- und Überschneidungsbeschränkung als Referenz und zusätzlich OR-Tools mit 10 %, 50 % und 80 % der benötigten fixierten Kanten verwendet. In dem Diagramm wurde der betrachtete Zeitraum auf 60 Sekunden begrenzt, da hier alle relevanten Informationen zu sehen sind. Es ist klar zu erkennen, dass der normale Solver mit Abstand am langsamsten ist. Wie zu erwarten, ist der Solver, bei dem nur 10 % der Kanten fixiert wurden, langsamer als die beiden anderen. Überraschend ist, dass es bei keiner Instanz einen Unterschied macht, ob 50 oder 80 % der Kanten fixiert wurden. Diese beiden Solver sind genau gleich schnell.

A.6. Größerer Timeout

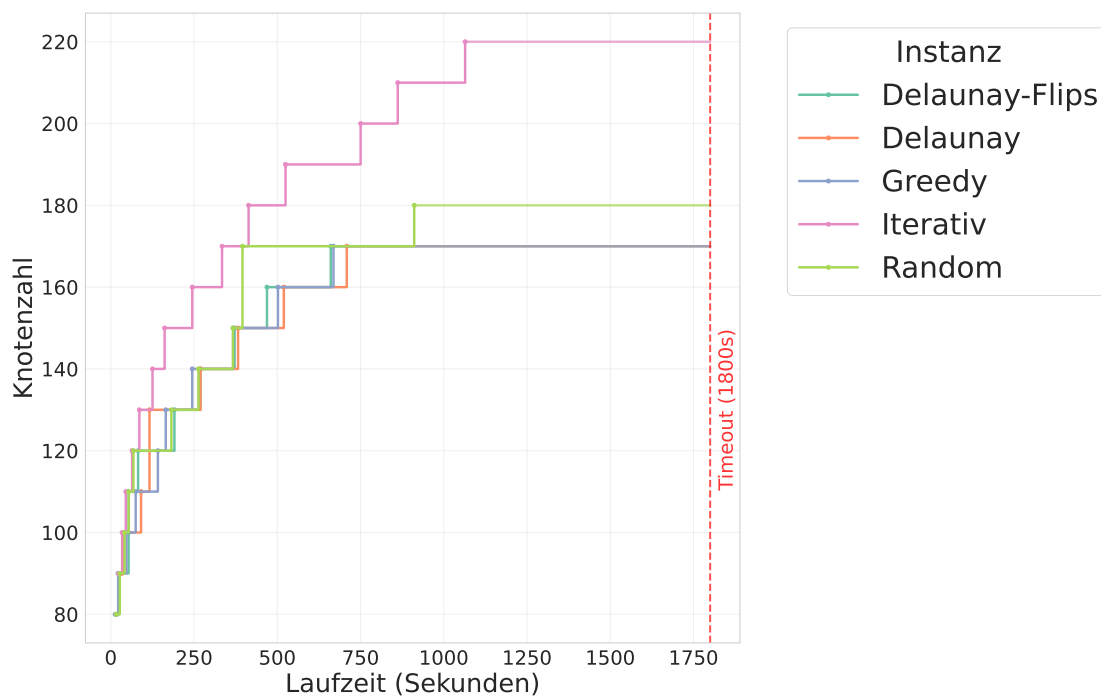


Abbildung A.14.: In diesem Experiment wurde das Zeitlimit von 5 Minuten auf 30 Minuten erhöht. Auf der Y-Achse wird in diesem Diagramm die Anzahl der Knoten angezeigt. Das bedeutet, sobald der Solver die erste Instanz der Knotengröße löst, wird die Einheit zu dieser Knotengröße gesetzt. Es ist zu erkennen, dass der Solver Knotengrößen bis zu 220 Knoten lösen konnte, allerdings nur für iterative Instanzen. Die meisten Instanzen schaffen nur 170 Knoten. Es ist zu erkennen, dass sich die Instanzen in verschiedene Kategorien einordnen lassen. So ist Iterativ am einfachsten, Random in der Mitte und Delaunay-Flips, Greedy und Delaunay am schwersten. Generell lässt sich aber sagen, dass bei vielen Durchläufen wegen des fehlenden Arbeitsspeichers kein Ergebnis gefunden wurde und nicht wegen des Timeouts.

A.7. Ja/Nein getrennt

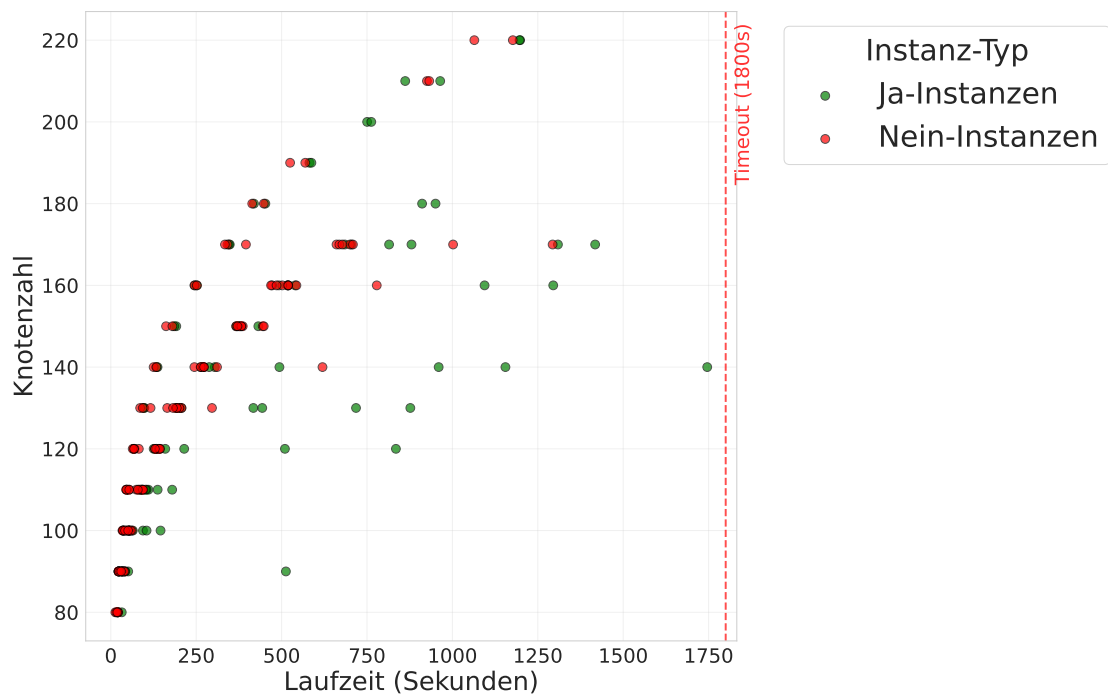


Abbildung A.15.: In diesem Experiment wurden die Daten aus Abschnitt 6.8 genutzt um zu untersuchen, ob der Solver bei Ja-Instanzen oder Nein-Instanzen schneller ist. In dem Diagramm wird nach Ja-Instanzen und Nein-Instanzen gruppiert. Das bedeutet, für jede Knotengröße wurden für Ja und Nein jeweils 10 Durchläufe mit verschiedenen Instanzenarten gemacht. Auf der X-Achse wird hierbei die Zeit angezeigt, wann die Lösung gefunden wurde, auf der Y-Achse für welche Knotenzahl die Lösung gefunden wurde. Es ist zu erkennen, dass beide bis zu 220 Knoten schaffen, allerdings ab 180 Knoten nicht wirklich viele Lösungen liefern. Generell werden Nein-Instanzen deutlich schneller gelöst als Ja-Instanzen. Interessant ist auch das für 200 Knoten keine Nein-Instanz gelöst wurde.

A.8. Zielfunktion

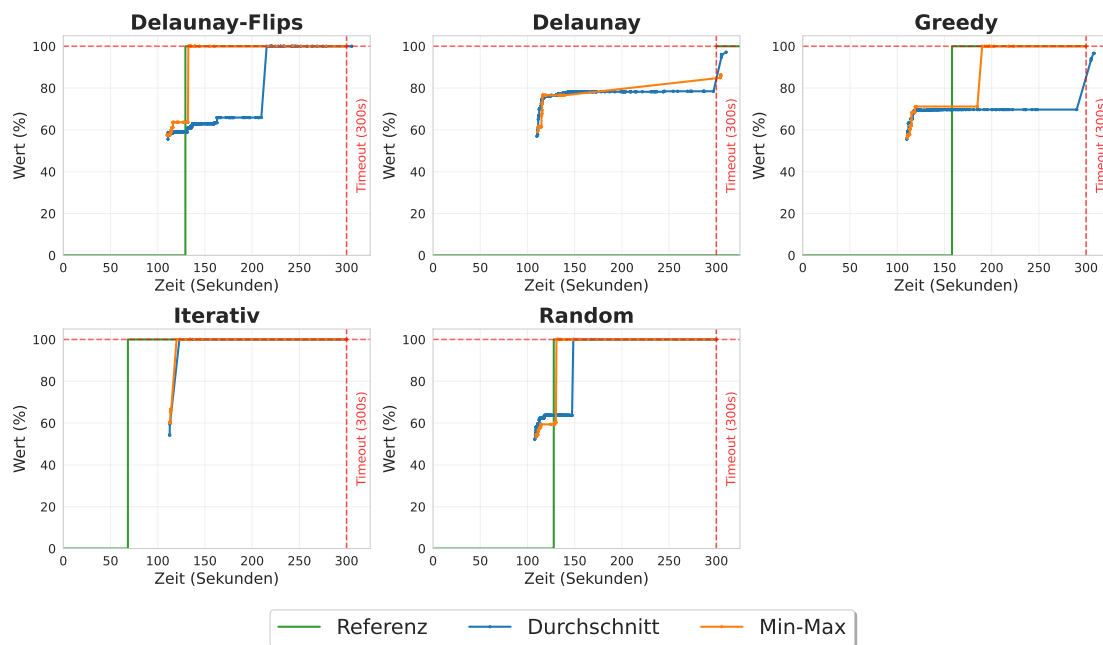


Abbildung A.16.: In diesem Experiment wird eine Greedy Ja-Instanz mit 120 Knoten als Optimierungsproblem betrachtet. Als Referenz wird der Solver aus Abschnitt 6.6 verwendet. Dieser betrachtet die Instanz als Entscheidungsproblem. Es ist zu sehen, dass der Min-Max-Ansatz in den meisten Fällen bessere Ergebnisse erzielt als der Durchschnittsansatz. Nur bei Delaunay erreichen beide Ansätze nicht die vollen 100 %. Der Durchschnittsansatz erzielt bei Delaunay jedoch höhere Werte als Min-Max. Bei Delaunay liefert der Referenzsolver kein Ergebnis. Ein klares Muster, welcher Ansatz insgesamt am besten abschneidet, ist nicht zu erkennen, da in jeder Instanz unterschiedliche Ansätze bessere Ergebnisse liefern. Bei Delaunay erzielt der Durchschnittsansatz die besten Resultate. Der Referenzsolver ist bei Iterativ und Greedy am schnellsten. Min-Max ist fast immer schneller als der Durchschnittsansatz und in drei Fällen mindestens genauso schnell wie der Referenzsolver.

A.9. Anzahl Lösungen

Knoten Anzahl	Delaunay Flips	Delaunay	Greedy	Iterativ	Random
30_1	1	1	1	1	1
30_2	1	1	1	1	1
40_1	1	1	1	1	1
40_2	1	1	1	1	1
50_1	1	2	1	1	1
50_2	1	1	1	1	1
60_1	1	1	1	1	1
60_2	1	1	1	1	1
70_1	1	1	1	1	1
70_2	1	1	1	1	1
80_1	1	1	1	1	1
80_2	1	1	1	1	1
90_1	1	-1	1	1	1
90_2	1	-1	1	1	1
100_1	1	-1	-1	1	1
100_2	1	-1	-1	1	1
110_1	-1	-1	-1	1	1
110_2	-1	-1	-1	1	1
120_1	-1	-1	-1	1	1
120_2	-1	-1	-1	1	1

Tabelle A.1.: In dieser Tabelle werden die Anzahl der Lösungen für die verschiedenen Ansätze dargestellt. In der ersten Spalte werden die Anzahl der Knoten der Instanz angegeben. Hierbei wurde jede Knotengröße zwei mal getestet. In den anderen Spalten sind jeweils die Anzahl der Lösungen für eine Instanz gegeben. Der Wert -1 bedeutet, dass nach einer Stunde ein Timeout erreicht wurde und die Anzahl der Lösungen somit nicht eindeutig festgestellt werden konnte. Auffällig ist, dass es nur eine einzige Instanz gibt, die zwei Lösungen besitzt.